Web Hosting Management System (WHMS)

By:

Hammad bin Iftikhar

45388

Muhammad Khalis

46263

Supervised by:

Mr. Muhammad Usman

Faculty of Computing

Riphah International University, Islamabad

Spring 2026

A Dissertation Submitted To

Faculty of Computing,

Riphah International University, Islamabad

As a Partial Fulfillment of the Requirement for the Award

of the Degree of

Bachelor's of Science in Software Engineering

Faculty of Computing

Riphah International University, Islamabad

Date: s

Final Approval

This is to certify that we have read the report submitted by **Hammad bin Iftikhar 45388 and Muhammad Khalis 46263**, for the partial fulfillment of the requirements for the degree of the Bachelor's of Science in Software Engineering (BSSE). It is our judgment that this report is of sufficient standard to warrant its acceptance by Riphah International University, Islamabad for the degree of Bachelor's of Science in Software Engineering (BSSE).

Committee:

1

_____

Mr. Muhammad Usman
(Supervisor)

2

_____

Dr. Musharraf Ahmad
(Head of Department)

Declaration

We hereby declare that this document "**Web Hosting Management System (WHMS)**" neither as a whole nor as a part has been copied out from any source. It is further declared that we have done this project with the accompanying report entirely based on our personal efforts, under the proficient guidance of our teachers, especially our supervisor **Mr. Muhammad Usman**. If any part of the system is proved to be copied from any source or found to be reproduction of any project from anywhere else, we shall stand by the consequences.

Hammad bin Iftikhar

45388

Muhammad Khalis

46263

Dedication

We dedicate this dissertation to our beloved parents, whose endless love, prayers, and sacrifices have been our greatest source of strength and inspiration throughout our lives. Their unwavering belief in our potential has fueled our journey to this milestone.

Finally, We thank our respected supervisor, Mr. Muhammad Usman . who guided us with patience and imparted the knowledge that made this project possible.

Acknowledgement

First of all, we are obliged to Allah Almighty the Merciful, the Beneficent and the source of all Knowledge, for granting us the courage and knowledge to complete this Project.

Hammad bin Iftikhar

45388

Muhammad Khalis

46263

Abstract

Table of Contents

Chapter 1 2
INTRODUCTION 2
1.1 OPPORTUNITY AND STAKEHOLDERS 2
1.2 MOTIVATION AND CHALLENGES 2
1.3 GOALS AND MOTIVATION 3
1.4 SCOPE AND LIMITATIONS 3
1.5 SOLUTION OVERVIEW 4
1.6 REPORT OUTLINE 4
Chapter 2 7
LITERATURE / MARKET SURVEY 7
2.1 INTRODUCTION 7
2.2 LITERATURE OVERVIEW/TECHNOLOGIES OVERVIEW 7
2.3 BRAINSTORMING 9
2.4 EXISTNG SYSTEMS 9
2.5 SUMMARY 11
Chapter 3 13
REQUIREMENT ENGINEERING 13
3.1 INTRODUCTION 13
3.2 PROBLEM SCENARIO 13
3.3 ELICITATION TECHNIQUES 16
3.4 FUNCTIONAL REQUIREMENTS 18
3.5 NON-FUNCTIONAL REQUIREMENTS 32
3.6 FR ↔ API ENDPOINT MAPPING 32
3.7 REQUIREMENT TRACEABILITY MATRIX (RTM) (IEEE-29148 COMPLIANT) 47
3.8 (SQA) – DEFECT DETECTION 66
Chapter 4 68
System Design 68
4.1 INTRODUCTION 68
4.2 ARCHITECTURE DIAGRAM 69
4.3 DETAILED DESIGN 70
4.4 DATABASE DESIGN 128
4.5 SOFTWARE QUALITY ASSURANCE (SQA) – DESIGN LEVEL DEFECT DETECTION 133

Chapter 5 139
Implementation 139
5.1 INTRODUCTION 139
5.2 DEVELOPMENT PROCESS AND TEAM COLLABORATION 139
5.3 TECHNOLOGY STACK AND TOOLS 140
5.4 OVERALL SYSTEM IMPLEMENTATION ARCHITECTURE 141
5.5 CORE EXECUTON FLOW 142
5.6 MODULE WISE IMPLEMENTATION DETAILS 143
5.7 API DESIGN AND IMPLEMENTATION 144
5.8 DATA PERSISTENCE AND DATABASE DESIGN 144
5.9 SECURTY CONSIDERATIONS IN IMPLEMENTATION 145
5.10 BEST PRACTICES AND CODING STANDARDS 146
5.11 LIMITATIONS AND DESIGN TRADE-OFFS 148
5.13 SUMMARY OF IMPLEMENTATION 152
Chapter 6 154
Testing & Evaluation 154
6.1 Auth Module 154
Chapter 7 184
Conclusion & Outlook 184
7.1 184
7.2 184
7.3 184
7.4 184
References 191

List of Tables

Table 2.1: Comparative Analysis of Existing Systems 9
Table 3.2.1 PS-01 Legacy Architecture & Limited Customization 13
3.2.2 PS-02 Fragile Third-Party Integrations 13
3.3.3 PS-03 Rigid UI/UX and Limited Customization 14
3.3.4 PS-04 Unstandardized Plugin Ecosystem 14
3.3.5 PS-05 Scalability & Performance Limitations 15
3.3.6 PS-06 Security & Update Cadence Limitations 15
3.4.1&3.4.2 Module FR ↔ API ENDPOINT MAPPING 32
3.4.3&3.4.5 Module FR ↔ API ENDPOINT MAPPING 33
3.4.6&3.4.7 Module FR ↔ API ENDPOINT MAPPING 33
3.4.8 & 3.4.9 & 3.4.10 Module FR ↔ API ENDPOINT MAPPING 34
3.4.10 & 3.4.11 & 3.4.12 Module FR ↔ API ENDPOINT MAPPING 35
3.4.13 & 3.4.14 & 3.4.15 Module FR ↔ API ENDPOINT MAPPING 36
3.4.16 & 3.4.17 & 3.4.18 Module FR ↔ API ENDPOINT MAPPING 38
3.4.19 & 3.4.20 & 3.4.21 Module FR ↔ API ENDPOINT MAPPING 40
3.4.22 & 3.23.9 & 3.4.24 Module FR ↔ API ENDPOINT MAPPING 43
3.4.25 & 3.4.26 & 3.4.28 Module FR ↔ API ENDPOINT MAPPING 45
Table 4.3.3.1.: Fully Dressed Use Case – UC-AUTH-01 76
Table 4.3.3.2: Fully Dressed Use Case – UC-AUTH-02 78
Table 4.3.3.3.: Fully Dressed Use Case – UC-PASS-01 80
Table 4.3.3.4.: Fully Dressed Use Case – UC-ADMIN-01 82
Table 4.3.3.5.: Fully Dressed Use Case – UC-IMP-01 84
Table 4.3.3.6.: Fully Dressed Use Case – UC-MFA-01 88
Table 4.3.3.7.: Fully Dressed Use Case – UC-API-01 90
Table 4.3.3.8.: Fully Dressed Use Case – UC-IP-01 93
Table 4.3.3.9.: Fully Dressed Use Case – UC-BACKUP-02 95
Table 4.3.3.10.: Fully Dressed Use Case – UC-BACKUP-03 98
Table 4.3.3.11.: Fully Dressed Use Case – UC-BACKUP-04 100
Table 4.3.3.12.: Fully Dressed Use Case – UC-AUTO-01 102
Table 4.3.3.13.: Fully Dressed Use Case – UC-AUTO-02 104
Table 4.3.3.14.: Fully Dressed Use Case – UC-AUTO-03 106
Table 4.3.3.15.: Fully Dressed Use Case – UC-AUTO-04 107
Table 4.3.3.16.: Fully Dressed Use Case – UC-AUTO-05 108
Table 4.3.3.17.: Fully Dressed Use Case – UC-PLUG-01 110

Table 4.3.3.17.: Fully Dressed Use Case – UC-PLUG-02 111
Table 4.3.3.18.: Fully Dressed Use Case – UC-PLUG-03 112
Table 4.3.3.19.: Fully Dressed Use Case – UC-PLUG-04 113
Table 4.3.3.20: Fully Dressed Use Case – UC-AUTO-01 114
Table 4.3.3.21: Fully Dressed Use Case – UC-AUTO-02 115
Table 4.3.3.22: Fully Dressed Use Case – UC-AUTO-03 116
4.5.2 Design-Level Test Cases (Architecture & Security) 134
4.5.3 Design-Level Test Cases (Workflow & interaction) 135
4.5.4 Design-Level Test Cases (Verification) 135
5.12.2 White-Box Test Cases – Authentication & Security 149
5.12.3 White-Box Test Cases – Impersonation 150
5.12.4 White-Box Test Cases – Plugin Execution 150
5.13.5 White-Box Test Cases – Automation 151
Auth Module API Test Cases (Tabular Summary) 154
Auth Module Test Cases 155
Email Verification Test Cases 158

List of Figures

Fig 4.1: Architecture Diagram 69
Fig 4.2: Use Case Diagram 70
Fig 4.3: Use Case Diagram 71
Fig 4.4: Use Case Diagram 72
Fig 4.5: System Sequence Diagram 117
Fig 4.6: System Sequence Diagram 118
Fig 4.7: System Sequence Diagram 119
Fig 4.8: System Sequence Diagram 119
Fig 4.9: System Sequence Diagram 120
Fig 4.10 System Sequence Diagram 121
Fig 4.11 System Sequence Diagram 121
Fig 4.12 System Sequence Diagram 121
Fig 4.13 System Sequence Diagram 122
Fig 4.14 System Sequence Diagram 122
Fig 4.15 System Sequence Diagram 123
Fig 4.16 System Sequence Diagram 123
Fig 4.17 System Sequence Diagram 125
Fig 4.18 System Sequence Diagram 126
Fig 4.19 System Sequence Diagram 127

Chapter 1:

Introduction

Chapter 1

INTRODUCTION

1.1 OPPORTUNITY AND STAKEHOLDERS

The rapid growth of online businesses has intensified the demand for robust, flexible, and cost-effective client management and automation solutions in the Web hosting Industry. While industry standards like WHMCS are widely used, they suffer legacy architectural limitations, poor developer experience, and costly licensing models. This presents a significant opportunity to design a next-generation, JavaScript-based platform that leverages modern backend and frontend frameworks for improved scalability, maintainability, and extensibility.

1.1.1 Stakeholders

The proposed web hosting management and automation platform targets the following primary stakeholders:

1. Web hosting companies
2. Resellers
3. End customers (hosting clients)

1.2 MOTIVATION AND CHALLENGES

1.2.1 Motivation

Modern web hosting management platforms are predominantly built on legacy, tightly coupled architectures that have accumulated significant technical debt over time. These systems often limit extensibility, increase maintenance overhead, and constrain architectural evolution. As hosting providers scale, such limitations negatively impact adaptability, security, and long-term sustainability.

The motivation for this project is to re-architect a Web Hosting Management System (WHMS) using modern software engineering principles, focusing on modularity, maintainability, and future scalability rather than feature inflation. The project aims to demonstrate how contemporary technology stack and clean architectural boundaries can reduce technical debt and improve system evolution.

From an academic perspective, this project serves as a comprehensive case study in large-scale system design, enabling practical exposure to architectural decision-

making, modular decomposition, and extensibility strategies within a real-world problem domain.

1.2.2 Challenges

The project does not aim to fully replicate the breadth of features or integrations provided by commercial enterprise platforms. Infrastructure-level provisioning and orchestration are abstracted or simulated to maintain focus on architectural design. Advanced distributed database techniques, such as sharding or geo-replication, are outside the immediate project scope and are identified as future enhancements.

1.3 GOALS AND MOTIVATION

The primary goal of this project is to design and implement a modular, microservices-ready architecture using a modular monolithic deployment using modern software engineering principles. A key objective is the adoption of an API-first architecture, in which all core system capabilities are defined, implemented, and validated through well-documented APIs before user interface concerns are addressed. This approach ensures a clear separation of concerns between backend services and frontend clients, enabling independent development, testing, and evolution of system components.

Additional objectives include delivering essential hosting management functionalities such as client management, automated billing, domain handling, support ticketing, automation workflows, and role-based access control (RBAC). The system is designed to support horizontal scalability at the application layer through stateless service interactions, while maintaining clear module boundaries that allow individual components to be extracted into independent microservices in future iterations. Collectively, these goals aim to produce a maintainable, extensible, and future-ready system that balances academic feasibility with industry relevance.

1.4 SCOPE AND LIMITATIONS

1.4.1 Scope

The scope of this project includes the architectural design and implementation of a functional WHMS with a strong emphasis on modularity, scalability, readiness, and security. The system focuses on management and orchestration of hosting-related operations, exposing all core functionality through APIs to support web interfaces, automation engines, and potential third-party integrations. Integration points for

external services, such as payment gateways and domain registrars, are designed to be extensible and loosely coupled.

1.4.2 Limitations

The project does not aim to fully replicate the breadth of features or integrations provided by commercial enterprise platforms. Infrastructure-level provisioning and orchestration are abstracted or simulated to maintain focus on architectural design. Advanced distributed database techniques, such as sharding or geo-replication, are outside the immediate project scope and are identified as future enhancements.

1.5 SOLUTION OVERVIEW

The proposed solution is a modular monolithic Web Hosting Management System structured around clearly defined functional modules, each responsible for a specific business capability. The system follows an API-first design, where all interactions between modules, frontend components, and external integrations occur through well-defined APIs. This design enables loose coupling, improves testability, and supports future architectural evolution.

The backend is implemented using Node.js and Express, providing asynchronous request handling and scalability at the application layer. The frontend is developed using Next.js (TypeScript) to deliver a responsive and modern user interface that consumes backend APIs. Role-Based Access Control (RBAC) is enforced consistently across the system to support multiple user roles and granular permissions. The architecture includes provisions for multi-tenancy, audit logging, and secure data handling, along with a plugin engine that enables controlled extensibility without modification of core system logic.

1.6 REPORT OUTLINE

This report is structured as follows:

Chapter 1 – Introduction: Introduces the project's background, motivation, objectives, scope, and solution overview.

Chapter 2 – Literature and Market Survey: Reviews existing WHMS platforms, relevant technologies, and architectural approaches.

Chapter 3 – System Analysis and Requirements: Defines functional and non-functional requirements.

Chapter 4 – System Design: Describes the overall architecture, module decomposition, data design, and security mechanisms.

Chapter 5 – Implementation: Details the technologies used, implementation strategies, and key design decisions.

Chapter 6 – Testing and Evaluation: Presents testing methodologies, test cases, and system evaluation results.

Chapter 7 – Conclusion and Future Work: Summarizes the project outcomes and outlines potential future enhancements.

Chapter 2:

Literature / Market Survey

Chapter 2

LITERATURE / MARKET SURVEY

2.1 INTRODUCTION

Web hosting management systems play a critical role in automating operational, financial, and administrative tasks for hosting providers. Over the last two decades, the demand for such systems has increased `significantly due to the growth of shared hosting, reseller businesses, and cloud-based services. These platforms are expected to handle automated billing, client lifecycle management, service provisioning, and secure access control in a reliable and scalable manner.

This chapter critically reviews existing web hosting management solutions from both academic and industry perspectives. It evaluates their architectural approaches, technological foundations, and functional capabilities, while identifying inherent limitations that affect extensibility, scalability, and long-term maintainability. The analysis aims to establish a clear rationale for developing a modern WHMCS alternative based on an API-first, modular monolithic architecture using a full JavaScript stack.

2.2 LITERATURE OVERVIEW/TECHNOLOGIES OVERVIEW

2.2.1 Historical Context

Early web hosting management solutions were primarily developed using procedural PHP and tightly coupled architectures. At the time, such designs were suitable due to lower system complexity and limited scalability requirements. However, as hosting providers expanded their offerings and customer bases, these systems accumulated significant technical debt, making them difficult to extend or modernize.

The industry has since moved toward automation-driven platforms, yet many dominant solutions continue to rely on legacy architectural patterns. This mismatch between modern operational demands and outdated system design has become a major constraint in the evolution of hosting management software.

2.2.2 Key Concepts and Theories

Several core software engineering concepts are central to modern hosting platforms:

Automation-Oriented System Design, which reduces manual intervention in billing, provisioning, and support of workflows.

Separation of Concerns, ensuring that business logic, presentation layers, and integrations remain loosely coupled.

Role-Based Access Control (RBAC) enabling secure and fine-grained authorization across multiple user roles.

Scalability Readiness, allowing systems to grow from small deployments to large multi-tenant environments without architectural redesign.

2.2.3 Related Work (Existing Platforms)

WHMCS

WHMCS is the most widely adopted commercial web hosting automation platform. It provides extensive functionality, including automated billing, domain management, support ticketing, and third-party integrations. Despite its market dominance, WHMCS is built on a legacy PHP-based monolithic architecture, resulting in high technical debt and limited extensibility.

Customizations often require direct modification of core files, increasing maintenance risk, and causing frequent upgrade conflicts. Additionally, its licensing and pricing model restrict accessibility for small hosting providers and startups.

Blesta

Blesta focuses on modularity and developer extensibility through plugins. While architecturally cleaner than WHMCS, it suffers from a comparatively smaller ecosystem, limited integrations, and a dated user interface. Its slower development pace limits its competitiveness in rapidly evolving hosting environments.

HostBill and WISECP

HostBill and WISECP are proprietary platforms known for their polished interfaces and modular feature offerings. HostBill targets enterprise-level customers but enforces restrictive licensing and high costs. WISECP offers a more modern UI and affordable pricing; however, both systems remain closed-sourced and rely on monolithic architectures, limiting deep customization and architectural transparency.

2.2.4 Gap Identification

The analysis of existing systems reveals consistent gaps:

Dependence on legacy technologies that hinder modernization.

Limited API-first support, restricting headless and multi-client usage.

Poor upgrade safety due to tight coupling

Inflexible RBAC implementations

Vendor lock-in caused by proprietary licensing models.

These gaps highlight the need for a system that prioritizes architectural clarity, extensibility, and future scalability.

2.3 BRAINSTORMING

Brainstorming sessions were conducted to explore architectural and functional alternatives to existing platforms. The focus was placed on eliminating unnecessary complexity while retaining essential hosting management capabilities. The outcome emphasized the importance of API-first design, modular boundaries, and stateless services to support scalability and future evolution.

2.4 EXISTNG SYSTEMS

This section presents a comparative review of widely used Web Hosting Management Systems (WHMS) with a focus on software engineering and architectural characteristics rather than feature lists alone. The comparison evaluates how existing platforms address modularity, scalability, extensibility, and long-term maintainability key concerns in modern system design.

2.4.1 Comparative Analysis of Existing Systems

Table 2.1: Comparative Analysis of Existing Systems

Criteria	**WHMCS**	**Blesta**	**HostBill**	**WISECP**
Core Architecture	Legacy Monolith	Monolith (Cleaner)	Monolith	Monolith
Technology Stack	PHP (Legacy)	PHP	PHP	PHP
API-First	No	Limited	Limited	Limited

Design				
Frontend Decoupling	No	No	No	No
Plugin Extensibility	Partial (Hooks)	Plugin-based	Module-based	Limited
Upgrade Safety	Low	Medium	Medium	Medium
Horizontal Scalability	Poor	Limited	Limited	Limited
RBAC Flexibility	Basic	Basic	Moderate	Basic
Vendor Lock-in	High	Medium	High	High
Microservices Readiness	No	No	No	No

2.4.2 Identified Software Engineering Gaps

The comparative analysis highlights several recurring software engineering gaps across existing WHMS platforms:

- Continued reliance on monolithic architectures, limiting scalability and modular evolution
- Absence of API-first design, restricting headless usage and third-party integrations
- Limited frontend extensibility, preventing independent UI evolution
- High vendor lock-in due to proprietary licensing and closed architectures
- Weak microservices migration paths, requiring major refactoring for architectural change

These gaps demonstrate that existing solutions prioritize functional completeness over architectural sustainability.

2.4.3 Positioning of the Proposed System

Unlike existing platforms, the proposed system is designed as an API-first, modular monolithic WHMS, explicitly addressing the identified gaps. By enforcing clean module boundaries, stateless service interactions, and well-defined APIs, the system provides a

scalable and extensible foundation that existing solutions lack. This architectural approach enables future migration to microservices while maintaining simplicity and feasibility within academic and practical constraints.

2.5 SUMMARY

This chapter presented a comprehensive literature and market survey of existing web hosting management systems with a focus on their architectural and software engineering characteristics. The analysis demonstrated that while current platforms such as WHMCS, Blesta, HostBill, and WISECP are functionally mature, they suffer from significant architectural limitations that hinder extensibility, scalability, and long-term maintainability.

The comparative review revealed common issues across these systems, including reliance on legacy monolithic architectures, lack of API-first design, tight coupling between frontend and backend components, limited upgrade safety, and strong vendor lock-in. These limitations restrict independent system evolution and make adaptation to modern deployment models both costly and risky.

Recent industry trends emphasize API-driven development, modular system design, stateless services, and scalable architectures that can evolve without extensive refactoring. The findings of this chapter clearly indicate a gap between these modern software engineering practices and the design of existing hosting management platforms.

Based on this analysis, the proposed system adopts an API-first, modular monolithic architecture that directly addresses the identified shortcomings. This approach enables clean separation of concerns, improved extensibility, safer upgrades, and a clear migration path toward microservices, while remaining feasible within academic and practical constraints. The conclusions drawn in this chapter therefore provide a strong technical justification for the architectural and technological choices discussed in subsequent chapters.

Chapter 3:

Requirement Engineering

Chapter 3
REQUIREMENT ENGINEERING

3.1 INTRODUCTION

This chapter presents a structured analysis of the problem space and requirements for the proposed Web Hosting Management System (WHMS). It identifies the key challenges faced by stakeholders in existing hosting management platforms and formalizes these challenges into well-defined problem statements. The chapter further explains the techniques used to elicit requirements and justifies their selection based on practical constraints and established software engineering practices.

Building upon the architectural gaps and limitations identified in Chapters 1 and 2, this chapter translates high-level motivations into concrete functional and non-functional requirements, ensuring that the proposed system addresses real-world technical, operational, and usability concerns.

3.2 PROBLEM SCENARIO

Table 3.2.1 PS-01 Legacy Architecture & Limited Customization

PS-01 Legacy Architecture & Limited Customization	
Description	Existing hosting management systems rely on tightly coupled, legacy monolithic architectures that restrict safe customization
Affected Stakeholders	Developers, Hosting Providers, System Administrators, End Users
Resulting Issues	High technical debt, fragile upgrades, limited extensibility, slow system evolution
Proposed System Benefits	Modular monolithic architecture enabling safe customization, independent module updates, and reduced maintenance effort

3.2.2 PS-02 Fragile Third-Party Integrations

PS-02 Fragile Third-Party Integrations	
Description	Integrations with payment gateways, domain registrars, and control panels are inconsistent and prone to failure after updates.
Affected Stakeholders	System Administrators, Support Teams
Resulting Issues	Automation failures, increased manual work, service delays
Proposed System Benefits	Unified API-first integration layer with reusable and stable service connectors

3.3.3 PS-03 Rigid UI/UX and Limited Customization

PS-03 Rigid UI/UX and Limited Customization	
Description	Current platforms provide limited UI flexibility and outdated user experiences.
Affected Stakeholders	Hosting Providers, End Users
Resulting Issues	Poor usability, reduced efficiency, limited branding and differentiation
Proposed System Benefits	Modern component-based UI enabling responsive design and workflow customization

3.3.4 PS-04 Unstandardized Plugin Ecosystem

PS-04 Unstandardized Plugin Ecosystem	
Description	Plugin ecosystems lack standardization, leading to inconsistent quality and compatibility issues.
Affected Stakeholders	Developers, System Administrators

Resulting Issues	Frequent integration failures, high maintenance overhead, unstable updates
Proposed System Benefits	Standardized, versioned plugin architecture ensuring compatibility and quality consistency

3.3.5 PS-05 Scalability & Performance Limitations

PS-05 Scalability & Performance Limitations	
Description	Existing systems are resource-heavy and optimized mainly for vertical scaling.
Affected Stakeholders	Hosting Providers
Resulting Issues	Performance degradation, limited concurrency, higher downtime risk
Proposed System Benefits	Asynchronous backend design supporting improved performance and readiness for horizontal scalability

3.3.6 PS-06 Security & Update Cadence Limitations

PS-06 Security & Update Cadence Limitations	
Description	Legacy systems depend on manual updates and infrequent security patches.
Affected Stakeholders	Clients, System Administrators
Resulting Issues	Increased vulnerability exposure, compliance risks, reduced system trust
Proposed System Benefits	Modern frameworks with faster updates, dependency checks, and consistent RBAC enforcement

3.3 ELICITATION TECHNIQUES

3.3.1 Elicitation Techniques

Requirements elicitation is a critical activity in requirements engineering, as it ensures that system requirements accurately reflect real-world problems and stakeholder needs. For the Web Hosting Management System (WHMS), a hybrid elicitation approach was adopted due to limited direct access to domain experts and hosting providers.

The following elicitation techniques were used:

3.3.1.1 Document Analysis

Existing documentation of established hosting management platforms (e.g., WHMCS manuals, API references, and developer guides) was analyzed to understand common workflows, system capabilities, and architectural constraints. This technique helped identify recurring limitations such as legacy architecture, upgrade fragility, and restricted extensibility.

3.3.1.2 Indirect Observation

Publicly available demo environments, video tutorials, and walkthroughs were reviewed to observe user interactions and administrative operations. This allowed the identification of usability issues, workflow rigidity, and operational bottlenecks without direct system access.

3.3.1.3 Online Research and Community Feedback

Discussions and feedback from hosting-related communities (e.g., WebHostingTalk, Reddit, cPanel forums) were examined to capture recurring pain points reported by developers and hosting providers, including high licensing costs, integration instability, and UI limitations.

3.3.1.4 Comparative System Analysis

A structured comparison of existing platforms such as WHMCS, Blesta, HostBill, and Clientexec was conducted to identify functional gaps and architectural weaknesses. This analysis directly informed the problem scenarios and requirement definitions.

3.3.1.5 Brainstorming and Internal Review

Regular team brainstorming sessions were held to consolidate findings, validate feasibility, and prioritize requirements. These sessions ensured alignment between identified problems, proposed solutions, and the selected technology stack.

3.3.2 Justification of Elicitation Approach

In requirement engineering, the selection of elicitation techniques must be guided by stakeholder accessibility, project constraints, and the nature of the problem domain. For the proposed Web Hosting Management System (WHMS), direct elicitation techniques such as interviews and workshops were limited due to restricted access to hosting providers, system administrators, and domain experts. Such constraints are common in academic and early-stage system development projects.

Established software engineering literature recognizes that when direct stakeholder interaction is not feasible, secondary and indirect elicitation techniques including document analysis, observation, and comparative system studies are valid and effective alternatives. According to **Kotonya and Sommerville (1998)**, requirements can be reliably elicited from existing systems, documentation, and domain knowledge, particularly when developing systems that replace or improve existing solutions. Similarly, **Sommerville (2015)** emphasizes that studying legacy systems and market-leading products is an essential method for understanding functional requirements and system constraints.

Furthermore, **Pressman and Maxim (2020)** state that online research, user community feedback, and comparative analysis are valuable sources of requirements, especially for software products with large existing user bases and well-documented pain points. In the context of hosting management systems, public forums, developer documentation, and user discussions provide rich insight into real-world operational issues such as integration fragility, upgrade instability, and usability limitations.

The hybrid elicitation approach adopted in this project—combining document analysis, indirect observation, community feedback, comparative analysis, and structured brainstorming—ensures that the identified requirements are **evidence-based, realistic, and aligned with industry practices**. This approach also mitigates the risk of biased or

incomplete requirements by cross-validating findings across multiple independent sources.

In accordance with **IEEE SWEBOK (2014)** guidelines, the use of multiple elicitation techniques improves requirement completeness and correctness, particularly when stakeholder availability is constrained. Therefore, the selected elicitation strategy is both **methodologically sound and academically justified**, enabling the development of a robust requirements specification that directly addresses the architectural and operational shortcomings identified in Chapters 1 and 2.

3.4 FUNCTIONAL REQUIREMENTS

3.4.1 Authentication and Session Management

3.4.1.1 User Account Module

FR-AUTH-01 The system shall allow users to register using an email address and password via a public API endpoint.

FR-AUTH-02 The system shall enforce the uniqueness of user email addresses at the persistence layer to prevent duplicate accounts.

FR-AUTH-03 The system shall automatically assign a default role to newly registered users.

FR-AUTH-04 The system shall generate a time-bound email verification token upon successful registration.

FR-AUTH-05 The system shall send an email verification link to the registered email address using an asynchronous email service.

FR-AUTH-06 The system shall allow users to request a new verification email via an authenticated or unauthenticated API endpoint.

FR-AUTH-07 The system shall verify user email addresses using a tokenized verification link.

FR-AUTH-08 The system shall update the user's email verification status upon successful token validation.

FR-AUTH-09 The system shall invalidate all previously issued verification tokens after successful email verification.

3.4.1.2 Authentication & Session Management

FR-AUTH-10 The system shall authenticate users using email and password credentials through an API endpoint.

FR-AUTH-11 The system shall issue JWT-based access and refresh tokens upon successful authentication.

FR-AUTH-12 The system shall store refresh tokens as persistent session records.

FR-AUTH-13 The system shall support multiple concurrent sessions per user across different devices.

FR-AUTH-14 The system shall associate IP address and user-agent metadata with each session.

FR-AUTH-15 The system shall allow users to logout by revoking the active session via an API call.

FR-AUTH-16 The system shall allow users to revoke individual or all active sessions via authenticated APIs.

FR-AUTH-17 The system shall expose a /me API endpoint to return the authenticated user context.

3.4.1.3 Password Management

FR-PASS-01 The system shall allow users to request a password reset using their registered email address via an API endpoint.

FR-PASS-02 The system shall generate cryptographically secure, time-limited password reset tokens.

FR-PASS-03 The system shall send password reset links via email using an asynchronous mail service.

FR-PASS-04 The system shall validate password reset tokens before allowing password updates.

FR-PASS-05 The system shall allow users to reset their password using a valid token.

FR-PASS-06 The system shall invalidate all active sessions after a successful password reset.

FR-PASS-07 The system shall log password reset request and completion of events.

3.4.1.4 Role-Based Profile Management

FR-PROFILE-01 The system shall automatically create role-specific profiles (client, admin, reseller, developer) based on assigned roles.

FR-PROFILE-02 The system shall maintain separate profile data for each portal type.

FR-PROFILE-03 The system shall expose profile data through APIs without embedding any UI-specific logic.

3.4.1.5 Administrative & User Management

FR-ADMIN-01 The system shall restrict administrative APIs using role-based access control.

FR-ADMIN-02 The system shall maintain a dedicated administrative profile for admin users.

FR-ADMIN-03 The system shall allow administrators to retrieve and manage user accounts via APIs.

FR-ADMIN-04 The system shall allow administrators to assign and revoke roles from users.

FR-ADMIN-05 The system shall allow administrators to deactivate user accounts.

FR-ADMIN-06 The system shall allow administrators to forcibly revoke all active sessions of a user.

FR-ADMIN-07 The system shall enforce permission-based authorization for all administrative operations.

3.4.1.6 Impersonation

FR-IMP-01 The system shall allow authorized administrators to initiate user impersonation via an API request.

FR-IMP-02 The system shall require a reason to be provided when starting impersonation.

FR-IMP-03 The system shall generate impersonation-specific access and refresh tokens.

FR-IMP-04 The system shall store impersonation sessions independently from standard user sessions.

FR-IMP-05 The system shall attach an impersonator identifier to the authenticated request context.

FR-IMP-06 The system shall allow administrators to perform actions as the impersonated user.

FR-IMP-07 The system shall allow administrators to stop impersonation at any time via an API.

FR-IMP-08 The system shall log impersonation start and stop events.

FR-IMP-09 The system shall expose impersonation status information via authenticated APIs.

3.4.1.7 Multi-Factor Authentication (MFA)

FR-MFA-01 The system shall allow users to enroll in multi-factor authentication via APIs.

FR-MFA-02 The system shall generate and store TOTP secrets securely.

FR-MFA-03 The system shall challenge MFA-enabled users during authentication.

FR-MFA-04 The system shall validate MFA codes before completing authentication.

FR-MFA-05 The system shall support one-time backup codes for MFA recovery.

FR-MFA-06 The system shall allow trusted devices to bypass MFA challenge

3.4.1.8 API Key Management

FR-API-01 The system shall allow authenticated users to create API keys.

FR-API-02 The system shall store API keys in hashed form.

FR-API-03 The system shall allow scoped permissions to be assigned to API keys.

FR-API-04 The system shall authenticate API requests using valid API keys.

FR-API-05 The system shall allow users to revoke API keys.

FR-API-06 The system shall log API key creation, usage, and revocation events.

3.4.1.9 Role-Based Access Control (RBAC)

FR-RBAC-01 The system shall load predefined roles and permissions during system initialization.

FR-RBAC-02 The system shall associate permissions with roles.

FR-RBAC-03 The system shall associate roles with users.

FR-RBAC-04 The system shall evaluate role and permission constraints during API request authorization.

FR-RBAC-05 The system shall expose role and permission data to client applications for UI-level access control.

3.4.1.10 IP Access Control

FR-IP-01 The system shall allow administrators to define IP allow-list and deny-list FR-IP-02. The system shall support exact IP addresses, wildcard ranges, and CIDR subnet rules.

FR-IP-03 The system shall evaluate IP access rules during authentication and authorization.

FR-IP-04 The system shall allow administrators to list, update, and delete IP access rules via APIs.

3.4.1.11 Audit Logging

FR-AUDIT-01 The system shall log all authentication attempts.

FR-AUDIT-02 The system shall log session creation, refresh, and revocation events.

FR-AUDIT-03 The system shall log impersonation activities.

FR-AUDIT-04 The system shall log password reset and email verification events.

FR-AUDIT-05 The system shall log administrative roles and permission changes.

3.4.1.12 Webhooks

FR-WEBHOOK-01 The system shall emit webhook events for authentication and security-related activities.

FR-WEBHOOK-02 The system shall deliver webhook events asynchronously.

FR-WEBHOOK-03 The system shall cryptographically sign webhook payloads using HMAC SHA-256.

FR-WEBHOOK-04 The system shall retry failed webhook deliveries using an exponential backoff strategy.

FR-WEBHOOK-05 The system shall store webhook delivery logs.

3.4.2 Automation Module

FR-AUTO-1: The system shall provide REST API endpoints to perform full CRUD operations (Create, Retrieve, Update, Delete) on automation profiles containing scheduling and configuration details.

FR-AUTO-2: The system shall interpret standard cron expressions via the API to automatically schedule and trigger execution cycles.

FR-AUTO-3: The system shall expose API endpoints to dynamically toggle profiles between "enabled" and "disabled" states, updating the active scheduler in real-time without requiring a system restart.

FR-AUTO-4: The system shall provide REST APIs to manage sub-resource automation tasks, allowing them to be added, updated, listed, or removed within a specific parent profile.

FR-AUTO-5: The system shall enforce a deterministic execution sequence for tasks within a profile based on a user-defined "order" attribute.

FR-AUTO-6: The system shall expose dedicated API endpoints to manually initiate the immediate execution of entire automation profiles or individual isolated tasks.

FR-AUTO-7: The system shall offload all automation task executions to a background job queue to maintain non-blocking API performance and eliminate request timeouts.

FR-AUTO-8: The system shall maintain an internal library of built-in automation actions, such as HTTP requests and notification triggers, accessible via a unified executor interface.

FR-AUTO-9: The system shall support the execution of plugin-defined actions using a standardized plugin:<id>:<action> interface, allowing for external logic injection.

FR-AUTO-10: The system shall record and expose the life cycle state of every run, including status (pending, running, success, failed), high-resolution timestamps, and structured results in persistent storage.

FR-AUTO-11: The system shall process automation jobs using a logically independent execution model that supports fault isolation and readiness for horizontal scalability across distributed nodes.

FR-AUTO-12: The system shall generate comprehensive audit logs for all operations, including profile modifications and execution failures, enriched with actor metadata and IP context.

FR-AUTO-13: The system shall assign a unique correlation ID to all automation errors to facilitate rapid log tracing and cross-service debugging.

FR-AUTO-14: The system shall implement strict schema validation for all incoming API payloads using AJV to ensure data integrity before persistence or execution.

FR-AUTO-15: The system shall capture and store execution metadata, including node identification and retry counts, for every automation job processed by the workers.

FR-AUTO-16: The system shall allow external plugins to dynamically register new automation actions and storage handlers via the ExecutorService.

FR-AUTO-17: The system shall forward internal life cycle events—such as completion or failure—to the audit and notification engines for secondary workflow triggers.

FR-AUTO-18: The system shall expose all automation logic exclusively through Restful APIs, adhering to a strict "headless" design without embedded user-interface logic.

3.4.3 Plugin Module

FR-PLUG-01 The system shall automatically discover and load plugins from a designated filesystem directory during application startup and runtime reload via a backend plugin loading API.

FR-PLUG-02 The system shall validate each plugin's manifest.json against a predefined JSON Schema through a backend validation service before registering or executing the plugin.

FR-PLUG-03 The system shall register validated plugin metadata (including plugin ID, name, version, type, and UI metadata) in an internal registry accessible through plugin management APIs.

FR-PLUG-04 The system shall register plugin-defined actions declared in the plugin manifest and expose them via an internal action registry for invocation by authorized backend services.

FR-PLUG-05 The system shall execute JavaScript-based plugin actions through a sandboxed VM environment using a controlled execution API to ensure isolation, timeout enforcement, and restricted host access.

FR-PLUG-06 The system shall expose plugin-provided user interfaces through a dedicated HTTP endpoint and render them within an iframe-based sandbox enforcing Content Security Policy (CSP) restrictions.

FR-PLUG-07 The system shall provide an API endpoint to retrieve a list of all installed plugins along with their metadata, enabled status, available actions, and UI capabilities.

FR-PLUG-08 The system shall provide API endpoints that allow plugins to define configurable settings, validate configuration data using optional schemas, and persist configuration values in the system database.

FR-PLUG-09 The system shall provide an authenticated API endpoint to upload, validate, and install plugins packaged as ZIP archives, including optional signature verification and runtime verification.

FR-PLUG-10 The system shall expose an internal execution API enabling the automation module to invoke registered plugin actions as part of automated workflows and task pipelines.

FR-PLUG-11 The system shall support execution of WebAssembly-based plugin actions through a dedicated WASM execution API with controlled filesystem access, runtime limits, and metadata input handling.

3.4.4 Automation + Plugin Integration

FR-ATPL-1 The system shall expose registered plugin actions as executable automation tasks through an internal automation execution API, allowing plugins to be selected as action types within automated workflows.

FR-ATPL-2 The system shall provide an API endpoint to manually trigger plugin actions outside of scheduled automation workflows for testing, debugging, and validation purposes.

FR-ATPL-3 The system shall allow the automation engine to execute multiple plugin-based actions sequentially as part of a single automation workflow, passing task-specific metadata to each action execution.

FR-ATPL-4 The system shall allow plugin-provided configuration user interfaces to be embedded within the automation task configuration process via a sandboxed UI endpoint for action parameter setup.

3.4.5 Marketplace Module

FR-M-01 System shall allow users to browse available marketplace plugins through an API.

FR-M-02 System shall allow users to view plugin specifications, metadata, and declared actions.

FR-M-03 System shall allow users to install plugins from the marketplace using the existing plugin installation service.

FR-M-04 System shall allow users to update installed plugins by reinstalling newer marketplace versions.

FR-M-05 System shall manage basic plugin licensing metadata for informational and validation purposes.

FR-M-06 System shall allow developers to submit plugins to the marketplace subject to predefined validation rules.

FR-M-07 System shall support plugin dependency declarations and perform dependency checks during installation.

FR-M-08 System shall perform plugin verification checks, including manifest validation and optional authenticity validation.

FR-M-09 System shall allow users to view basic plugin usage and installation information.

FR-M10 System shall allow users to submit and view basic plugin ratings.

FR-M11 System shall allow authorized users to manage marketplace plugin metadata.

3.4.6 System Backup

FR-BACKUP-1 The system shall support multiple backup storage providers through a pluggable provider registry.

FR-BACKUP-2 The system shall expose APIs to retrieve a list of available backup storage providers along with their configuration schemas.

FR-BACKUP-3 The system shall allow authorized users to create backup storage configurations via API endpoints.

FR-BACKUP-4 The system shall store backup storage configuration credentials in encrypted form at the persistence layer.

FR-BACKUP-5 The system shall expose an API endpoint to validate backup storage provider configurations before use.

FR-BACKUP-6 The system shall allow backup storage configurations to be updated or revoked via authenticated APIs.

FR-BACKUP-7 The system shall expose an API endpoint to allow authorized users to manually initiate backup operations.

FR-BACKUP-8 The system shall support multiple backup types, including file-system backups, database-only backups, and full backups.

FR-BACKUP-9 The system shall perform file-system backups by compressing selected file paths into a streamed archive format.

FR-BACKUP-10 The system shall perform database backups by exporting database schema and data into a backup archive.

FR-BACKUP-11 The system shall stream backup archives directly to the selected storage provider without relying on large local temporary files.

FR-BACKUP-12 The system shall execute backup operations asynchronously using a background job queue.

FR-BACKUP-13 The system shall create a persistent backup record for each backup request.

FR-BACKUP-14 The system shall log backup execution steps, status changes, and metadata for auditing purposes.

FR-BACKUP-15 The system shall record backup attributes including type, storage provider, size, timestamps, and execution status.

FR-BACKUP-16 The system shall expose APIs to retrieve backup records and associated execution logs.

FR-BACKUP-17 The system shall enforce role-based access control for all backup-related APIs.

FR-BACKUP-18 The system shall restrict backup visibility such that users can only access backups they are authorized to view.

FR-BACKUP-19 The system shall expose an API endpoint to allow authorized users to download backup archives directly from the storage provider.

FR-BACKUP-20 The system shall expose an API endpoint to initiate backup restore operations.

FR-BACKUP-21 The system shall restrict restore operations to authorized administrative roles.

FR-BACKUP- 22 The system shall support restoring file-system data from backup archives.

FR-BACKUP- 23 The system shall support optional database restoration from backup archives.

FR-BACKUP-24 The system shall execute restore operations asynchronously using a background job queue.

FR-BACKUP-25 The system shall log restore execution steps, status, and results for auditing purposes.

FR-BACKUP-26 The system shall allow retention policies to be defined for backup records.

FR-BACKUP-27 The system shall automatically identify expired backups based on configured retention policies.

FR-BACKUP-28 The system shall remove expired backup archives from storage providers automatically.

FR-BACKUP-29 The system shall update backup metadata to reflect deletion or expiration events.

FR-BACKUP-30 The system shall emit backup lifecycle events, including backup initiation, completion, and failure.

FR-BACKUP-31 The system shall emit restore lifecycle events, including restore initiation, completion, and failure.

FR-BACKUP- 32 The system shall forward backup and restore events to the automation engine for workflow execution.

FR-BACKUP- 33 The system shall allow external plugins to register new backup storage providers dynamically.

FR-BACKUP-34 The system shall allow plugins to subscribe to backup and restore lifecycle events.

FR-BACKUP-35 The system shall expose all backup and restore functionality exclusively through APIs without embedding any user-interface logic.

3.4.7 Domain Management

FR-DOM-01 The system shall provide a unified API endpoint to check domain availability using configured external registrar providers

FR-DOM-02 The system shall support real-time domain availability checks for supported TLDs through active registrar integrations.

FR-DOM-03 The system shall return standardized availability results including availability status, premium indicator, and base pricing information

FR-DOM-04 The system shall validate domain name syntax and format before submitting availability requests to a registrar.

FR-DOM-05 The system shall normalize registrar-specific availability responses into a consistent internal data structure.

FR-DOM-06 The system shall support domain registration requests through registrar integration APIs for authorized system users.

FR-DOM-07 The system shall persist domain registration metadata including domain name, registrar provider, registration timestamp, and expiry date.

FR-DOM-08 The system shall maintain domain ownership linkage between registered domains and system users.

FR-DOM-09 The system shall support initiating domain transfer requests using authorization (EPP) codes via supported registrars.

FR-DOM-10 The system shall track the operational lifecycle state of each domain (e.g., pending, active, expired, transfer-pending)

FR-DOM-11 The system shall expose domain lifecycle and status information through administrative APIs.

FR-DOM-12 The system shall integrate with domain registrars using a provider-based abstraction layer.

FR-DOM-13 The system shall allow registrar providers to be enabled or disabled through configuration stored in the database.

FR-DOM-14 The system shall securely store registrar API credentials in encrypted form within the persistence layer.

FR-DOM-15 The system shall route domain operations exclusively through registrar providers marked as active.

FR-DOM-16 The system architecture shall support future registrar integrations without requiring changes to existing domain management APIs.

FR-DOM-17 The system shall retrieve base domain pricing information from registrar APIs where supported.

FR-DOM-18 The system shall synchronize registrar-provided TLD pricing data into the local database on administrative request.

FR-DOM-19 The system shall store registration, renewal, and transfer pricing attributes separately for each supported TLD.

FR-DOM-20 The system shall allow administrators to configure pricing markups on registrar-provided domain prices

FR-DOM-21 The system shall calculate final domain prices dynamically using base registrar pricing and configured markup rules

FR-DOM-22 The system shall allow administrators to activate or deactivate individual TLDs for sale.

FR-DOM-23 The system shall allow administrators to retrieve a list of all domains managed by the system.

FR-DOM-24 The system shall allow administrators to view detailed domain metadata including registrar, pricing, and lifecycle information.

FR-DOM-25 The system shall support soft deletion of domain records while preserving historical data.

FR-DOM-26 The system shall record domain-related administrative actions within the system audit logging mechanism.

FR-DOM-27 The system shall restrict domain management operations to authorized administrative roles.

FR-DOM-28 The system shall expose domain-related operations through APIs suitable for integration with internal automation components.

FR-DOM-29 The system shall allow automation workflows to trigger domain-related synchronization or validation actions.

FR-DOM-30 The system shall emit internal system events for significant domain lifecycle changes (e.g., registration, renewal, expiry update).

3.4.8 Clients

3.4.9 Services

FR-SRV-01 The system shall expose an API to create a new hosting service.

FR-SRV-02 The system shall require each hosting service to have a unique service code.

FR-SRV-03 The system shall allow administrators to update service name, description, and activation status.

FR-SRV-04 The system shall allow administrators to deactivate a service without deleting it.

FR-SRV-05 The system shall expose an API to retrieve all services, including inactive ones.

FR-SRV-06 The system shall allow administrators to create multiple service plans under a service.

FR-SRV-07 The system shall enforce unique plan names within the scope of a single service.

FR-SRV-08 The system shall allow administrators to activate or deactivate individual service plans.

FR-SRV-09 The system shall allow administrators to define pricing for service plans per billing cycle.

FR-SRV-10 The system shall allow administrators to define pricing for service plans per billing cycle.

FR-SRV-11 The system shall prevent duplicate pricing entries for the same plan and billing cycle.

FR-SRV-12 The system shall expose a client-facing API to retrieve only active services.

FR-SRV-13 The system shall expose only active plans and pricing to client-facing APIs

FR-SRV-14 The system shall preserve historical service configuration using immutable service snapshots.

FR-SRV-15 The system shall ensure that pricing changes do not affect already purchased services.

FR-SRV-16 The system shall restrict service and pricing management APIs to authorized administrative roles.

FR-SRV-17 The system shall record administrative service actions within the centralized audit logging system.

3.4.10 Orders

FR-ORD-01 The system shall expose an API to allow authenticated clients to place service orders.

FR-ORD-02 The system shall create an immutable snapshot of service, plan, and pricing data at the time of order placement.

FR-ORD-03 The system shall associate each order with the authenticated client who placed it.

FR-ORD-04 The system shall allow clients to retrieve a list of their own orders.

FR-ORD-05 The system shall allow clients to retrieve details of a specific order they own.

FR-ORD-06 The system shall expose an administrative API to retrieve all orders in the system.

FR-ORD-07 The system shall initialize newly created orders in a pending state.

FR-ORD-08 The system shall allow clients to cancel their own orders while in a pending state.

FR-ORD-09 The system shall prevent cancellation of orders that are no longer pending.

FR-ORD-10 The system shall restrict order creation and retrieval to authenticated users.

FR-ORD-11 The system shall prevent clients from accessing orders belonging to other users.

FR-ORD-12 The system shall restrict administrative order listing to authorized administrative roles.

FR-ORD-13 The system shall record order creation and status changes within the audit logging mechanism.

3.5 NON-FUNCTIONAL REQUIREMENTS

3.6 FR ↔ API ENDPOINT MAPPING

3.4.1&3.4.2 Module FR ↔ API ENDPOINT MAPPING

FR-ID	API ENDPOINT	METHOD	DESCRIPTION
FR-AUTH-01	/api/auth/register	POST	Register user
FR-AUTH-02	/api/auth/register	POST	Enforce unique email
FR-AUTH-04	/api/auth/email/send	POST	Send verification email
FR-AUTH-06	/api/auth/email/send-verification	POST	Resend verification
FR-AUTH-07	/api/auth/email/verify	GET	Verify email token
FR-AUTH-10	/api/auth/login	POST	User login
FR-AUTH-11	/api/auth/logout	POST	Refresh token
FR-AUTH-14	/api/auth/refresh	POST	logout
FR-AUTH-17	/api/auth/me	GET	Current User

FR-SESSION-01	/api/sessions	GET	List sessions
FR-SESSION-04	/api/sessions/:id	DELETE	Revoke session
FR-SESSION-5	/api/sessions/others/all	DELETE	Revoke all session

3.4.3&3.4.5 Module FR ↔ API ENDPOINT MAPPING

FR-ID	**API ENDPOINT**	**METHOD**	**DESCRIPTION**
FR-PASS-01	/api/auth/password/request-reset	POST	Requests a password reset link for a registered email.
FR-PASS-04	/api/auth/password/reset	POST	Resets the user password using a valid reset token.
FR-PASS-05	/api/auth/password/reset	POST	Invalidates existing sessions after password reset.
FR-ADMIN-03	/api/admin/users	GET	Retrieves a list of user accounts.
FR-ADMIN-04	/api/admin/users/:id/roles	POST	Updates roles assigned to a user.
FR-ADMIN-05	/api/admin/users/:id/deactivate	POST	Deactivates a user account.
FR-ADMIN-06	/api/admin/users/:id/logout	POST	Forces logout of all user sessions.

3.4.6&3.4.7 Module FR ↔ API ENDPOINT MAPPING

FR-ID	**API ENDPOINT**	**METHOD**	**DESCRIPTION**
FR-IMP-01	/api/auth/impersonate/start	POST	Starts impersonation of a target user.

FR-IMP-07	/api/auth/impersonate/stop	POST	Stops an active impersonation session.
FR-IMP-09	/api/auth/impersonation-status	GET	Returns current impersonation status.
FR-ADMIN-06	/api/admin/impersonation/logs	GET	Retrieves impersonation activity logs.
FR-MFA-01	/api/auth/mfa/setup	POST	Initiates MFA setup for a user.
FR-MFA-03	/api/auth/mfa/verify-login	POST	Verifies MFA code during login.
FR-MFA-05	/api/auth/trusted-devices	POST	Registers a trusted device for MFA bypass.

3.4.8 & 3.4.9 & 3.4.10 Module FR ↔ API ENDPOINT MAPPING

FR-ID	**API ENDPOINT**	**METHOD**	**DESCRIPTION**
FR-API-01	/api/auth/api-keys	POST	Creates a new API key.
FR-API-02	/api/auth/api-keys	GET	Lists existing API keys.
FR-API-05	/api/auth/api-keys/:id	DELETE	Revokes an API key.
FR-IP-01	/api/auth/ip-rules	POST	Creates an IP access rule.
FR-IP-04	/api/auth/ip-rules/:id	DELETE	Deletes an IP access rule.
FR-IP-07	/api/auth/ip-rules	GET	List all IP access rule.
FR-WEBHOOK-01	Internal-Event Dispatcher	Async	Dispatches webhook events asynchronously.
FR-WEBHOOK-05	/api/admin/webhooks/logs	GET	Retrieves webhook delivery logs.

3.4.10 & 3.4.11 & 3.4.12 Module FR ↔ API ENDPOINT MAPPING

FR-ID	API ENDPOINT	METHOD	DESCRIPTION
FR-BACKUP-01	/api/backup/providers	GET	Retrieve available backup storage providers and schemas
FR-BACKUP-03	/api/backup/test-storage	POST	Validate backup storage provider configuration
FR-BACKUP-04	/api/storage-config	POST	Create encrypted storage configuration
FR-BACKUP-06	/api/storage-config	GET	List storage configurations
FR-BACKUP-06	/api/storage-config/:id	GET	Retrieve single storage configuration
FR-BACKUP-06	/api/storage-config/:id	PATCH	Update storage configuration
FR-BACKUP-06	/api/storage-config/:id	DELETE	Delete storage configuration
FR-BACKUP-07	/api/backup	POST	Manually trigger backup operation
FR-BACKUP-09	/api/backup	GET	List backups for authorized user
FR-BACKUP-13	/api/backup/:id	GET	Retrieve backup metadata and details
FR-BACKUP-11	/api/backup/:id/download	GET	Download backup archive

FR-BACKUP-20	/api/backup/:id/restore	POST	Initiate restore operation (Admin only)
FR-BACKUP-24	/api/backup/:id/logs	GET	Retrieve backup execution step logs
FR-BACKUP-26	/api/backup/retention	GET	Retrieve retention policy configuration
FR-BACKUP-26	/api/backup/retention	POST	Configure retention policy
FR-BACKUP-28	/api/backup/retention/run	POST	Trigger retention cleanup job
FR-BACKUP-30	/api/backup/events	GET	Retrieve backup and restore lifecycle events

3.4. Automation Module

3.4.13 & 3.4.14 & 3.4.15 Module FR ↔ API ENDPOINT MAPPING

FR-ID	**API Endpoint**	**Method**	**Description**
FR-AUTO-1	/api/automation/profiles	GET	Retrieve a list of all automation profiles.
FR-AUTO-1	/api/automation/profiles	POST	Create a new automation profile with a cron-based schedule.
FR-AUTO-1	/api/automation/profiles/:id	GET	Retrieve details of a specific automation profile.
FR-AUTO-1	/api/automation/profiles/:id	PUT	Update an existing automation profile

			and refresh its scheduler.
FR-AUTO-1	/api/automation/profiles/:id	DELETE	Delete an automation profile and stop its scheduled execution.
FR-AUTO-3	/api/automation/profiles/:id/enable	POST	Enable the execution schedule of an automation profile.
FR-AUTO-3	/api/automation/profiles/:id/disable	POST	Disable the execution schedule of an automation profile.
FR-AUTO-4	/api/automation/profiles/:profileId/tasks	GET	Retrieve all tasks associated with a specific automation profile.
FR-AUTO-4	/api/automation/profiles/:profileId/tasks	POST	Add a new task to an automation profile.
FR-AUTO-4	/api/automation/tasks/:taskId	GET	Retrieve configuration details of a specific automation task.
FR-AUTO-4	/api/automation/tasks/:taskId	PUT	Update the action type or metadata of

			an automation task.
FR-AUTO-4	/api/automation/tasks/:taskId	DELETE	Remove a task from its associated automation profile.
FR-AUTO-6	/api/automation/run/:profileId	POST	Manually trigger execution of an entire automation profile.
FR-AUTO-6	/api/automation/tasks/:taskId/run	POST	Manually trigger execution of a single automation task.
FR-AUTO-8	/api/automation/actions	GET	Retrieve a list of built-in automation actions (e.g., HTTP, Email).
FR-AUTO-9	/api/automation/test-plugin-run	POST	Test execution of plugin-defined automation actions.

3.4.16 & 3.4.17 & 3.4.18 Module FR ↔ API ENDPOINT MAPPING

FR-ID	**API Endpoint(s)**	**Method**	**Purpose / Description**
FR-PLUG-01	*(Startup/Reload)*/api/plugins/reload *(internal)*	POST	Triggers plugin loader to scan filesystem and load plugins automatically
FR-PLUG-02	/api/plugins/install/zip	POST	Validates manifest.json using JSON Schema before

			installation
FR-PLUG-03	/api/plugins	GET	Returns registered plugin metadata from internal registry
FR-PLUG-04	/api/plugins/:pluginId/actions	GET	Lists actions registered for a specific plugin
FR-PLUG-05	/api/plugins/:pluginId/actions/:actionName	POST	Executes JavaScript plugin action inside VM sandbox
FR-PLUG-06	/Plugins/ui/:pluginId/frame	GET	Serves plugin UI inside iframe sandbox with CSP
FR-PLUG-07	/api/plugins	GET	Lists all installed plugins with metadata
FR-PLUG-08	/api/plugins/:pluginId/config	GET / POST	Retrieve and persist plugin configuration in database
FR-PLUG-09	/api/plugins/install/zip	POST	Uploads, validates, installs plugin archive
FR-PLUG-10	*(Internal)*ExecutorService.run()	Internal API	Allows automation engine to invoke plugin actions
FR-PLUG-	/api/plugins/:pluginId/actions/:actionName	POST	Executes

11			WASM-based plugin actions

3.4.19 & 3.4.20 & 3.4.21 Module FR ↔ API ENDPOINT MAPPING

FR-ID	**API Endpoint**	**Method**	**Description**
FR-DOM-01	/api/domains/check-availability	POST	Check domain availability via an external registrar using a unified API.
FR-DOM-02	/api/domains/check-availability	POST	Perform real-time availability checks for supported TLDs using active registrar providers.
FR-DOM-03	/api/domains/check-availability	POST	Return structured availability results including availability status, premium flag, and pricing metadata.
FR-DOM-04	Internal Validation Layer	—	Validate domain name format before initiating registrar availability checks.
FR-DOM-05	Internal Mapping Layer	—	Normalize registrar-specific availability responses into a standardized internal format.
FR-DOM-06	/api/domains/register	POST	Register an available domain through the active registrar provider.
FR-DOM-07	/api/domains/register	POST	Persist registered domain metadata including

			registrar, registration date, and expiry date.
FR-DOM-08	/api/domains/:domainId	GET	Retrieve persisted domain registration and expiry metadata.
FR-DOM-09	/api/domains/transfer	POST	Initiate domain transfer using registrar authorization code.
FR-DOM-10	Internal Lifecycle Manager	—	Track domain lifecycle states such as pending, active, expired, or transferred.
FR-DOM-11	/api/admin/domains/:id/lifecycle	GET	Retrieve domain lifecycle information for administrative inspection.
FR-DOM-12	Internal Registrar Interface	—	Integrate domain operations through a provider-based abstraction layer.
FR-DOM-13	/api/admin/providers/:providerId	PUT	Enable or disable registrar providers via database-stored configuration.
FR-DOM-14	Internal Credential Vault	—	Securely store registrar API credentials in encrypted form.
FR-DOM-15	Internal Registrar Resolver	—	Route all domain operations through the currently active registrar provider.
FR-DOM-16	Registrar Adapter Interface	—	Allow future registrar integrations without

			modifying existing domain APIs.
FR-DOM-17	/api/domains/pricing/sync	POST	Retrieve domain pricing information from registrar APIs.
FR-DOM-18	/api/domains/pricing/sync	POST	Synchronize TLD pricing data from registrar into the local database.
FR-DOM-19	/api/admin/tlds	GET	Retrieve registration, renewal, and transfer prices for each supported TLD.
FR-DOM-20	/api/admin/tlds/:tldId	PUT	Configure markup percentages on registrar-provided pricing.
FR-DOM-21	Internal Pricing Engine	—	Calculate final domain prices dynamically using base price and markup.
FR-DOM-22	/api/admin/tlds/:tldId/enable	POST	Enable or disable individual TLDs for domain registration.
FR-DOM-23	/api/admin/domains	GET	Retrieve a list of all domains managed by the system.
FR-DOM-24	/api/admin/domains/:id	GET	Retrieve detailed domain metadata for administrative use.
FR-DOM-25	/api/admin/domains/:id	DELETE	Soft delete a domain record without permanent data loss.
FR-DOM-	Internal Audit Logger	—	Record domain-related

26			administrative actions in the audit logging subsystem.
FR-DOM-27	Auth Middleware	—	Restrict domain management APIs using role-based access control.
FR-DOM-28	/api/domains/*	—	Expose domain-related APIs suitable for automation workflows.
FR-DOM-29	/api/automation/domain-sync	POST	Allow automation profiles to trigger domain-related synchronization actions.
FR-DOM-30	Internal Event Bus	—	Emit internal events for critical domain lifecycle changes.

3.4.22 & 3.23.9 & 3.4.24 Module FR ↔ API ENDPOINT MAPPING

FR-ID	**API Endpoint**	**Method**	**Description**
FR-SERV-01	/api/admin/services	POST	Create a new hosting service definition with name, code, and description.
FR-SERV-02	/api/admin/services	GET	Retrieve a list of all services (active and inactive) for

			administrative management.
FR-SERV-03	/api/admin/services/:id	GET	Retrieve full details of a specific service, including associated plans.
FR-SERV-04	/api/admin/services/:id	PUT	Update service metadata such as name, description, or activation status.
FR-SERV-05	/api/admin/services/:id	DELETE	Deactivate a service to prevent new purchases while preserving historical data.
FR-SERV-06	/api/client/services	GET	Retrieve a list of active services visible to clients.
FR-SERV-07	/api/admin/services/:serviceId/plans	POST	Create a new plan under a specific service (e.g., Basic, Pro).
FR-SERV-08	/api/admin/plans/:planId	PUT	Update metadata of an existing service plan.
FR-SERV-09	/api/admin/plans/:planId/pricing	POST	Define pricing for a service plan based on billing cycle.
FR-SERV-10	/api/admin/plans/:planId/pricing	POST	Enforce unique pricing per billing cycle for a service

			plan.
FR-SERV-11	/api/admin/plans/:planId/policies	POST	Define policy rules or constraints associated with a service plan.
FR-SERV-12	/api/internal/services/:serviceId/snapshot	POST	Generate an immutable snapshot of service, plan, and pricing data for historical integrity.
FR-SERV-13	/api/admin/services/:id	PUT	Restrict service modification operations to authorized administrative users only.
FR-SERV-14	/api/admin/services/:id	GET	Handle invalid or non-existent service identifiers with controlled error responses.

3.4.25 & 3.4.26 & 3.4.28 Module FR ↔ API ENDPOINT MAPPING

FR-ID	**API Endpoint**	**Method**	**Description**
FR-ORD-01	/api/orders	POST	Create a new order for an authenticated client using a selected service plan and pricing configuration.
FR-ORD-02	/api/orders	GET	Retrieve a list of all

			orders belonging to the authenticated client.
FR-ORD-03	/api/orders/:id	GET	Retrieve detailed information of a specific order owned by the authenticated client.
FR-ORD-04	/api/admin/orders	GET	Retrieve a list of all client orders for administrative oversight.
FR-ORD-05	/api/orders/:id/cancel	POST	Allow a client to cancel their own order while it is in a pending state.
FR-ORD-06	/api/orders/:id	GET	Restrict access to order details to the owning client or authorized administrators.
FR-ORD-07	/api/internal/orders/:id/snapshot	POST	Generate and associate an immutable snapshot of service, plan, and pricing data at order creation time.
FR-ORD-08	/api/orders	POST	Initialize newly created orders with a default pending lifecycle status.
FR-ORD-09	/api/orders/:id/cancel	POST	Prevent cancellation of orders that are not in a pending state.
FR-ORD-10	/api/orders/:id	GET	Handle invalid or non-existent order identifiers with controlled error

			responses.
FR-ORD-11	/api/admin/orders	GET	Restrict administrative order listing operations to authorized administrative users only.
FR-ORD-12	/api/orders	POST	Ensure that pricing or service changes do not affect existing orders through snapshot isolation.
FR-ORD-13	/api/internal/orders/events	POST	Emit internal system events for significant order lifecycle actions such as creation or cancellation.

3.7 REQUIREMENT TRACEABILITY MATRIX (RTM) (IEEE-29148 COMPLIANT)

Standard Reference:

IEEE 29148:2018 – Systems and Software Engineering — Life Cycle Processes — Requirements Engineering

Purpose:

This RTM establishes bi-directional traceability **between** functional requirements**,** system interfaces (APIs)**,** design components**, and** verification evidence**, as required by IEEE-29148.**

1.1.1 RTM – Functional Requirements

Req ID	Requirement Description	Module	API / Interface	Design/Backend Component
FR-AUTH-01	Register user via API	Authentication	/api/auth/register	AuthService
FR-AUTH-04	Send verification email	Email Verification	/api/auth/email/send	EmailService
FR-AUTH-07	Verify email via token	Email Verification	/api/auth/email/verify	EmailTokenService
FR-AUTH-10	Authenticate user	Authentication	/api/auth/login	AuthService
FR-AUTH-11	Refresh authentication tokens	Authentication	/api/auth/refresh	TokenService
FR-AUTH-14	Logout user	Session	/api/auth/logout	SessionService
FR-AUTH-17	Retrieve authenticated user	Authentication	/api/auth/me	AuthController
FR-PASS-01	Request password reset	Password Management	/api/auth/password/request-reset	PasswordResetService
FR-PASS-04	Reset password	Password Management	/api/auth/password/reset	PasswordResetService
FR-	List users	Admin	/api/admin/users	UserService

ADMIN-03		Management		
FR-ADMIN-04	Assign user roles	Admin Management	/api/admin/users/:id/roles	RBACService
FR-ADMIN-05	Deactivate user	Admin Management	/api/admin/users/:id/deactivate	UserService
FR-ADMIN-06	Force logout user	Admin Management	/api/admin/users/:id/logout	SessionService
FR-IMP-01	Start impersonation	Impersonation	/api/auth/impersonate/start	ImpersonationService
FR-IMP-07	Stop impersonation	Impersonation	/api/auth/impersonate/stop	ImpersonationService
FR-IMP-09	Retrieve impersonation status	Impersonation	/api/auth/impersonation-status	ImpersonationService
FR-MFA-01	Setup MFA	MFA	/api/auth/mfa/setup	MFAService
FR-MFA-03	Verify MFA at login	MFA	/api/auth/mfa/verify-login	MFAService
FR-API-01	Create API key	API Keys	/api/auth/api-keys	ApiKeyService
FR-API-05	Revoke API key	API Keys	/api/auth/api-keys/:id	ApiKeyService

FR-IP-01	Create IP rule	IP Access Control	/api/auth/ip-rules	IpAccessService
FR-IP-07	List IP rules	IP Access Control	/api/auth/ip-rules	IpAccessService
FR-AUDIT-01	Log auth events	Audit Logging	Internal	AuditService
FR-WEBHOOK-01	Dispatch webhook events	Webhooks	Internal Dispatcher	WebhookService

FR-ID	**Functional Requirement**	**API Endpoint(s)**	**Design/Backend Component**
FR-BACKUP-01	Support multiple storage providers	/api/backup/providers	Provider Registry
FR-BACKUP-02	Encrypted storage configurations	/api/storage-config	storageConfig.service
FR-BACKUP-03	Validate provider configuration	/api/backup/test-storage	Provider test()
FR-BACKUP-04	File-system backup	/api/backup	streamTarGz
FR-BACKUP-05	Database backup	/api/backup	pgDumpStream
FR-BACKUP-06	Store backup archives	/api/backup	Provider upload
FR-BACKUP-07	Backup logs & metadata	/api/backup/:id/logs	backupStepLog
FR-BACKUP-	Asynchronous	/api/backup	Backup Queue

08	execution		
FR-BACKUP-09	Manual backup trigger	/api/backup	Backup Controller
FR-BACKUP-10	Scheduled backups	Automation Hooks	EventBus
FR-BACKUP-11	Download backups	/api/backup/:id/download	Provider stream
FR-BACKUP-12	Restore system	/api/backup/:id/restore	Restore Runner
FR-BACKUP-13	Async restore	/api/backup/:id/restore	Restore Queue
FR-BACKUP-14	Retention policies	/api/backup/retention	Retention Runner
FR-BACKUP-15	Emit lifecycle events	EventBus	Plugin Hooks
FR-BACKUP-16	Role-based access control	Middleware	RBAC
FR-BACKUP-17	Provider plugins	Plugin API	Provider Registry

FR-ID	**Functional Requirement**	**API Endpoint(s) / Interface**	**Design/Backend Component**
FR-AUTO-01	Manage automation profiles (CRUD)	/api/automation/profiles	profile.controller, profileStore
FR-AUTO-02	Schedule profiles using cron	Cron Scheduler	scheduler.redis.service
FR-AUTO	Enable / disable profiles	/profiles/:id/enable, /profiles/:id/disable	profile.controller, scheduler.redis.service

-03			
FR-AUTO-04	Manage tasks within profiles	/profiles/:profileId/tasks, /tasks/:taskId	task.controller, taskStore
FR-AUTO-05	Execute tasks in defined order	Task ordering logic (order: "asc")	worker.js
FR-AUTO-06	Manual trigger of profiles/tasks	/run/:profileId, /tasks/:taskId/run	run.controller, task.controller
FR-AUTO-07	Asynchronous task execution	Job Queue (BullMQ)	jobQueue, worker, task.controller
FR-AUTO-08	Execute built-in actions	/api/automation/actions	builtInActions.service, executor.service
FR-AUTO-09	Execute plugin actions	Plugin interface (plugin:id:action)	executor.service
FR-AUTO-10	Track status and results	Execution log store	executionLogStore, worker
FR-AUTO-11	Independent worker processes	Worker processes (NODE_ID)	worker.js
FR-AUTO-12	Audit logging for operations	Audit service	audit.service, auditContext

Req ID	Requirement Description	Module	API / Interface	Design / Backend Component
FR-PLUG-01	Automatically load plugins from filesystem	Plugin Engine	Startup / Reload API (Internal)	PluginLoader
FR-PLUG-02	Validate plugin manifest schema	Plugin Engine	/api/plugins/install/zip	ManifestValidatorService
FR-PLUG-03	Register plugin metadata	Plugin Engine	Internal Registry API	PluginRegistry
FR-PLUG-04	Register plugin-defined actions	Plugin Engine	Internal Action Registry	PluginRegistry
FR-PLUG-05	Execute JavaScript plugin actions securely	Plugin Execution	/api/plugins/:pluginId/actions/:actionName	VMExecutor (vm2)
FR-PLUG-06	Serve plugin UI in sandboxed iframe	Plugin UI	/plugins/ui/:pluginId/frame	PluginUI Router + CSP Middleware
FR-	Retrieve	Plugin	/api/plugins	PluginController /

PLUG-07	list of installed plugins	Management		Registry
FR-PLUG-08	Store and retrieve plugin configuration	Plugin Configuration	/api/plugins/:pluginId/config	PluginConfigRoutes + Prisma
FR-PLUG-09	Upload and install plugin via ZIP archive	Plugin Installation	/api/plugins/install/zip	InstallerController
FR-PLUG-10	Invoke plugin actions from automation workflows	Automation Engine	Internal Executor Interface	Automation ExecutorService
FR-PLUG-11	Execute WebAssembly-based plugins	Plugin Execution	/api/plugins/:pluginId/actions/:actionName	WASMExecutor

Req ID	**Requirement Description**	**Module**	**API / Interface**	**Design / Backend Component**
FR-ATPL-01	Use plugin actions as automation tasks	Automation	Internal Action Dispatcher	ExecutorService
FR-	Manually	Plugin	/api/plugins/:pluginId/te	PluginConfigRoutes

ATPL -02	trigger plugin actions	Management	st-action	
FR-ATPL -03	Execute chained plugin actions in automation	Automation	Automation Task Pipeline	Automation Engine
FR-ATPL -04	Embed plugin UI in automation configuration	Automation UI	/plugins/ui/:pluginId/frame	Plugin UI Sandbox

Req ID	**Requirement Description**	**Module**	**API / Interface**	**Design / Backend Component**
FR-DOM -01	Check domain availability via external registrar	Domain Management	/api/domains/check-availability	DomainService → RegistrarAdapter
FR-	Perform	Domain	/api/domains/check-	ActiveRegistrarResolver

DOM -02	real-time availability checks for supported TLDs	Management	availability	
FR-DOM -03	Return structured availability data (status, premium, pricing)	Domain Management	/api/domains/check-availability	AvailabilityResponseMapper
FR-DOM -04	Validate domain name format before availability check	Domain Management	Internal Validator	DomainValidator
FR-DOM -05	Normalize registrar-specific responses into standard format	Domain Management	Internal Mapping Layer	RegistrarAdapter
FR-DOM -06	Register available domains via registrar API	Domain Management	/api/domains/register	DomainRegistrationService
FR-DOM	Persist registered	Domain Management	/api/domains/register	DomainRepository

-07	domain metadata			
FR-DOM-08	Store registration and expiry dates	Domain Management	Internal Persistence	DomainRepository
FR-DOM-09	Initiate domain transfer using authorization code	Domain Management	/api/domains/transfer	DomainTransferService
FR-DOM-10	Track domain lifecycle state	Domain Management	Internal Lifecycle Handler	DomainLifecycleService
FR-DOM-11	Expose domain lifecycle information	Admin Domain Management	/api/admin/domains/:id	DomainQueryService
FR-DOM-12	Integrate registrars via provider-based abstraction	Domain Management	Registrar Provider Interface	RegistrarFactory
FR-DOM-13	Enable or disable registrar providers via	Provider Management	/api/admin/providers/:id	ProviderConfigService

	configuration			
FR-DOM-14	Securely store registrar API credentials in encrypted form	Provider Management	Internal Encryption Layer	CredentialVault
FR-DOM-15	Route all domain operations through active registrar	Domain Management	Registrar Resolver	ActiveRegistrarResolver
FR-DOM-16	Allow future registrar integrations without API changes	Domain Management	Provider Interface	RegistrarAdapter Contract
FR-DOM-17	Retrieve domain pricing from registrar APIs	Domain Pricing	/api/domains/pricing/sync	PricingSyncService
FR-DOM-18	Synchronize TLD pricing into local database	Domain Pricing	/api/domains/pricing/sync	TldPricingRepository
FR-	Store	Domain	Internal Pricing	TldPricingRepository

DOM -19	registration, renewal, and transfer prices per TLD	Pricing	Model	
FR-DOM -20	Configure markup percentage on registrar pricing	Domain Pricing	/api/admin/tlds/:id	PricingMarkupService
FR-DOM -21	Calculate final domain prices dynamically	Domain Pricing	Internal Pricing Engine	PricingCalculator
FR-DOM -22	Enable or disable individual TLDs	Domain Pricing	/api/admin/tlds/:id	TldManagementService
FR-DOM -23	Retrieve list of all managed domains	Admin Domain Management	/api/admin/domains	DomainQueryService
FR-DOM -24	View detailed domain metadata	Admin Domain Management	/api/admin/domains/:id	DomainQueryService
FR-DOM -25	Soft delete domain records	Admin Domain Management	/api/admin/domains/:id	DomainDeletionService
FR-DOM	Record domain-	Audit Logging	Internal Audit Hook	AuditLogger

-26	related admin actions in audit logs			
FR-DOM-27	Restrict domain APIs using RBAC	Security	Auth Middleware	AccessControlGuard
FR-DOM-28	Expose domain APIs for automation workflows	Automation	/api/domains/*	Automation-Compatible APIs
FR-DOM-29	Trigger domain-related automation actions	Automation	/api/jobs/domain-sync	AutomationEngine
FR-DOM-30	Emit events on critical domain lifecycle changes	Event System	Internal Event Bus	DomainEventEmitter

Req ID	**Requirement Description**	**Module**	**API / Interface**	**Design / Backend Component**
FR-	Create a	Services	/api/admin/services	ServiceService

SERV -01	new hosting service definition			
FR-SERV -02	Retrieve list of all services (admin view)	Services	/api/admin/services	ServiceService
FR-SERV -03	Retrieve details of a specific service	Services	/api/admin/services/:id	ServiceService
FR-SERV -04	Update service metadata and activation status	Services	/api/admin/services/:id	ServiceService
FR-SERV -05	Deactivate a service (soft delete)	Services	/api/admin/services/:id	ServiceService
FR-SERV -06	Retrieve active services for clients	Services	/api/client/services	ServiceService
FR-SERV -07	Create a plan under a service	Service Plans	/api/admin/services/:serviceId /plans	ServicePlanService
FR-SERV -08	Update service plan metadata	Service Plans	/api/admin/plans/:planId	ServicePlanService

FR-SERV-09	Define pricing for a service plan	Service Pricing	/api/admin/plans/:planId/pricing	ServicePricingService
FR-SERV-10	Enforce unique pricing per billing cycle	Service Pricing	Database Constraint	ServicePricingService
FR-SERV-11	Define policy rules for a service plan	Service Policies	/api/admin/plans/:planId/policies	ServicePolicyService
FR-SERV-12	Generate immutable service snapshot	Services (Internal)	Internal Utility	ServiceSnapshotService
FR-SERV-13	Restrict service management to admin users	Services	Middleware (RBAC)	RBACService
FR-SERV-14	Handle invalid service, plan, or pricing requests	Services	API Error Handling	ServiceController
FR-SERV-15	Preserve historical integrity during pricing	Services	Snapshot Mechanism	ServiceSnapshotService

	changes			

Req ID	Requirement Description	Module	API / Interface	Backend Component
FR-ORD-01	Create a new service order for an authenticated client	Orders	/api/orders	Order.Service
FR-ORD-02	Generate and persist an immutable snapshot of service, plan, and pricing data at order creation	Orders	Internal Snapshot Utility	Order.Snapshot.Service
FR-ORD-03	Retrieve all orders belonging to the authenticated client	Orders	/api/orders	Order.Service
FR-ORD-04	Retrieve details of a specific order owned by the authenticated client	Orders	/api/orders/:id	Order.Service
FR-ORD-05	Restrict clients from	Orders	Internal Access Control	OrderService

	accessing orders belonging to other users			
FR-ORD-06	Cancel a pending order initiated by the owning client	Orders	/api/orders/:id/cancel	OrderService
FR-ORD-07	Prevent cancellation of orders that are not in a pending state	Orders	Internal Validation	OrderService
FR-ORD-08	Initialize newly created orders with a default pending lifecycle status	Orders	Internal	OrderService
FR-ORD-09	Retrieve a list of all orders for administrative oversight	Orders	/api/admin/orders	OrderService
FR-ORD-10	Allow administrators to retrieve details of any client order	Orders	/api/admin/orders/:id	OrderService

FR-ORD-11	Restrict administrative order management APIs to authorized roles only	Orders	RBAC Middleware	RBACService
FR-ORD-12	Ensure service or pricing changes do not affect existing orders through snapshot isolation	Orders	Internal	OrderSnapshotService
FR-ORD-13	Handle invalid or non-existent order identifiers with controlled error responses	Orders	Internal Error Handling	OrderController
FR-ORD-14	Record order creation and cancellation events within the audit logging system	Orders	Internal	AuditService

IEEE-29148 Compliance Statement

Forward traceability:

Each requirement maps to a concrete API and backend component

Backward traceability:

Every major API endpoint traces back to at least one documented requirement.

Verification traceability:

All requirements are verifiable through database state, logs, or API responses.

Architecture alignment:

All requirements are implemented through **stateless APIs**, confirming an **API-first, headless system design**.

3.8 (SQA) – DEFECT DETECTION

Chapter 4:

System Design

Chapter 4

System Design

4.1 INTRODUCTION

System design provides a structural and architectural view of how the proposed Web Hosting Management System (WHMS) satisfies the functional and non-functional requirements defined in Chapter 3. This chapter focuses on translating abstract requirements into concrete architectural constructs, design components, interfaces, and data models that collectively define the system's internal organization and interaction patterns.

The design of WHMS follows an API-first and headless architectural approach, where all system capabilities are exposed through well-defined RESTful APIs, and no core business logic is embedded within user interface components. As a result, the system design emphasizes service boundaries, execution flow, data relationships, and interaction contracts, rather than user-interface-driven workflows. This design philosophy directly influences the choice of modeling techniques used in this chapter.

In accordance with established software engineering literature, no single diagram type is sufficient to represent all aspects of a complex software system. Booch, Rumbaugh, and Jacobson emphasize that UML diagrams represent views of a system rather than the system itself, and that large systems should be documented using multiple complementary views at appropriate levels of abstraction. Similarly, IEEE-29148:2018 recommends that system models be selected to maximize clarity and stakeholder understanding while avoiding unnecessary redundancy or over-specification.

Based on these principles, this chapter adopts a selective and purpose-driven modeling strategy. High-level architectural concerns are represented using an architecture diagram that illustrates system layers, module boundaries, execution environments, and external integrations. Structural relationships and persistent data design are documented using domain-specific Entity Relationship Diagrams (ERDs), rather than a single monolithic ERD, to reflect the modular nature of the system. Behavioral aspects are primarily captured using use case diagrams, sequence diagrams, and fully dressed use cases, which are more suitable for API-driven, stateless systems.

Activity diagrams are intentionally limited within this design chapter. While activity diagrams are effective for modeling human-centric or UI-driven workflows, their extensive use in an API-first, headless system can introduce redundancy and reduce clarity. Sommerville (2015) notes that behavior already defined through formal interfaces, execution sequences, and requirements specifications does not benefit from repetitive workflow modeling. Therefore, activity diagrams are included only for architecturally significant, multi-step lifecycle processes, such as plugin installation and execution, where fault handling, validation, and transactional behavior must be explicitly illustrated.

This design approach ensures that each diagram included in this chapter serves a clear and justifiable purpose, directly supports traceability to functional requirements, and aligns with modern software architecture documentation practices. By avoiding diagram overuse and focusing on clarity, modularity, and correctness, the system design remains both academically rigorous and practically representative of the implemented system.

4.2 ARCHITECTURE DIAGRAM

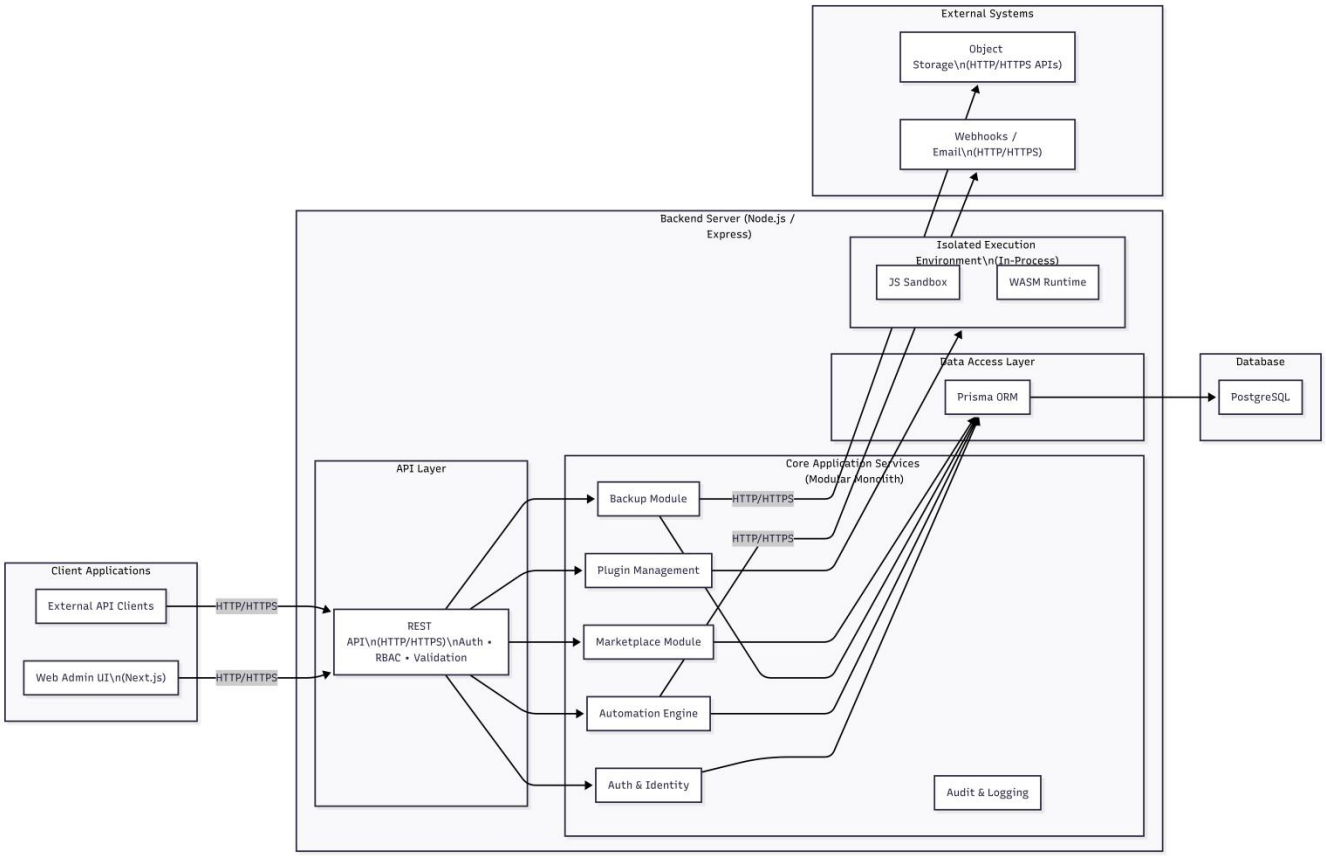


Fig 4.1: Architecture Diagram

4.3 DETAILED DESIGN

4.3.1 Use Case Diagrams

4.3.1.1 USE CASE DIAGRAM

Scope: Authentication & Identity Management API (System)

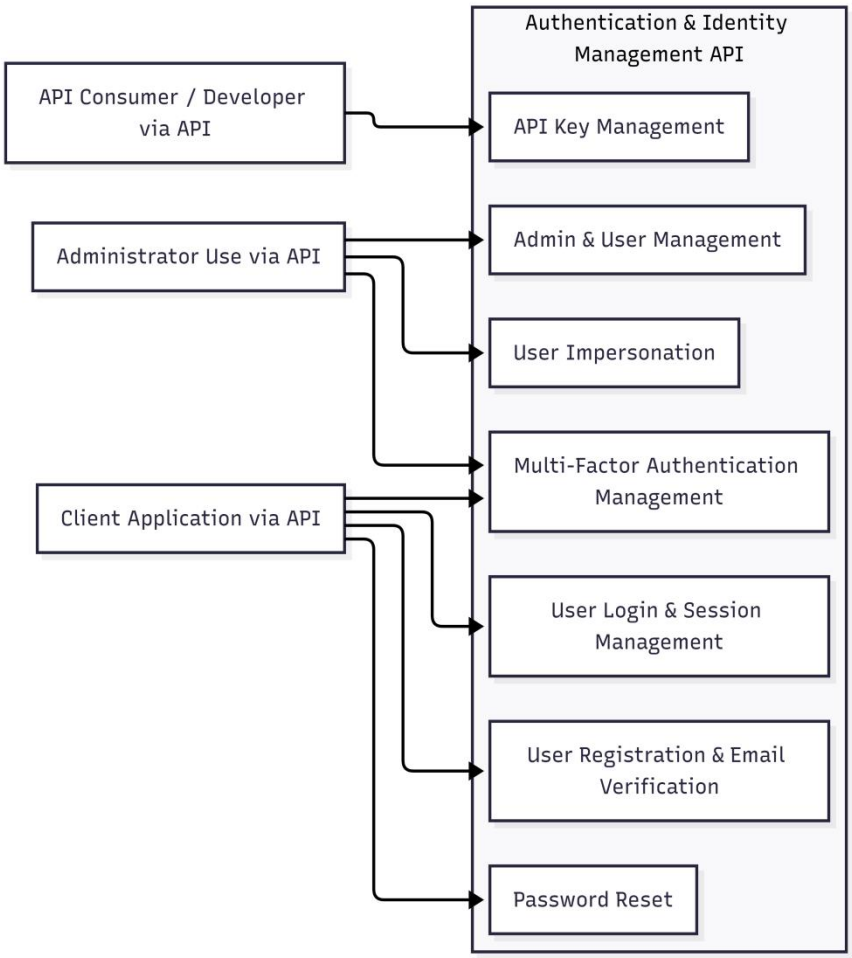


Fig 4.2: Use Case Diagram

4.3.1.2 USE CASE DIAGRAM — Backup Module

Note: All actors interact with the Backup Management API exclusively through authenticated API requests. No user-interface logic is embedded within the system.

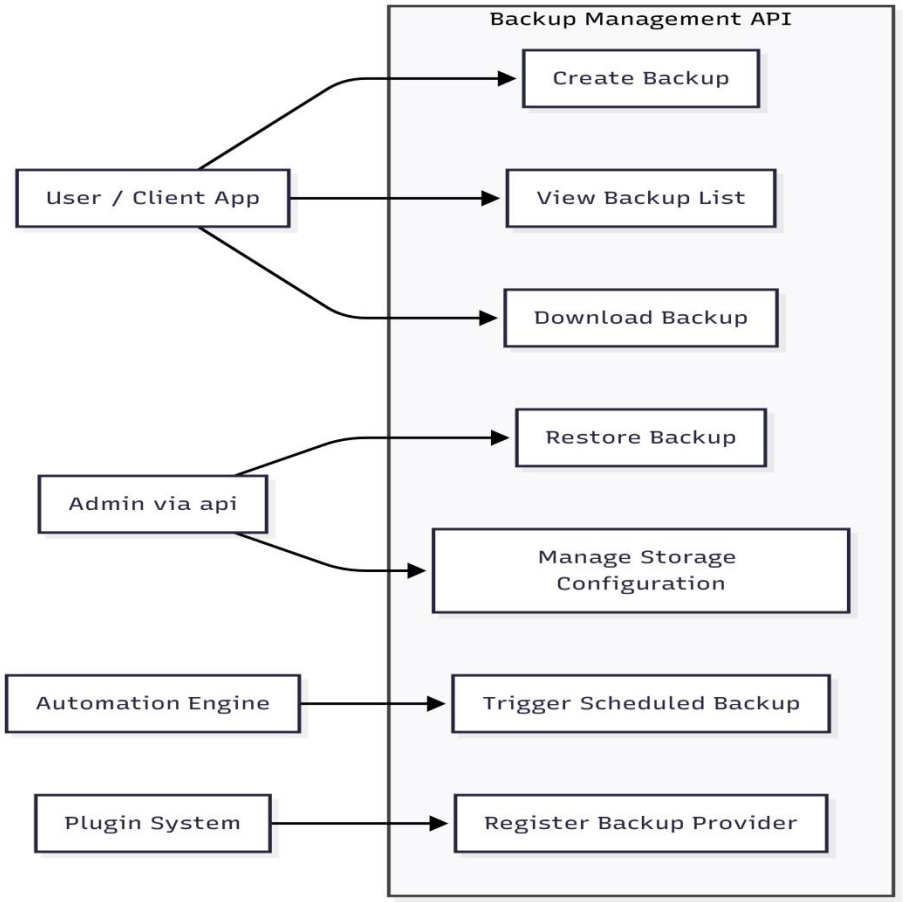


Fig 4.3: Use Case Diagram

4.3.1.3 USE CASE DIAGRAM — Plugin Module

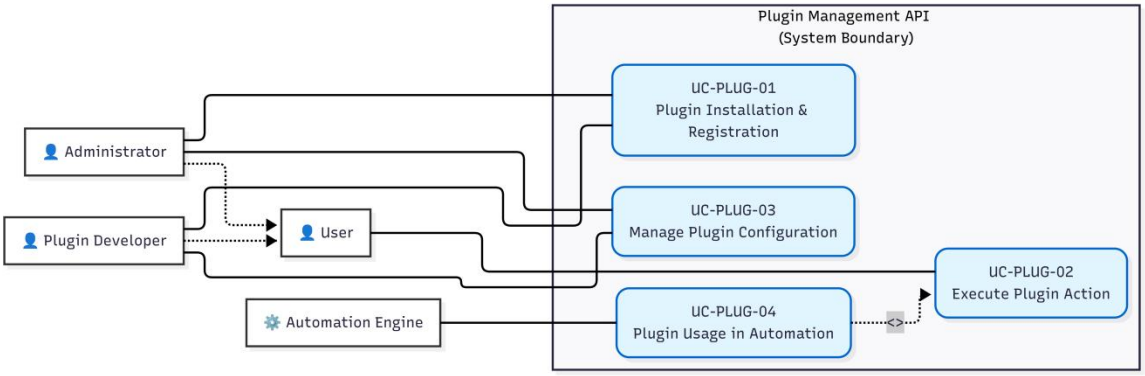


Fig 4.4: Use Case Diagram

4.3.2 FR–UC Traceability Matrix

Only architecturally significant, multi-step workflows are modeled as fully dressed use cases. Supporting, policy-driven, and cross-cutting behaviors are specified directly through functional requirements.

User Registration & Email Verification

FR ID	UC ID
FR-AUTH-01	UC-AUTH-01
FR-AUTH-02	UC-AUTH-01
FR-AUTH-03	UC-AUTH-01
FR-AUTH-04	UC-AUTH-01
FR-AUTH-05	UC-AUTH-01
FR-AUTH-06	UC-AUTH-01
FR-AUTH-07	UC-AUTH-01
FR-AUTH-08	UC-AUTH-01
FR-AUTH-09	UC-AUTH-01
FR-AUTH-10	UC-AUTH-02
FR-AUTH-11	UC-AUTH-02
FR-AUTH-12	UC-AUTH-02
FR-AUTH-13	UC-AUTH-02
FR-AUTH-14	UC-AUTH-02
FR-AUTH-15	UC-AUTH-03

FR-AUTH-16	UC-AUTH-03
FR-AUTH-17	UC-AUTH-02
FR-PASS-01	UC-PASS-01
FR-PASS-02	UC-PASS-01
FR-PASS-03	UC-PASS-01
FR-PASS-04	UC-PASS-01
FR-PASS-05	UC-PASS-01
FR-PASS-06	UC-PASS-01
FR-PASS-07	UC-PASS-01
FR-PROFILE-01	UC-AUTH-02
FR-PROFILE-02	UC-AUTH-02
FR-PROFILE-03	UC-AUTH-02
FR-ADMIN-01	UC-ADMIN-01
FR-ADMIN-02	UC-ADMIN-01
FR-ADMIN-03	UC-ADMIN-01
FR-ADMIN-04	UC-ADMIN-01
FR-ADMIN-05	UC-ADMIN-01
FR-ADMIN-06	UC-ADMIN-01
FR-ADMIN-07	UC-ADMIN-01
FR-IMP-01	UC-IMP-01
FR-IMP-02	UC-IMP-01
FR-IMP-03	UC-IMP-01
FR-IMP-04	UC-IMP-01
FR-IMP-05	UC-IMP-01
FR-IMP-06	UC-IMP-01
FR-IMP-07	UC-IMP-01
FR-IMP-08	UC-IMP-01
FR-IMP-09	UC-IMP-01
FR-MFA-01	UC-MFA-01
FR-MFA-02	UC-MFA-01

FR-MFA-03	UC-MFA-01
FR-MFA-04	UC-MFA-01
FR-MFA-05	UC-MFA-01
FR-MFA-06	UC-MFA-01
FR-API-01	UC-API-01
FR-API-02	UC-API-01
FR-API-03	UC-API-01
FR-API-04	UC-API-01
FR-API-05	UC-API-01
FR-API-06	UC-API-01
FR-RBAC-01	UC-ADMIN-01
FR-RBAC-02	UC-ADMIN-01
FR-RBAC-03	UC-ADMIN-01
FR-RBAC-04	UC-ADMIN-01
FR-RBAC-05	UC-ADMIN-01
FR-IP-01	UC-ADMIN-01
FR-IP-02	UC-ADMIN-01
FR-IP-03	UC-AUTH-02
FR-IP-04	UC-ADMIN-01
FR-AUDIT-01	UC-AUTH-02
FR-AUDIT-02	UC-AUTH-02
FR-AUDIT-03	UC-IMP-01
FR-AUDIT-04	UC-PASS-01
FR-AUDIT-05	UC-ADMIN-01
FR-WEBHOOK-01	UC-AUTH-02
FR-WEBHOOK-02	UC-AUTH-02
FR-WEBHOOK-03	UC-AUTH-02
FR-WEBHOOK-04	UC-AUTH-02
FR-WEBHOOK-05	UC-AUTH-02

FR ID	UC ID
FR-BACKUP-01 – FR-BACKUP-06	UC-BACKUP-01
FR-BACKUP-07 – FR-BACKUP-19	UC-BACKUP-02
FR-BACKUP-20 – FR-BACKUP-25	UC-BACKUP-03
FR-BACKUP-26 – FR-BACKUP-29	UC-BACKUP-04
FR-BACKUP-30 – FR-BACKUP-32	UC-BACKUP-02 / UC-BACKUP-03
FR-BACKUP-33 – FR-BACKUP-35	UC-BACKUP-01

FR ID	UC ID
FR-AUTO-01	UC-AUTO-01
FR-AUTO-03	UC-AUTO-02
FR-AUTO-04-FR-AUTO-05	UC-AUTO-03
FR-AUTO-06-FR-AUTO-07	UC-AUTO-04
FR-AUTO-10	UC-AUTO-05

FR ID Range	UC ID
FR-PLUG-01 – FR-PLUG-04	UC-PLUG-01
FR-PLUG-05 – FR-PLUG-06	UC-PLUG-02
FR-PLUG-07 – FR-PLUG-09	UC-PLUG-03
FR-PLUG-10 – FR-PLUG-11	UC-PLUG-04

FR ID Range	UC ID
FR-ATPL-01 – FR-ATPL-02	UC-AUTO-01
FR-ATPL-03	UC-AUTO-02
FR-ATPL-04	UC-AUTO-03

4.3.3 Fully Dresses Use Cases

Table 4.3.3.1.: Fully Dressed Use Case – UC-AUTH-01

Field	**Description**	
Use Case ID	UC-AUTH-01	
Use Case Name	User Registration & Email Verification	
Primary Actor	Client Application (Web UI, Mobile App, API Consumer)	
Secondary Actors	Email Service	
System Boundary	Authentication & Identity Management API	
Trigger	Client application submits a registration request via public API	
Preconditions	1. User is not already registered. 2. Registration API endpoint is publicly accessible.	
Postconditions (Success)	1. User account is created with a default role. 2. Email verification status is set to unverified. 3. Verification email is sent to the user. 4. Verification token is generated and stored.	
Main Success Flow	**Actor**	**System**
	1. Client submits registration data (email, password) via API.	2. System validates request payload and email uniqueness. 3. System creates user account and assigns default role. 4. System generates email verification token. 5. System sends verification email asynchronously.

		6. System returns registration success response.
Alternative flow	**Actor**	**System**
		A1 – Duplicate Email A1.1 System detects existing email. A1.2 System rejects registration request. A1.3 System returns error response.
	A2.1 Client requests resend via API.	**A2 – Resend Verification Email** A2.2 System generates new verification token. A2.3 System invalidates previous tokens. A2.4 System sends new verification email.
Exceptional Flow	**E1 – Invalid or Expired Token** E1.1 System detects invalid or expired token. E1.2 System rejects verification request. E1.3 System returns error response.	
Business Rules	1. Email addresses must be unique. 2. Verification tokens are time-bound. 3. Email verification is required before sensitive actions.	
Related Functional Requirements	FR-AUTH-01, FR-AUTH-02, FR-AUTH-03, FR-AUTH-04, FR-AUTH-05, FR-AUTH-06, FR-AUTH-07, FR-AUTH-08,	

	FR-AUTH-09

Table 4.3.3.2: Fully Dressed Use Case – UC-AUTH-02

Field	**Description**	
Use Case ID	UC-AUTH-02	
Use Case Name	User Login & Session Management	
Primary Actor	Client Application (Web UI, Mobile App, API Consumer)	
Secondary Actors	None	
System Boundary	Authentication & Identity Management API	
Trigger	Client application submits login credentials via API	
Preconditions	1. User account exists. 2. User account is not disabled. 3. Login API endpoint is accessible.	
Postconditions (Success)	1. User is authenticated successfully. 2. Access and Refresh tokens are issued. 3. A new session is created and stored. 4. User roles, permissions, and portal access are resolved. 5. MFA challenge is initiated if required by policy.	
Main Success Flow	**Actor Actions**	**System Response**
	1. Client application submits email and password via login API.	2. System validates credentials. 3. System evaluates account status and security policies (email verification, IP rules, role policies). 4. If MFA is not required, system issues access and refresh tokens. 5. System creates a persistent session with IP and user-agent metadata.

		6.System returns authentication response to the client.
Alternate Flow	**Actor**	**System**
		A1.1 System detects MFA is enabled or required by policy. A1.2 System returns an MFA challenge response. A1.3 Client submits MFA code via MFA verification API. A1.4 System validates the MFA code. A1.5 System issues access and refresh tokens. A1.6 System creates a new session.
Exceptional flow	**E1-INVALID CREDENTIALS** E1.1 System detects invalid email or password. E1.2. System rejects the login request. E1.3. System returns an authentication error response. **E2-DISABLED ACCOUNT** E2.1. System detects the account is disabled. E2.2. System blocks authentication. E2.3. System returns an access denied response. **E3-IP RESTRICTED LOGIN** E3.1. System detects login attempts from a blocked IP. E3.2. System denies authentication. E3.3. System returns an IP restriction error.	
Business rules	1. Authentication is performed via APIs only. 2. Multiple concurrent sessions are allowed	

	3. MFA is enforced based on user and role policies. 4. Sessions must be revocable by users and administrators.
Related Functional Requirements	FR-AUTH-10, FR-AUTH-11, FR-AUTH-12, FR-AUTH-13, FR-AUTH-14, FR-AUTH-17, FR-PROFILE-01, FR-PROFILE-02, FR-PROFILE-03, FR-IP-03, FR-AUDIT-01, FR-AUDIT-02, FR-WEBHOOK-01

Table 4.3.3.3.: Fully Dressed Use Case – UC-PASS-01

Field	**Description**	
Use Case ID	UC-PASS-01	
Use Case Name	Password Reset	
Primary Actor	Client Application (Web UI, Mobile App, API Consumer)	
Secondary Actors	Email Service	
System Boundary	Authentication & Identity Management API	
Trigger	Client application submits a password reset request via API	
Preconditions	1. User account exists. 2. Password reset API endpoint is publicly accessible.	
Postconditions (Success)	1. Password reset token is generated and stored. 2. Password reset email is sent to the user. 3.User password is updated successfully upon token validation. 4. All active sessions are invalidated after password reset.	
Main Success Flow	**Actor**	**System**

	1. Client submits password reset request with registered email via API.	2. System validates email and generates a time-bound reset token. 3. System sends password reset email asynchronously. 4. Client submits new password with valid reset token via API. 5. System validates reset token and updates user password. 6. System invalidates all active user sessions. 7. System returns password reset success response.
Alternative Flow	**Actors**	**System**
		A1 – Invalid Email A1.1 System detects email does not exist. A1.2 System rejects password reset request. A1.3 System returns error response. **A2 – Expired or Invalid**

		Token A2.1 System detects expired or invalid reset token. A2.2 System rejects password reset attempt. A2.3 System returns error response.
Exceptional Flow	**E1 – Reset Link Reuse Attempt** E1.1 System detects previously used reset token. E1.2 System rejects password reset request. E1.3 System returns error response.	
Business Rules	1. Password reset operations are API-based only. 2. Reset tokens are single-use and time-bound. 3. Password reset invalidates all active sessions. 4. Email delivery is handled asynchronously.	
Related Functional Requirements	FR-PASS-01, FR-PASS-02, FR-PASS-03, FR-PASS-04, FR-PASS-05, FR-PASS-06, FR-PASS-07, FR-AUDIT-07, FR-WEBHOOK-06	

Table 4.3.3.4.: Fully Dressed Use Case – UC-ADMIN-01

Field	Description
Use Case ID	UC-ADMIN-01
Use Case Name	Admin & User Management
Primary Actor	Administrator (via Admin Client Application / API Consumer)
Secondary Actors	None
System Boundary	Administration & Identity Management API
Trigger	Administrator submits a management request via secured admin API
Preconditions	1. Administrator is authenticated.

	2. Administrator has required roles and permissions. 3. Admin API endpoints are accessible.	
Postconditions (Success)	1. User data is retrieved or modified successfully. 2. User roles and permissions are updated if requested. 3. User account status is updated if requested. 4. User sessions are revoked if requested. 5. Administrative action is recorded in audit logs.	
Main Success Flow	**Actor**	**System**
		1. Administrator requests list of users via admin API. 2. System retrieves user accounts and related metadata. 3. Administrator selects a user and submits management action (update role, deactivate, revoke sessions). 4. System validates administrator permissions. 5. System applies requested changes to the user account. 6. System records the action in audit logs. 7. System returns success response to administrator.
Alternate Flow	**Actor**	**System**
	A1.1 Administrator submits role update request for a user.	**A1 – Role Assignment or Update** A1.2 System validates role and permission mappings. A1.3 System updates user role assignments.

	A2.1 Administrator submits session revocation request.	**A2 – Force Logout User** A2.2 System invalidates all active sessions for the user.
Exceptional Flow	**E1 – Insufficient Permissions** E1.1 System detects administrator lacks required permission. E1.2 System rejects the request. E1.3 System returns authorization error response. **E2 – User Not Found** E2.1 System detects invalid user identifier. E2.2 System rejects the request. E2.3 System returns not found error response.	
Business Rules	1. All administrative operations are protected by RBAC. 2. Only authorized administrators can manage users. 3. Role changes take effect immediately. 4. Administrative actions must be auditable.	
Related Functional Requirements	FR-ADMIN-01, FR-ADMIN-02, FR-ADMIN-03, FR-ADMIN-04, FR-ADMIN-05, FR-ADMIN-06, FR-ADMIN-07, FR-RBAC-01, FR-RBAC-02, FR-RBAC-03, FR-RBAC-04, FR-RBAC-05, FR-AUDIT-08	
Notes	This use case is API-first and headless. Administrative functionality is exposed exclusively through secured APIs, independent of any UI implementation.	

Table 4.3.3.5.: Fully Dressed Use Case – UC-IMP-01

Field	**Description**
Use Case ID	UC-IMP-01
Use Case Name	User Impersonation
Primary Actor	Administrator (via Admin Client Application / API Consumer)
Secondary Actors	None

System Boundary	Authentication & Impersonation Management API	
Trigger	Administrator submits an impersonation request via secured API	
Preconditions	1. Administrator is authenticated. 2. Administrator has impersonation permission. 3. Target user account exists and is active.	
Postconditions (Success)	1. Impersonation session is created successfully. 2. Impersonation-specific access and refresh tokens are issued. 3. Impersonator identifier is attached to session context. 4. Impersonation activity is recorded in audit logs.	
Main Success Flow	**Actor Actions**	**System Actions**
	1.Administrator submits an impersonation request via the secured API endpoint, providing the target user identifier and a mandatory impersonation reason.	
		2. System authenticates the administrator's session and evaluates role-based permissions to confirm that impersonation is authorized for the requesting administrator. 3. System validates the target user account, ensuring the user exists, is active, and is eligible for impersonation.

		4. System creates a new impersonation session, distinct from standard user sessions, and marks it explicitly as an impersonation context. 5. System generates impersonation-specific access and refresh tokens, binding them to the newly created impersonation session. 6. System attaches impersonation metadata to the session context, including the impersonator identifier, impersonation reason, and session linkage. 7. System records the impersonation start event in the audit log, capturing administrator identity, target user identity, timestamp, IP address, and user-agent metadata. 8. System returns a successful impersonation response, containing the impersonation access and refresh tokens to the administrator. 9. System returns a successful impersonation response, containing the impersonation access and refresh tokens to the administrator.

	A1 – Stop Impersonation A1.1 Administrator submits stop impersonation request via API.	A1.2 System identifies active impersonation session. A1.3 System terminates impersonation session. A1.4 System restores administrator's original session context.
Exceptional Flow	**E1 – Unauthorized Impersonation Attempt** E1.1 System detects missing or insufficient impersonation permission. E1.2 System rejects impersonation request. E1.3 System returns authorization error response. **E2 – Invalid Target User** E2.1 System detects invalid or inactive target user. E2.2 System rejects impersonation request. E2.3 System returns error response.	
Business Rules	1. Only authorized administrators may impersonate users. 2. Impersonation sessions are isolated from standard user sessions. 3. All impersonation actions must be auditable. 4. Impersonation can be terminated at any time by the administrator.	
Related Functional	FR-IMP-01, FR-IMP-02, FR-IMP-03, FR-IMP-04, FR-IMP-	

Requirements	05, FR-IMP-06, FR-IMP-07, FR-IMP-08, FR-IMP-09, FR-AUDIT-04, FR-AUDIT-05
Notes	This use case is API-first and headless. Impersonation is controlled exclusively via secure APIs and is independent of any UI implementation.

Table 4.3.3.6.: Fully Dressed Use Case – UC-MFA-01

Field	**Description**	
Use Case ID	UC-MFA-01	
Use Case Name	Multi-Factor Authentication (MFA) Management	
Primary Actor	Client Application (Web UI, Mobile App, API Consumer)	
Secondary Actors	None	
System Boundary	Authentication & Security Management API	
Trigger	Client application submits an MFA-related request via API	
Preconditions	1. User is authenticated. 2. User account exists and is active. 3. MFA management API endpoints are accessible.	
Post-conditions (Success)	1. MFA is enabled or disabled successfully as requested. 2. MFA secrets and recovery data are securely stored. 3. Trusted devices are registered if provided. 4. MFA configuration changes are recorded.	
Main Success Flow	**Actor**	**System**
	1. Client requests MFA setup via API.	
		2. System generates MFA secret and provisioning data.
	3. Client submits MFA verification code.	
		4. System validates the MFA code.

		5. System enables MFA for the user account. 6. System returns MFA setup success response
Alternative flow	**Actor**	**System**
	A1 – Disable MFA A1.1 Client submits MFA disable request via API.	
		A1.2 System validates user authentication A1.3 System disables MFA for the user account.
	A2 – Trusted Device Registration A2.1 Client submits trusted device information.	
		A2.2 System stores trusted device fingerprints for future logins.

Exceptional Flow	**E1 – Invalid MFA Code** E1.1 System detects invalid or expired MFA code. E1.2 System rejects MFA verification request. E1.3 System returns error response. **E2 – MFA Not Configured** E2.1 System detects MFA is not enabled for the user. E2.2 System rejects disable or verification request. E2.3 System returns error response.	
Business Rules	1. MFA operations are performed via APIs only. 2. MFA secrets must be stored securely. 3. MFA verification is required during login when enabled. 4. Trusted devices may bypass MFA based on policy.	
Related Functional Requirements	FR-MFA-01, FR-MFA-02, FR-MFA-03, FR-MFA-04, FR-MFA-05, FR-MFA-06, FR-AUDIT-01.	
Notes	This use case is API-first and headless. MFA configuration and enforcement are handled entirely through secure APIs without UI dependency.	

Table 4.3.3.7.: Fully Dressed Use Case – UC-API-01

Field	**Description**
Use Case ID	UC-API-01
Use Case Name	API Key Management
Primary Actor	Client Application (Web UI, Developer Tool, API Consumer)
Secondary Actors	None
System Boundary	Authentication & API Access Management API
Trigger	Client application submits an API key management request via API
Preconditions	1. User is authenticated.

	2. User has permission to manage API keys. 3. API key management endpoints are accessible.	
Postconditions (Success)	1. API key is created, listed, regenerated, or revoked as requested. 2. API key is securely stored in hashed form. 3. API key permissions are applied. 4. API key activity is logged.	
Main Success Flow	**Actor**	**System**
	1. Client requests creation of a new API key via API.	2. System validates user permissions. 3. System generates a new API key. 4. System stores the API key securely in hashed form. 5. System returns the API key metadata and secret once to the client.
Alternative flow	**Actor**	**System**
	A1 – List API Keys A1.1 Client requests list of existing API keys. **A2 – Regenerate API Key** A2.1 Client submits API key regeneration request.	A2.2 System invalidates old API key. A2.3 System generates and stores a new API key.

	A3 – Revoke API KeyA3.1 Client submits API key revocation request.	A3.2 System revokes the API key and blocks further usage.
Exceptional Flow	**E1 – Unauthorized Access** E1.1 System detects missing or insufficient permissions. E1.2 System rejects API key management request. E1.3 System returns authorization error response. **E2 – Invalid API Key Identifier** E2.1 System detects invalid API key reference. E2.2 System rejects request. E2.3 System returns error response.	
Business Rules	1. API keys are used for programmatic access only. 2. API keys must be stored securely using hashing. 3. Revoked API keys must be immediately invalidated.	

	4. API key usage must be auditable.
Related Functional Requirements	FR-API-01, FR-API-02, FR-API-03, FR-API-04, FR-API-05, FR-API-06
Notes	This use case is API-first and headless. All API key lifecycle operations are exposed exclusively through secure APIs and are independent of any UI implementation.

Table 4.3.3.8.: Fully Dressed Use Case – UC-IP-01

Field	**Description**	
Use Case ID	UC-IP-01	
Use Case Name	IP Access Control Management	
Primary Actor	Administrator (via Admin Client Application / API Consumer)	
Secondary Actors	None	
System Boundary	Security & Access Control API	
Trigger	Administrator submits an IP rule management request via API	
Preconditions	1. Administrator is authenticated 2. Administrator has permission to manage IP rules. 3. IP access control API endpoints are accessible.	
Postconditions (Success)	1. IP allow or deny rule is created, updated, or deleted successfully. 2. IP access rules are persisted and activated immediately. 3. Changes are recorded for audit purposes.	
Main Success Flow	**Actor**	**System**
	1. Administrator submits request to create an IP access rule via API.	2. System validates administrator permissions. 3. System validates IP format (single IP, wildcard, or CIDR). 4. System stores the IP access rule.

		5. System applies the rule to authentication and authorization flows. 6. System returns success response.
Alternative flow	**Actor**	**System**
	A1 – Delete IP Rule A1.1 Administrator submits IP rule deletion request. **A2 – List IP Rules** A2.1 Administrator requests list of IP access rules.	A1.2 System validates request and removes the IP rule. A2.2 System retrieves and returns all configured rules.

Exceptional Flow	**E1 – Invalid IP Format** E1.1 System detects invalid IP or CIDR format. E1.2 System rejects rule creation request. E1.3 System returns validation error response. **E2 – Unauthorized Access** E2.1 System detects insufficient permissions. E2.2 System rejects request. E2.3 System returns authorization error response.
Business Rules	1. IP access rules are enforced during authentication and authorization. 2. Both allow-list and deny-list modes are supported. 3. Changes to IP rules take effect immediately. 4. IP rule modifications must be auditable.
Related Functional Requirements	FR-IP-01, FR-IP-02, FR-IP-03, FR-IP-04, FR-IP-05, FR-IP-06, FR-IP-07, FR-AUDIT-09
Notes	This use case is API-first and headless. IP access control is managed exclusively through secure APIs and applies uniformly across all client applications.

Table 4.3.3.9.: Fully Dressed Use Case – UC-BACKUP-02

Field	**Description**
Use Case ID	UC-BACKUP-02
Use Case Name	Backup Creation, Execution & Download
Primary Actor	Authenticated User
Secondary Actors	Job Queue, Storage Provider, Event System
System Boundary	Backup Execution API
Trigger	User submits a backup creation request via API
Preconditions	1. User is authenticated. 2. User has permission to create backups. 3. A valid storage configuration exists.

Postconditions (Success)	1. Backup job is queued for asynchronous execution. 2. Backup archive is created and stored. 3. Backup metadata and logs are persisted. 4. Backup lifecycle events are emitted.	
Main Success Flow	**Actor**	**System**
	1.User submits backup request via API. 5. Worker executes file-system and/or database backup.	2. System validates authorization and request payload. 2. System creates backup record with status *queued*. 4. System enqueues backup job. 6. Backup archive is streamed to storage provider. 7. System logs backup steps and metadata. 8.System marks backup as completed. 9. System returns backup details.
Alternative flow	**Actor**	**System**
		A1 – File-System Only Backup A1.1 System executes file backup only A1.1 System executes file backup only.

	A3 – Download Backup A3.1 User requests backup download via API.	**A2 – Database Only Backup** A2.1 System executes database dump only. A3.2 System streams archive from provider.
Exceptional Flow	**E1 – Backup Execution Failure** E1.1 System detects failure during execution. E1.2 System updates backup status to **failed**. E1.3 System logs failure and emits failure event.	

Business Rules	1. Backup execution must be asynchronous. 2. Backup archives must be streamed without large temporary files. 3. Users may only access authorized backups.
Related Functional Requirements	FR-BACKUP-07 – FR-BACKUP-19, FR-BACKUP-30 – FR-BACKUP-32
Notes	This use case represents the core backup lifecycle and is fully API-driven.

Table 4.3.3.10.: Fully Dressed Use Case – UC-BACKUP-03

Field	**Description**	
Use Case ID	UC-BACKUP-03	
Use Case Name	Backup Restore Management	
Primary Actor	Administrator	
Secondary Actors	Restore Queue, Storage Provider, Database Engine	
System Boundary	Backup Restore API	
Trigger	Administrator submits a restore request via API	
Preconditions	1. Administrator is authenticated. 2. Administrator has restore permissions. 3. Target backup exists and is accessible.	
Postconditions (Success)	1. Restore job is queued for asynchronous execution. 2. Backup archive is extracted. 3. File-system and/or database state is restored. 4. Restore execution logs are persisted.	
Main Success Flow	**Actor**	**System**
	1.Administrator submits restore request via API.	2.System validates administrator authorization. 3. System enqueues restore job.

		4. Worker downloads backup archive. 5. System extracts archive contents. 6. System restores files and/or database. 7. System logs restore steps. 8.System emits restore completion event.
Alternative flow	**Actor**	**System**
	A1 – Partial Restore A1.1 Administrator selects file-only or database-only restore.	

Exceptional Flow	**E1 – Restore Failure** E1.1 System detects restore error. E1.2 System logs failure. E1.3 System emits restore failure event.	
Business Rules	1. Restore operations are restricted to administrators. 2. Restore operations must be asynchronous. 3. Restore actions must be auditable.	
Related Functional Requirements	FR-BACKUP-20 – FR-BACKUP-25, FR-BACKUP-30 – FR-BACKUP-32	
Notes	Restore execution is isolated from active system operations.	

Table 4.3.3.11.: Fully Dressed Use Case – UC-BACKUP-04

Field	**Description**
Use Case ID	UC-BACKUP-04
Use Case Name	Backup Retention & Lifecycle Management
Primary Actor	System (Automated Process)
Secondary Actors	Storage Providers
System Boundary	Backup Retention API & Worker
Trigger	Scheduled retention job execution
Preconditions	1.Retention policies are configured. 2. Backup records exist.
Postconditions (Success)	1.Expired backups are identified. 2.Backup archives are deleted from storage providers.

	3. Backup metadata is updated accordingly.	
Main Success Flow	**Actor**	**System**
	1. Retention job is triggered.	2.System scans backup records. 3.System identifies expired backups. 4.System deletes backup archives. 5.System updates backup records. 6. System logs retention actions.
Alternative flow	**Actor**	**System**
		A1 – No Expired Backups A1.1 System completes retention scan with no deletions.

Exceptional Flow	**E1 – Deletion Failure** E1.1 System logs deletion failure. E1.2 System retries or flags backup for manual review.	
Business Rules	1.Retention policies must be enforced automatically. 1.Retention must not affect active backups. 3. Deletion operations must be auditable.	
Related Functional Requirements	FR-BACKUP-26 – FR-BACKUP-29	
Notes	Retention management is fully automated and API-independent.	

Table 4.3.3.12.: Fully Dressed Use Case – UC-AUTO-01

Field	**Description**
Use Case ID	UC-AUTO-01
Use Case Name	Manage Automation Profiles
Primary Actor	Client Application (Admin Panel / API Consumer)
Secondary Actors	None
System Boundary	Automation Management API
Trigger	Client submits request to create, update, retrieve, list, or delete an automation profile via API
Preconditions	1.Actor is authenticated. 2.Actor has Automation.Read or Automation.Write permission.
Postconditions	1. Automation profile is created, updated, deleted, or

(Success)	retrieved. 2. Profile configuration is persisted in storage.	
Main Success Flow	**Actor**	**System**
	1. Client submits profile management request via API.	2. System validates request payload and permissions. 3. System persists profile configuration. 4. System returns profile data in API response.
Alternative flow	**Actor**	**System**
		A1 – Invalid Profile Data A1.1 System detects invalid input. A1.2 System rejects request and returns validation error.

Exceptional Flow	E1 – Profile Not Found E1.1 System cannot locate requested profile. E1.2 System returns not found response.	
Business Rules	Profile identifiers must be unique. 2. Cron expressions must be valid if provided.	
Related Functional Requirements	FR-AUTO-01	
Notes	Profile management is API-driven and independent of execution.	

Table 4.3.3.13.: Fully Dressed Use Case – UC-AUTO-02

Field	**Description**	
Use Case ID	UC-AUTO-02	
Use Case Name	Enable / Disable Automation Profile	
Primary Actor	Client Application (Admin Panel / API Consumer)	
Secondary Actors	Scheduler Service	
System Boundary	Automation Scheduling API	
Trigger	Client submits enable or disable request for an automation profile	
Preconditions	1. Automation profile exists. 2. Actor has Automation.Write permission.	
Postconditions (Success)	Profile enabled/disabled state is updated. 2. Scheduler is registered or unregistered accordingly.	
Main Success Flow	**Actor**	**System**
	1.Client sends	

	enable/disable request via API.	2.System validates profile existence and permissions. 3.System updates profile state. 4.System registers or unregisters scheduler job. 5.System returns success response.
Alternative flow	**Actor**	**System**
	1 – No State Change Required A1.1 Profile already in requested state.	A1.2 System returns success response without modification.
Exceptional Flow	E1 – Scheduler Failure E1.1 System fails to update scheduler.E1.2 System logs error and returns failure response.	
Business Rules	1. Disabled profiles must not execute. 2. Enabled profiles must be schedulable.	
Related Functional	FR-AUTO-03	

Requirements	

Table 4.3.3.14.: Fully Dressed Use Case – UC-AUTO-03

Field	**Description**	
Use Case ID	UC-AUTO-03	
Use Case Name	Manage Automation Tasks	
Primary Actor	Client Application (Admin Panel / API Consumer)	
Secondary Actors	None	
System Boundary	Automation Task Management API	
Trigger	Client submits request to add, update, list, or remove automation tasks	
Preconditions	1.Automation profile exists. 2.Actor has Automation.Write permission.	
Postconditions (Success)	1.Automation tasks are created, updated, or removed. 2. Task execution order is persisted.	
Main Success Flow	**Actor**	**System**
	1.Client submits task management request via API.	2.System validates task payload and profile association. 3.System stores task configuration and order. 4.System returns updated task list.
	Actor	**System**
		A1 – Invalid Task Configuration A1.1 System detects invalid task payload.

		A1.2 System returns validation error.
Exceptional Flow	**E1 – Profile Not Found** E1.1 System cannot locate profile. E1.2 System rejects task operation.	
Business Rules	1. Tasks must belong to a valid profile. 2. Task order must be deterministic.	
Related Functional Requirements	FR-AUTO-04, FR-AUTO-05	

Table 4.3.3.15.: Fully Dressed Use Case – UC-AUTO-04

Field	**Description**	
Use Case ID	UC-AUTO-04	
Use Case Name	Manually Execute Automation	
Primary Actor	Client Application (Admin Panel / API Consumer)	
Secondary Actors	Job Queue, Worker Process	
System Boundary	Automation Execution API	
Trigger	Client submits manual execution request for profile or task	
Preconditions	1. Profile or task exists. 2. Actor has Automation.Execute permission.	
Postconditions (Success)	1. Execution run record is created. 2. Execution job is queued asynchronously.	
Main Success Flow	**Actor**	**System**
	1. Client submits manual run request via API.	2. System validates target and

		permissions. 3. System creates execution run record. 4. System enqueues job for background processing. 5. System returns queued response.
	Actor	**System**
		A1 – Invalid Execution Target A1.1 System rejects execution request. A1.2 System returns error response.
Exceptional Flow	**E1 – Queue Failure** E1.1 System fails to enqueue job. E1.2 System logs error and returns failure response.	
Business Rules	1. Manual execution must be asynchronous. 2. Execution must not block API response.	
Related Functional Requirements	FR-AUTO-06, FR-AUTO-07	
Notes	Worker execution is decoupled from API lifecycle.	

Table 4.3.3.16.: Fully Dressed Use Case – UC-AUTO-05

Field	Description	
Use Case ID	UC-AUTO-05	
Use Case Name	View Automation Execution Status	
Primary Actor	Client Application (Admin Panel / API Consumer)	
Secondary Actors	None	
System Boundary	Automation Monitoring API	
Trigger	Client requests execution status or logs via API	
Preconditions	1. Execution run exists.2. Actor has Automation.Read permission.	
Postconditions (Success)	1. Execution status, timestamps, and logs are returned.	
Main Success Flow	**Actor**	**System**
	1. Client requests execution status via API.	2. System validates access. 3.System retrieves execution records. 4. System returns execution details.
Alternative Flow	**Actor**	**System**
		A1 – No Execution Records A1.1 System returns empty execution result

Exceptional Flow	**E1 – Execution Not Found** E1.1 System returns not found response.	
Business Rules	1.Execution logs must be immutable. 2. Execution records must be auditable.	
Related Functional Requirements	FR-AUTO-10	
Notes	Execution status retrieval is read-only and non-intrusive.	

Table 4.3.3.17.: Fully Dressed Use Case – UC-PLUG-01

Field	**Description**	
Use Case ID	UC-PLUG-01	
Use Case Name	Plugin Installation & Registration	
Primary Actor	User, API Consumer	
Secondary Actors	Filesystem, Plugin Loader	
System Boundary	Plugin Management API	
Trigger	User uploads a plugin package, or system startup triggers plugin discovery.	
Preconditions	1. Plugin archive or folder is accessible. 2. Plugin Management API is available.	
Postconditions (Success)	1. Plugin files are stored in the plugin directory. 2. Plugin manifest is validated. 3. Plugin metadata is registered in the system. 4. Plugin actions are registered and available for execution.	
Main Success Flow	Actor	System
	1. User uploads plugin ZIP/Admin User places	

	plugin in filesystem.	2. System extracts plugin files. 3. System validates plugin manifest schema. 4. System registers plugin metadata. 5. System registers plugin actions. 6. System reloads plugin engine. 7. System returns successful installation response.
Alternative Flow	**Actor**	**System**
		A1 – Invalid Plugin Manifest A1.1 System detects manifest validation failure. A1.2 System rejects plugin installation. A1.3 System returns validation error response.
Exceptional Flow	**E1 – Plugin Load Failure** E1.1 System encounters runtime or filesystem error. E1.2 System aborts installation. E1.3 System logs failure and returns error response.	
Business Requirements	1. Plugin ID must be unique. 2.Manifest schema validation is mandatory.	
Related Functional Requirements	FR-PLUG-01, FR-PLUG-02, FR-PLUG-03, FR-PLUG-04	
Notes	System recognizes admin developer and reseller as User with different roles and permissions	

Table 4.3.3.17.: Fully Dressed Use Case – UC-PLUG-02

Field	**Description**

Use Case ID	UC-PLUG-02	
Use Case Name	Plugin Action Execution	
Primary Actor	API Consumer / Automation Engine	
Secondary Actors	Plugin Runtime (VM2 / WASM)	
System Boundary	Plugin Execution API	
Trigger	API request to execute plugin action	
Preconditions	1. Plugin is installed 2. Action is registered	
Postconditions (Success)	1. Plugin action executed successfully 2. Result returned	
Main Success Flow	**Actor Actions**	**System Response**
		1. System receives execute request 2. System resolves plugin action 3. System executes action in sandbox 4. System returns execution result
Alternative Flow	**Actor**	**System**
		A1 – WASM execution path
Exceptional Flow	E1 – Runtime execution error	
Business Requirements	Execution must be isolated and secure	
Related Functional Requirements	FR-PLUG-05, FR-PLUG-1	
Notes	Supports JS (VM2) and WASM	

Table 4.3.3.18.: Fully Dressed Use Case – UC-PLUG-03

Field	**Description**
Use Case ID	UC-PLUG-03

Use Case Name	Plugin Installation & Configuration	
Primary Actor	Administrator / Marketplace	
Secondary Actors	Filesystem, Database	
System Boundary	Plugin Management API	
Trigger	Plugin upload or install API call	
Preconditions	1. Valid plugin archive 2. Admin privileges	
Postconditions (Success)	1. Plugin installed 2. Plugin settings stored	
Main Success Flow	**Actor Actions**	**System Response** 1. System receives plugin archive. 2. System extracts plugin 3. System validates manifest 4. System reloads plugin engine 5. Plugin settings stored in DB
Alternative Flow	**Actor**	**System**
		A1 – Folder-based installation
Exceptional Flow	E1 – Extraction failure	
Business Requirements	Installation must be atomic	
Related Functional Requirements	FR-PLUG-07, FR-PLUG-08, FR-PLUG-09	
Notes	API-first installation	

Table 4.3.3.19.: Fully Dressed Use Case – UC-PLUG-04

Field	**Description**
Use Case ID	UC-PLUG-04
Use Case Name	Plugin Execution via Automation

Primary Actor	Automation Engine	
Secondary Actors	Plugin Runtime	
System Boundary	Automation + Plugin Integration API	
Trigger	Automation workflow execution	
Preconditions	1. Plugin installed 2. Action exposed	
Postconditions (Success)	Plugin action executed as automation task	
Main Success Flow	**Actor Actions**	**System Response**
		1. Automation engine resolves plugin action 2. System executes plugin 3. Result passed to workflow
Alternative Flow	**Actor**	**System**
		A1 – Plugin disabled
Exceptional Flow	E1 – Action execution failure	
Business Requirements	Plugins must behave as first-class automation actions	
Related Functional Requirements	FR-PLUG-10	
Notes	Tight integration, no UI dependency	

Table 4.3.3.20: Fully Dressed Use Case – UC-AUTO-01

Field	**Description**
Use Case ID	UC-AUTO-01
Use Case Name	Use Plugin as Automation Action
Primary Actor	Automation Engine
Secondary	Plugin Runtime

Actors		
System Boundary	Automation Execution API	
Trigger	Automation workflow execution	
Preconditions	Plugin action registered	
Postconditions (Success)	Plugin executed as automation step	
Main Success Flow	**Actor Actions**	**System Response**
		1.Workflow reaches plugin step 2. Plugin executor invoked 3. Result returned
Alternative Flow	**Actor**	**System**
		A1 – Plugin disabled
Exceptional Flow	E1 – Plugin runtime error	
Business Requirements	Automation must be extensible	
Related Functional Requirements	FR-ATPL-01, FR-ATPL-02	
Notes	Zero coupling with UI	

Table 4.3.3.21: Fully Dressed Use Case – UC-AUTO-02

Field	**Description**
Use Case ID	UC-AUTO-02
Use Case Name	Chained Plugin Automation
Primary Actor	Automation Engine
Secondary Actors	Multiple Plugins
System Boundary	Automation Orchestration API

Trigger	Automation workflow execution	
Preconditions	Multiple plugin actions configured	
Postconditions (Success)	All plugin actions executed sequentially	
Main Success Flow	**Actor Actions**	**System Response**
		1. Execute first plugin. 2. Pass output to next plugin 3. Continue until completion
Alternative Flow	**Actor**	**System**
		A1 – Conditional skip
Exceptional Flow	E1 – One step fails	
Business Requirements	Workflow reliability	
Related Functional Requirements	FR=APTL-03	
Notes	Supports complex automation	

Table 4.3.3.22: Fully Dressed Use Case – UC-AUTO-03

Field	**Description**	
Use Case ID	UC-AUTO-03	
Use Case Name	Plugin UI Embedded in Automation Builder	
Primary Actor	Administrator	
Secondary Actors	Plugin UI Sandbox	
System Boundary	Automation Builder UI + API	
Trigger	Automation configuration request	
Preconditions	Plugin exposes UI	
Postconditions (Success)	Plugin UI rendered in sandbox	
Main Success Flow	**Actor Actions**	**System Response**

		1. System loads plugin UI iframe 2. UI embedded securely
Alternative Flow	**Actor**	**System**
		A1 –UI not available
Exceptional Flow	A1 – UI not available	
Business Requirements	E~1 – UI load failure	
Related Functional Requirements	FR-ATPL-04	
Notes	CSP & iframe sandbox enforced, System is headless, Clear Segregation Between frontend and backend.	

4.3.4 System Sequence Diagrams

UC-AUTH-01 — User Registration & Email Verification

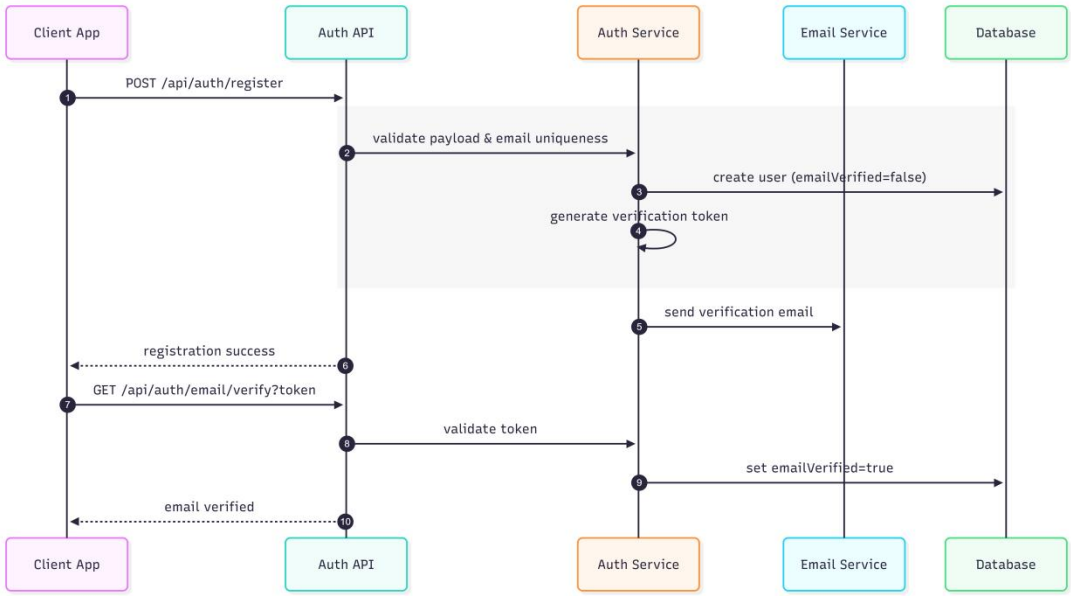


Fig 4.5: System Sequence Diagram

UC-AUTH-02 — User Login & Session Management

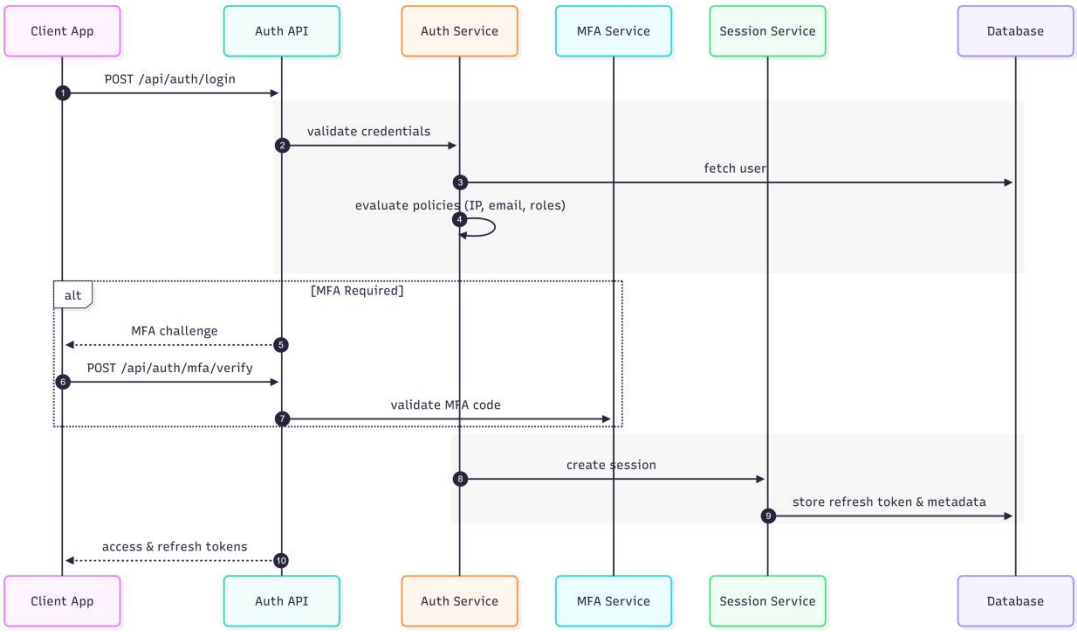


Fig 4.6: System Sequence Diagram

UC-PASS-01 — Password Reset

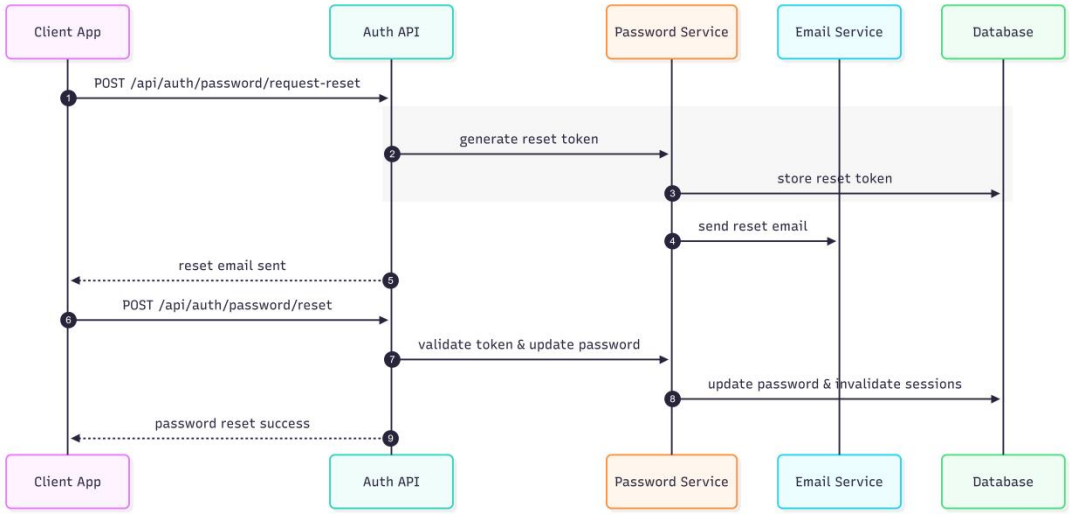


Fig 4.7: System Sequence Diagram

UC-ADMIN-01 — Admin & User Management

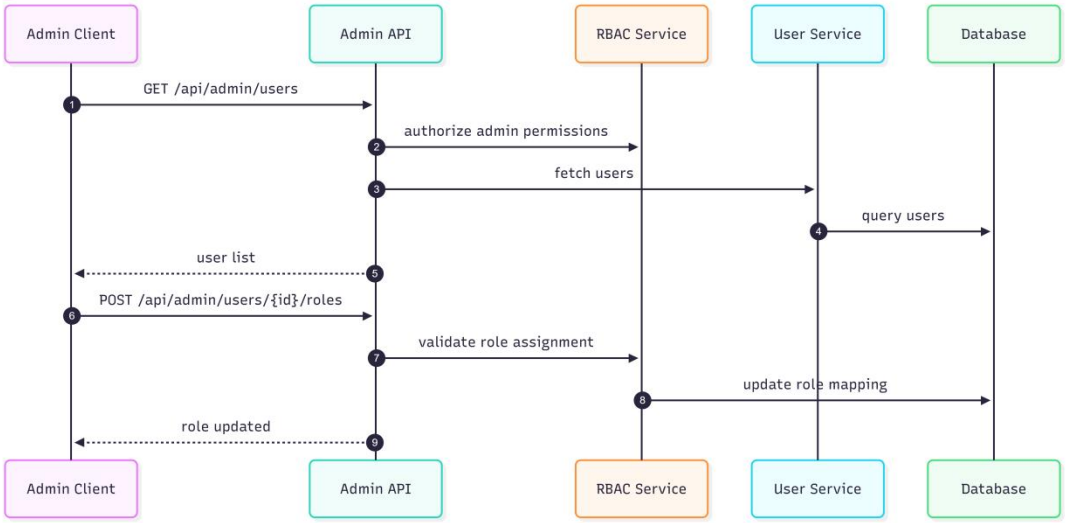


Fig 4.8: System Sequence Diagram

UC-IMP-01 — User Impersonation

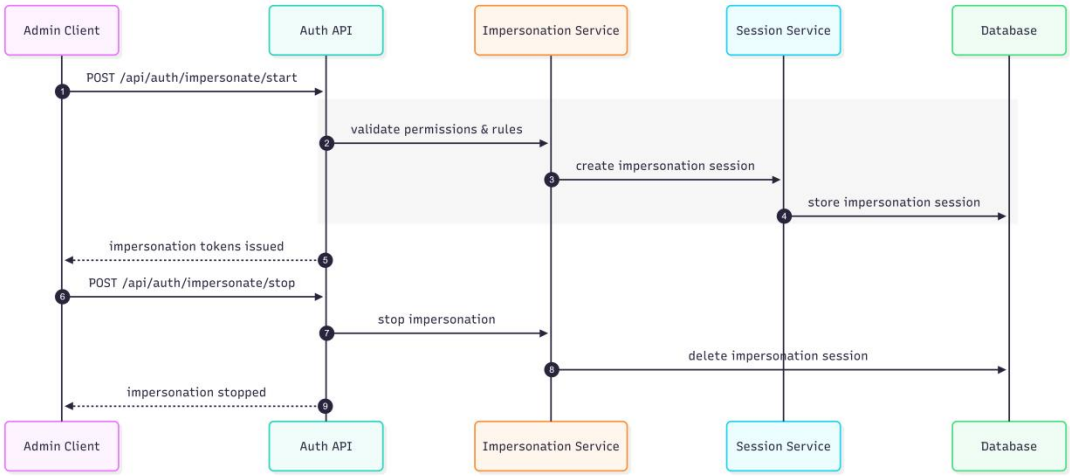


Fig 4.9: System Sequence Diagram

UC-MFA-01 — Multi-Factor Authentication Management

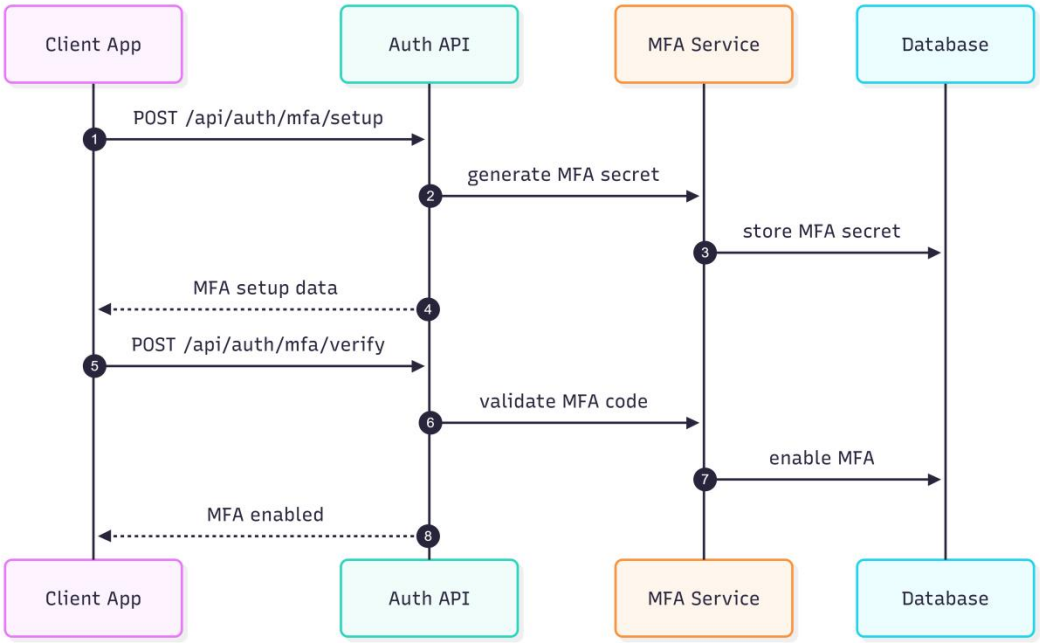


Fig 4.10 System Sequence Diagram

UC-API-01 — API Key Management

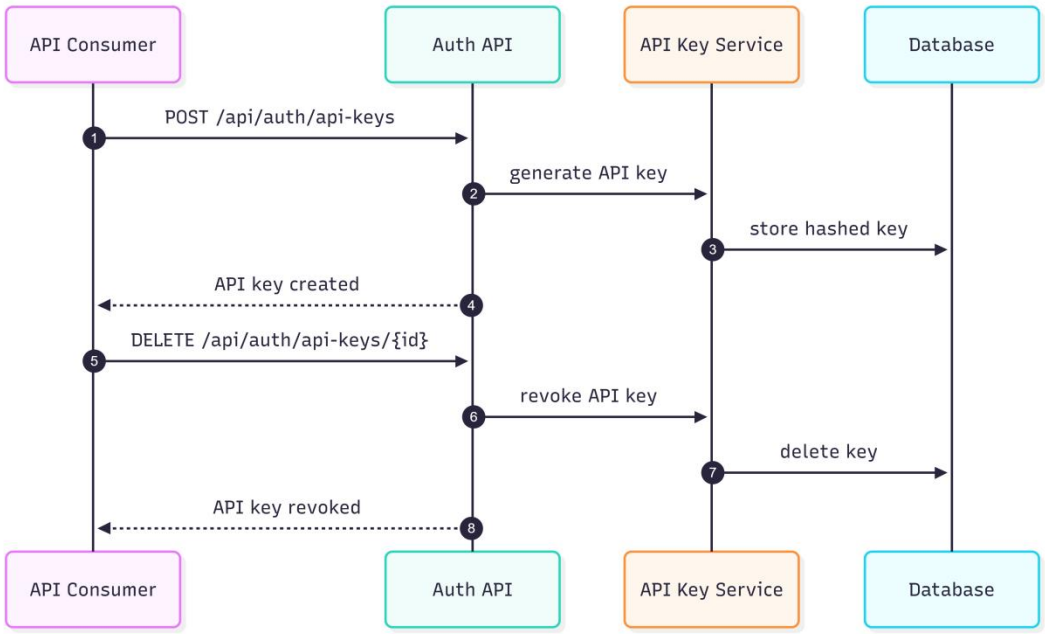


Fig 4.11 System Sequence Diagram

AUTOMATION

UC#1

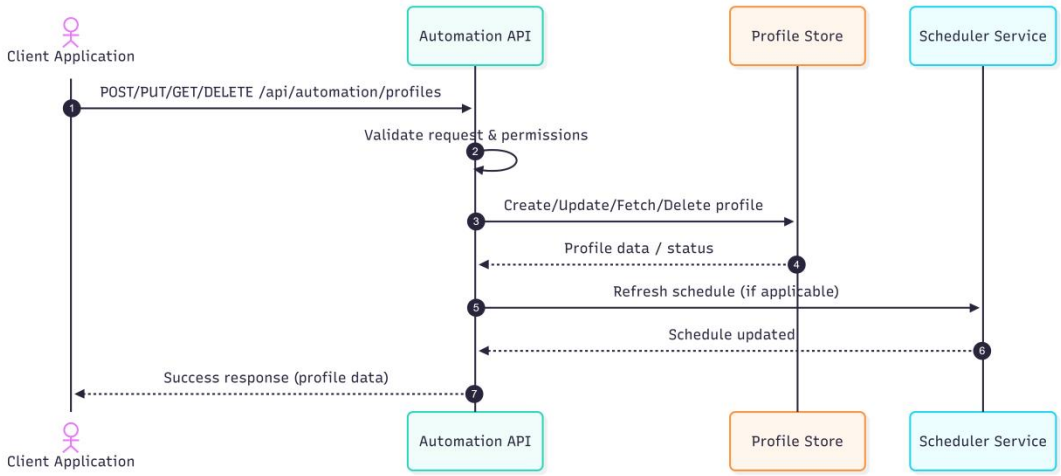


Fig 4.12 System Sequence Diagram

UC#2

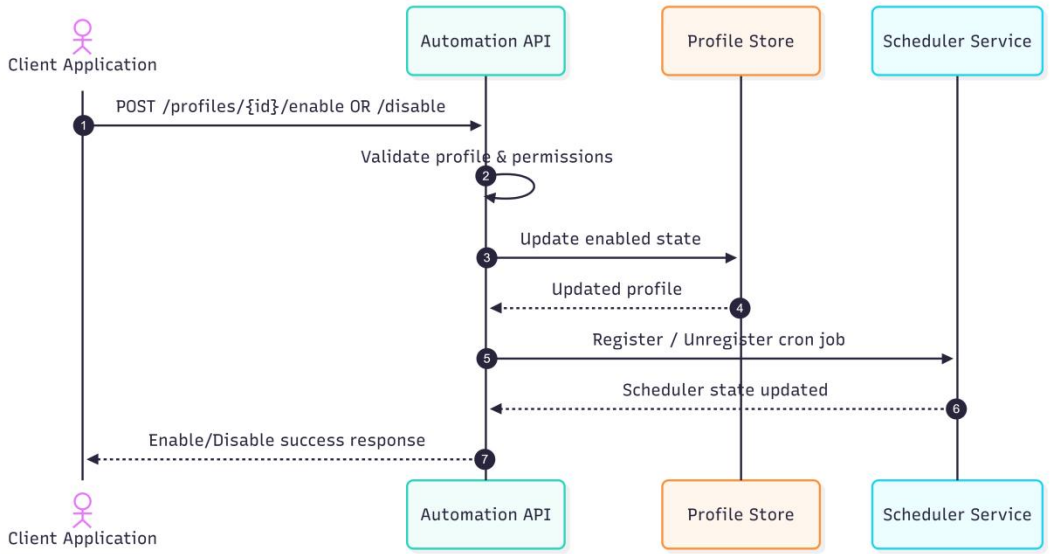


Fig 4.13 System Sequence Diagram

UC#3

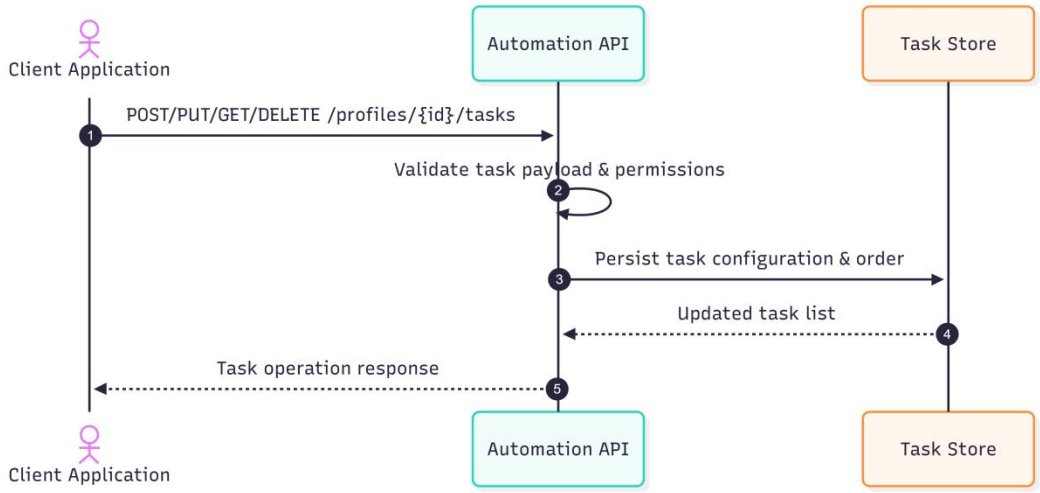


Fig 4.14 System Sequence Diagram

UC#4

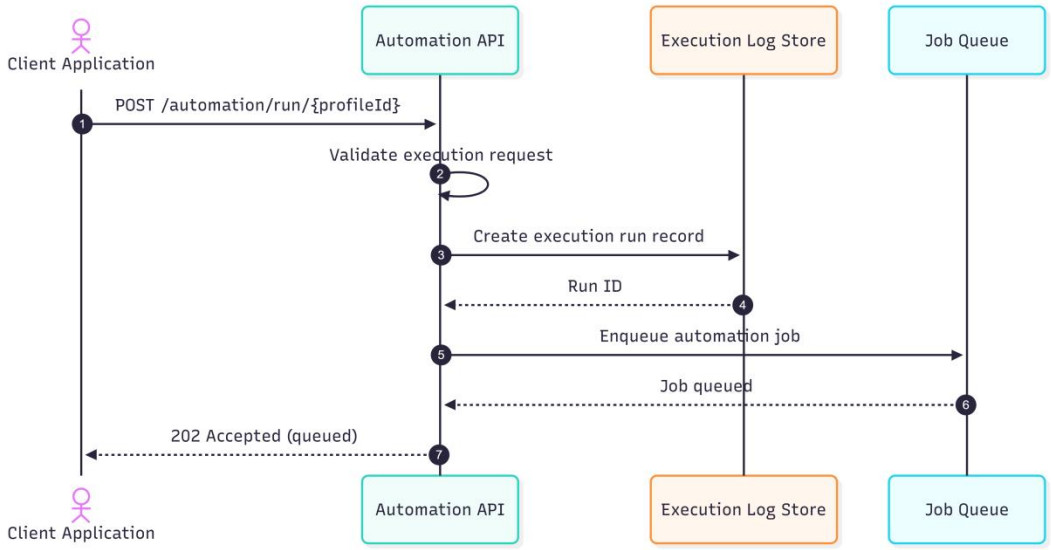


Fig 4.15 System Sequence Diagram

UC#5

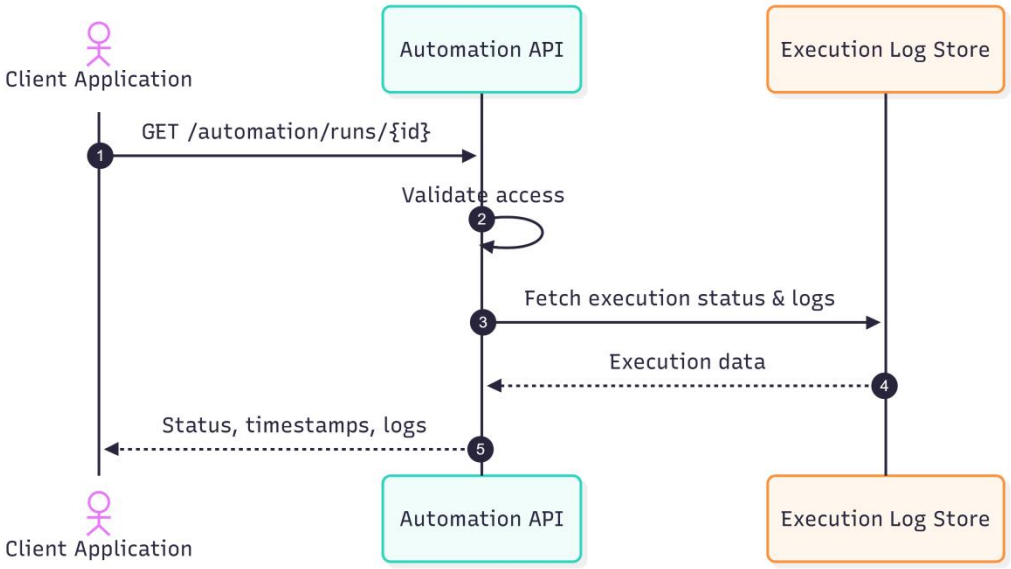


Fig 4.16 System Sequence Diagram

Plugin Module

UC-PLUG-01

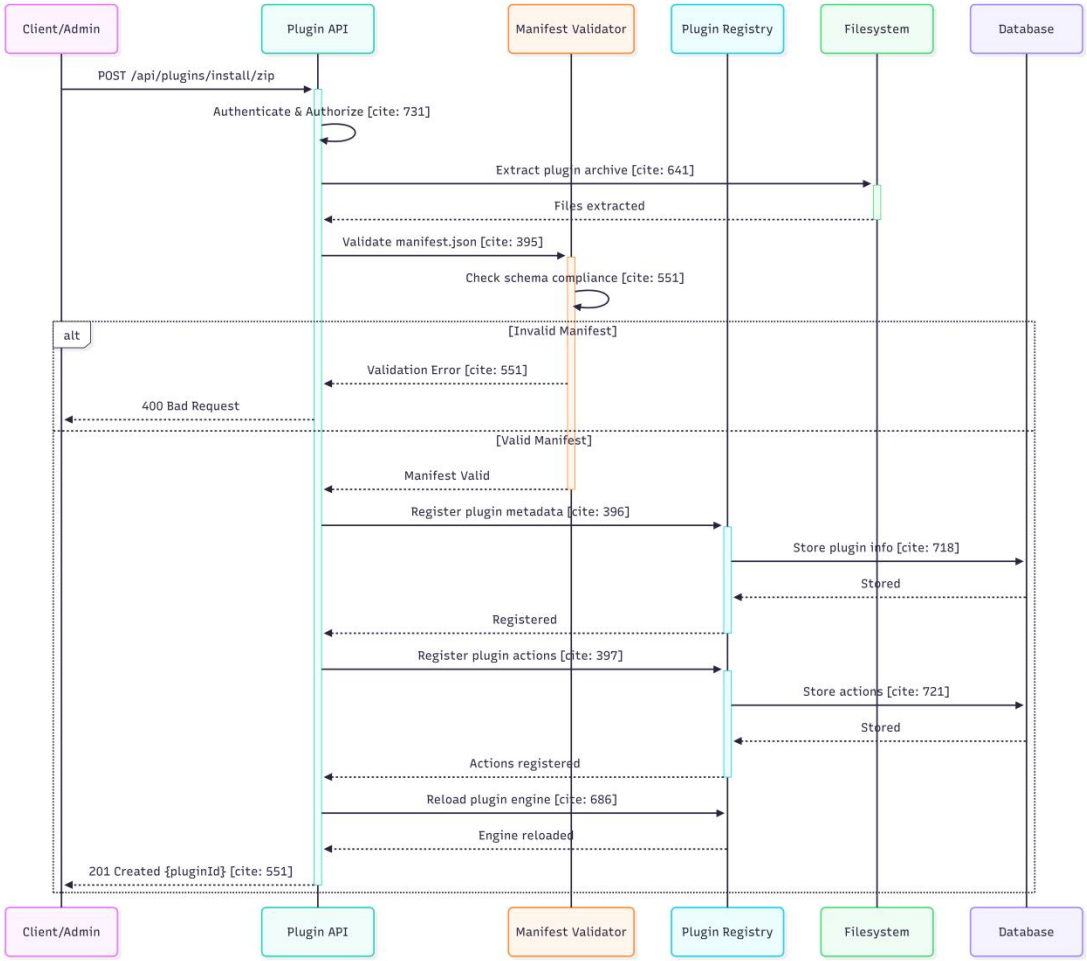


Fig 4.17 System Sequence Diagram

Secure Plugin Execution (UC-PLUG-02)

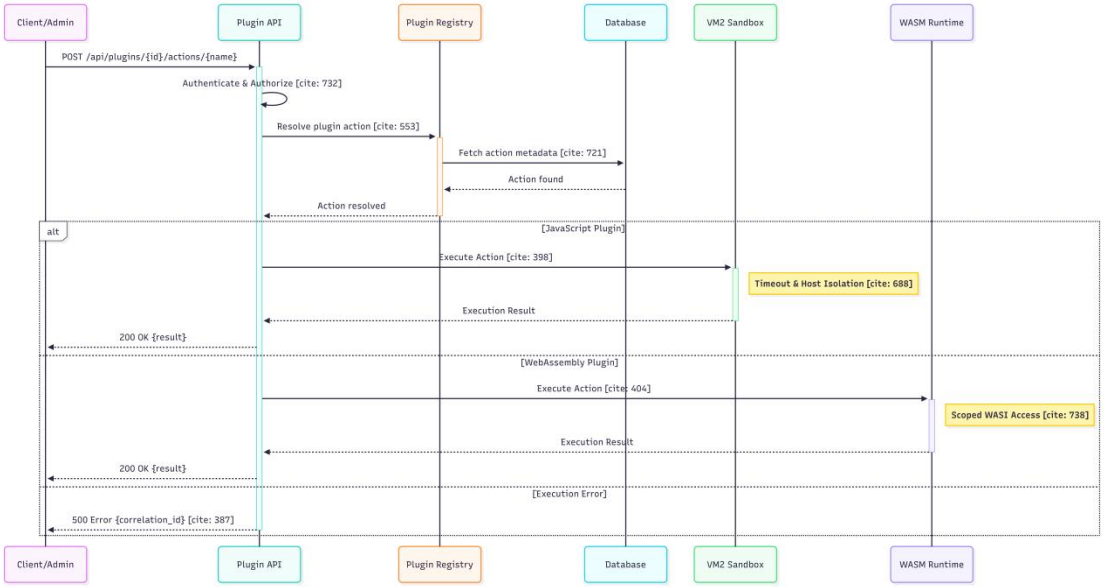


Fig 4.18 System Sequence Diagram

Management: Configuration & Discovery (UC-PLUG-03)

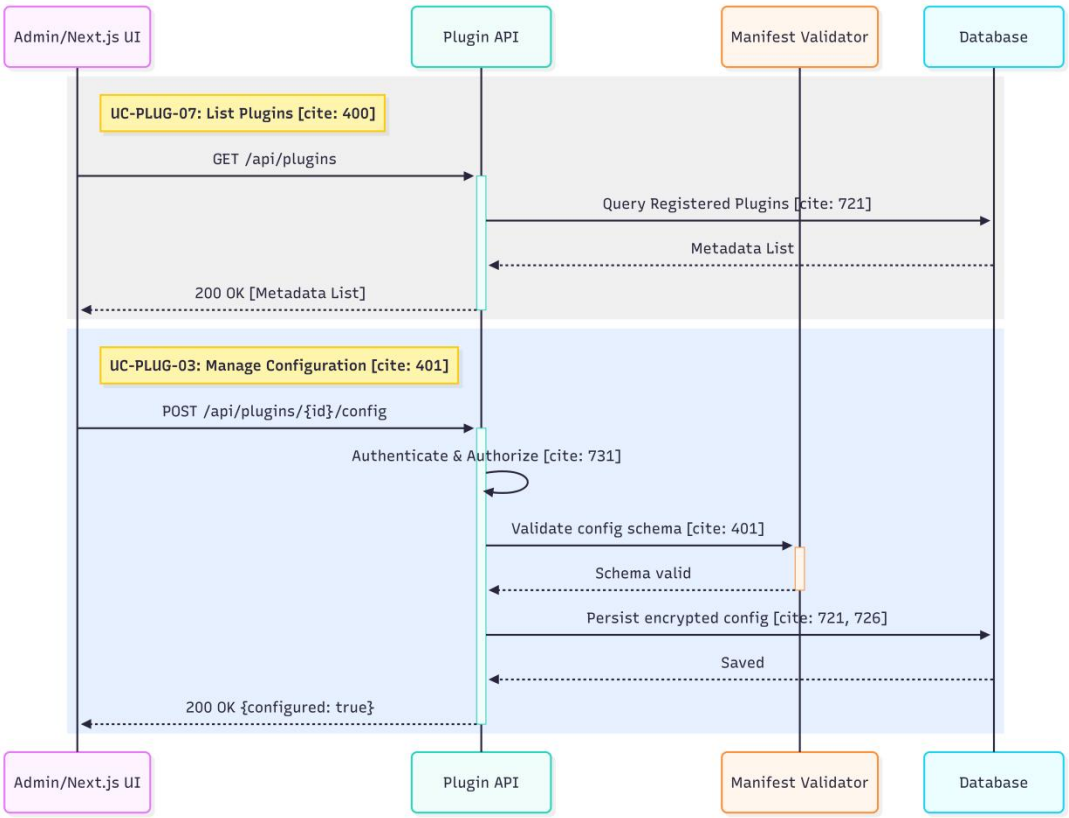
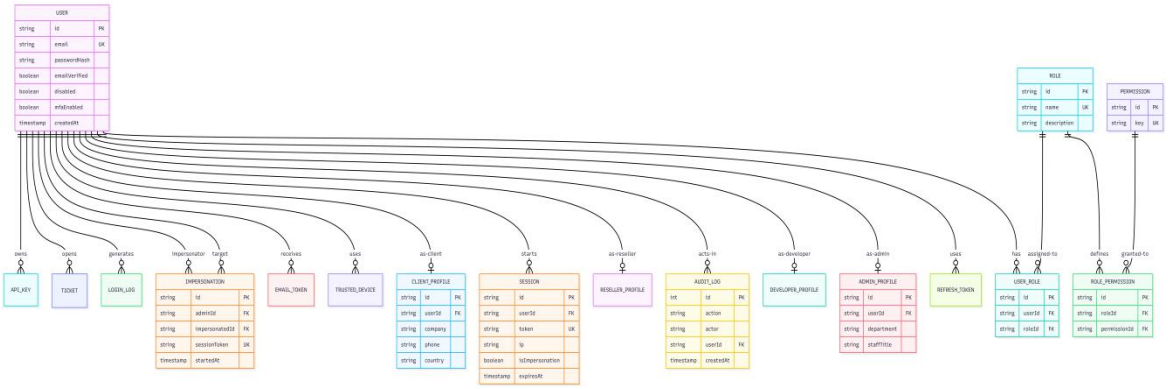


Fig 4.19 System Sequence Diagram

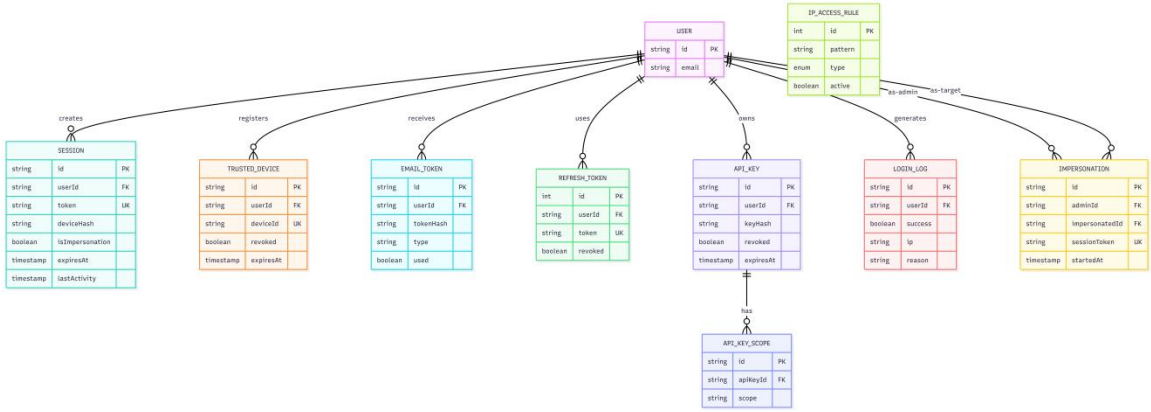
4.4 DATABASE DESIGN

The complete ERD of the system has been intentionally abstracted in this phase to maintain clarity and focus on core architectural concepts. Due to the large scale and modular nature of the system, a full ERD would introduce unnecessary complexity, reduce readability, and distract from the primary design objectives being evaluated. Therefore, only the most critical entities and relationships relevant to authentication, authorization, and user management are presented, while the remaining modules follow the same standardized design principles and can be expanded independently as the system evolves.

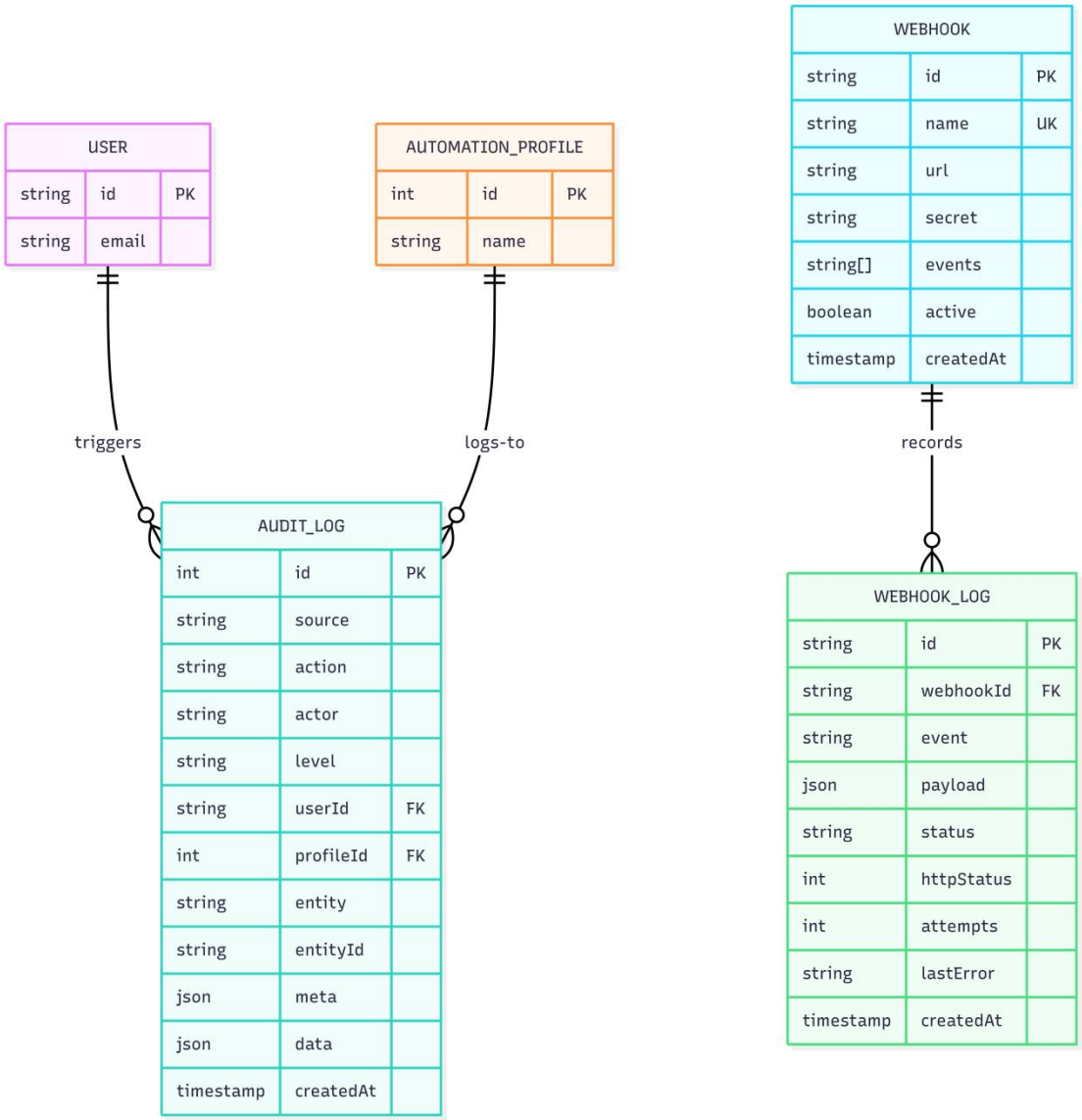
4.4.1 Core Identity & Access Control



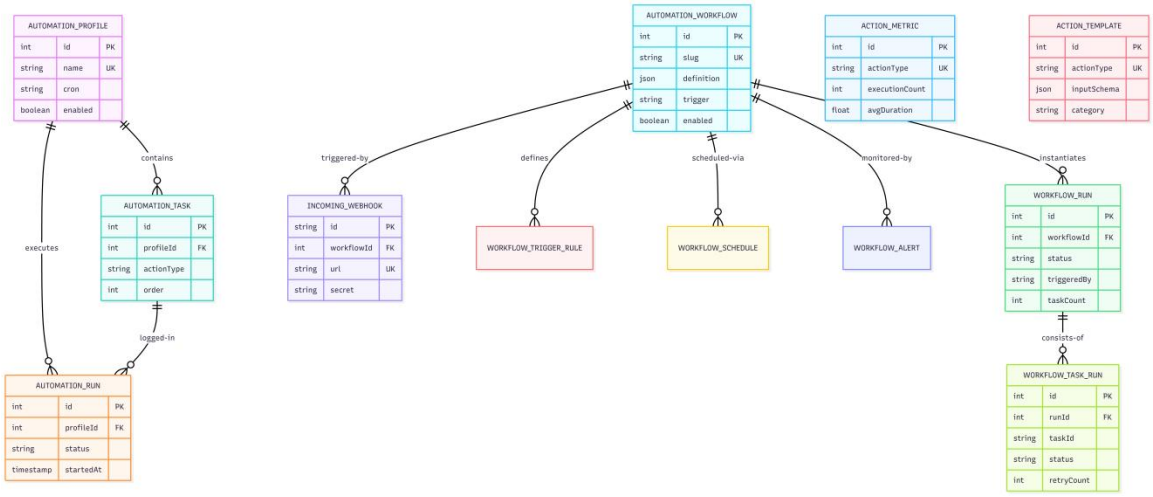
4.4.2 Authentication & Session Management



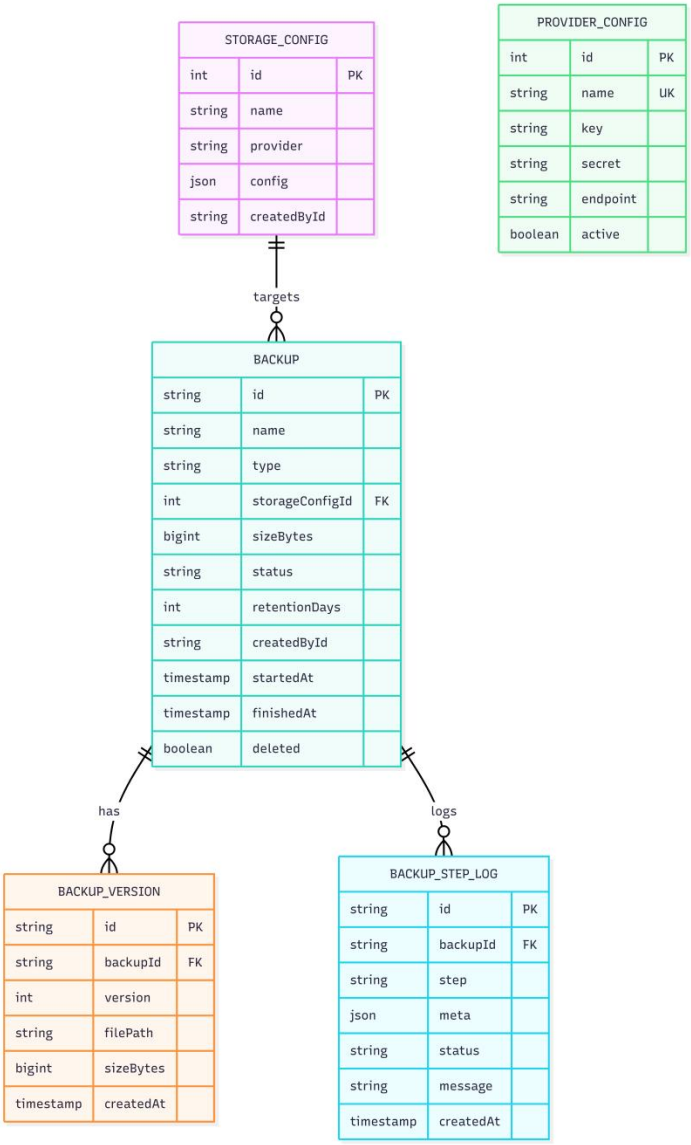
4.4.3 Audit Logs



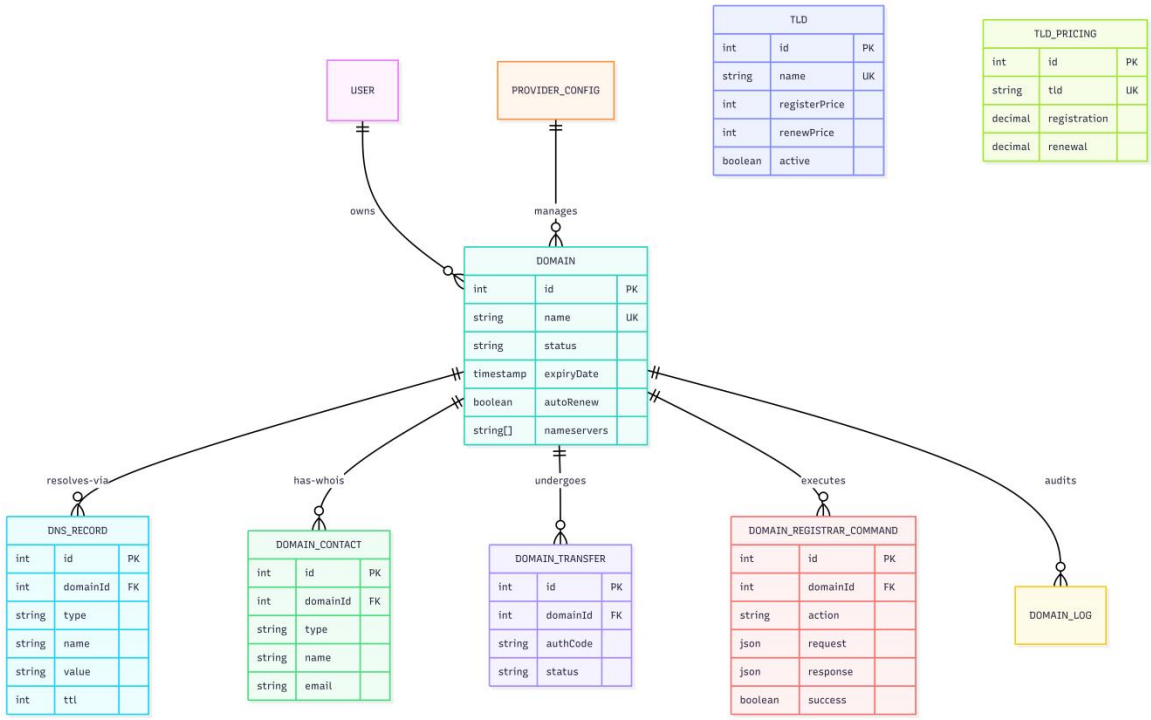
4.4.4 Automation and Workflows



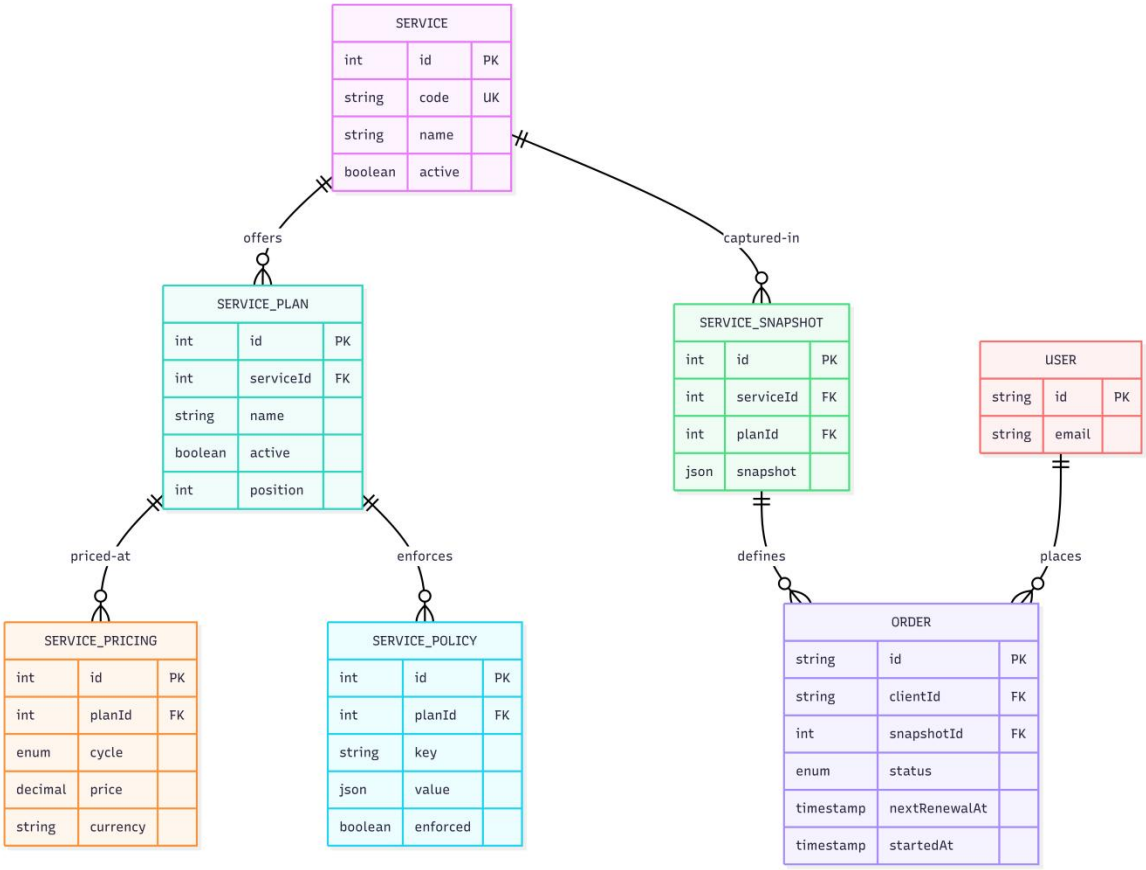
4.4.5 Backup Module



4.4.6 Domain Management



4.5.7 Services and Order Module



4.5 SOFTWARE QUALITY ASSURANCE (SQA) – DESIGN LEVEL DEFECT DETECTION

Software Quality Assurance (SQA) activities at the design level focus on validating that the proposed system architecture and design artifacts correctly satisfy the functional requirements defined in Chapter 3. The objective of this phase is to detect design-level defects such as architectural inconsistencies, missing interactions, incorrect responsibility allocation, and security enforcement gaps before implementation begins.

As the Web Hosting Management System (WHMS) follows an API-first and headless architecture, design-level SQA emphasizes verification of module boundaries, interaction flows, data organization, and security placement, rather than user interface behavior.

4.5.1 Design Validation Approach

Design validation was performed by systematically reviewing architectural diagrams, use case models, sequence diagrams, and domain-specific ERDs against the documented requirements. Each design artifact was examined to ensure that:

Every functional requirement is supported by at least one architectural or design element

No design component exists without a corresponding requirement

Interactions between modules are explicitly defined through APIs

Security-critical operations are enforced at architectural boundaries

To support defect detection at this stage, design-level test cases were defined. These test cases do not validate code execution but instead verify that the design supports correct behavior under expected scenarios.

4.5.2 Design-Level Test Cases (Architecture & Security)

Test Case ID	**Design Scenario**	**Expected Design Outcome**	**Related FRs**
DTC-01	User requests access to protected API	Architecture enforces authentication and RBAC at API boundary	FR-AUTH-10, FR-ADMIN-03
DTC-02	Administrator initiates impersonation	Design provides isolated impersonation session with audit logging	FR-IMP-01
DTC-03	Plugin executes custom action	Execution delegated to sandboxed runtime, isolated from core services	FR-PLUG-05
DTC-04	Plugin UI request	UI rendered via isolated iframe, no	FR-PLUG-06

		direct DOM access	

4.5.3 Design-Level Test Cases (Workflow & Interaction)

Design-level workflows were validated using sequence and activity diagrams for architecturally significant processes.

4.5.3 Design-Level Test Cases (Workflow & interaction)

Test Case ID	**Design Scenario**	**Expected Design Outcome**	**Related FRs**
DTC-05	System startup	Plugin loader scans filesystem and registers plugins before API availability	FR-PLUG-01
DTC-06	Automation executes plugin action	Automation engine invokes plugin executor through internal dispatcher	FR-ATPL-01
DTC-07	Automation task failure	Failure isolated and reported without affecting other services	FR-AUTO-11

These test cases ensure that responsibility allocation and interaction ordering are clearly defined at the design level.

4.5.4 Design-Level Data Verification Test Cases

The database design was reviewed to ensure that data structures support system behavior while maintaining modularity and integrity.

4.5.4 Design-Level Test Cases (Verification)

Test Case ID	**Design Scenario**	**Expected Design**	**Related FRs**

		Outcome	
DTC-08	User assigned multiple roles	RBAC schema supports multiple role assignments without duplication	FR-AUTH-03
DTC-09	Plugin configuration update	Plugin settings stored independently of plugin runtime metadata	FR-PLUG-08
DTC-10	Automation execution logging	Execution results persist without modifying workflow definition	FR-AUTO-05

The database design was reviewed to ensure that data structures support system behavior while maintaining modularity and integrity.

4.5.5 Summary

Design-level SQA activities ensure that the system architecture, security model, data design, and interaction flows collectively provide a correct and robust foundation for implementation. By defining focused design-level test cases, architectural defects are detected early without relying on implementation details. This approach aligns with the software development lifecycle and supports a smooth transition into the implementation phase.

Chapter 5:

Implementation

Chapter 5

Implementation

5.1 INTRODUCTION

This chapter presents the technical implementation of the Web Hosting Management System (WHMS). It focuses on how the system architecture, modules, and APIs were realized in practice. Rather than detailing low-level source code, this chapter emphasizes architectural decisions, module interactions, execution workflows, and design trade-offs adopted during development. The implementation follows an API-first, modular architecture, enabling extensibility, automation, and secure runtime execution.

5.2 DEVELOPMENT PROCESS AND TEAM COLLABORATION

The development of the Web Hosting Management System (WHMS) followed an Increament and software development approach, adapted to the scope and constraints of an academic Final Year Project. The project was executed in short, incremental iterations, where requirements, design, and implementation activities progressed in parallel rather than as strictly sequential phases.

During Part-I, the focus was on requirement analysis, architectural decisions, and development of core system foundations. In Part-II, the emphasis shifted toward complete module implementation, integration, security enforcement, testing, and refinement based on continuous evaluation. This phased focus ensured steady progress while maintaining alignment with academic milestones and evaluation criteria.

5.1.1 Team

The project was developed by a two-member team with clearly defined responsibilities, while maintaining collaborative decision-making for critical technical aspects.

1.Hammad bin Iftikhar (45388)

Responsible for requirements analysis, backend architecture design, API-first system implementation, core service development, security mechanisms, and overall system integration.

2.Muhammad Khalis (46263)

Responsible for frontend development, documentation, testing support, and assistance in validation and integration activities.

Both team members jointly participated in sprint-level reviews, requirement validation, and architectural discussions to ensure consistency and shared understanding of system goals.

5.2.2 Work Breakdown Structure (WBS)

Project work was decomposed into deliverables using a Work Breakdown Structure aligned with Incremental Approach, enabling incremental development and continuous progress tracking. Major work items included analysis, design, core module implementation, testing, and documentation; each revisited and refined across multiple iterations.

The detailed Work Breakdown Structure is provided in the Appendix.

5.2.3 Roles and Responsibility Allocation

Responsibilities were assigned based on technical expertise and module ownership to ensure accountability while avoiding development silos. Backend services, architectural components, and security-critical modules were primarily handled by Hammad bin Iftikhar, whereas frontend development, documentation, and testing coordination were primarily handled by Muhammad Khalis.

Integration testing, requirement verification, and iteration reviews were performed collaboratively. The detailed Roles and Responsibility Matrix is provided in the Appendix.

5.3 TECHNOLOGY STACK AND TOOLS

5.3.1 Backend Technologies

The backend of the system is implemented using Node.js with Express.js as the primary web framework. All system functionality is exposed through RESTful APIs, ensuring a clear separation between client applications and backend services. Prisma ORM is used for database access and schema management, providing a structured, maintainable, and type-safe data access layer.

Additional backend technologies utilized in key system components include AJV for JSON Schema validation, vm2 for sandboxed execution of JavaScript-based plugins, WebAssembly (WASM) with WASI for secure and language-agnostic plugin execution, and Multer along with AdmZip for handling plugin archive uploads and extraction.

Other supporting libraries and tools are used across different modules as required; however, they are not discussed in detail as they do not play a significant role in the core system architecture.

5.2.2 Frontend Technologies

The administrative interface is implemented using Next.js as a client application that consumes backend services exclusively through RESTful APIs, in accordance with the API-first architecture of the system. The frontend does not contain business logic and acts solely as a presentation and interaction layer.

ShadCN UI components are used to ensure a consistent, modular, and reusable user interface design across administrative views. Plugin-provided user interfaces are rendered within isolated iframe containers and accessed through dedicated backend endpoints, preventing direct DOM integration with the host application and enforcing strict separation between core system UI and extensible plugin interfaces.

5.2.3 Development and Supporting Tools

Following tools are being used primarily for development of WHMS

Git for version control

Postman for API testing and validation

Linux-based environment for development and deployment

Vs Code as Primary IDE

5.4 OVERALL SYSTEM IMPLEMENTATION ARCHITECTURE

The system follows a service-oriented, API-first architecture. All system functionality is exposed through RESTful APIs, allowing multiple clients such as web dashboards, automation engines, and external services to interact with the backend consistently.

The backend is divided into independent modules, such as:

Authentication and Identity Management

Plugin Management and Execution

Plugin Marketplace

Automation Engine

Backup Management

Client Management

Provisioning

Public APIs are used for client-facing operations, while internal APIs and service interfaces are used for secure inter-module communication.

5.5 CORE EXECUTON FLOW

This section describes the execution flow of critical system operations within the Web Hosting Management System (WHMS). Rather than presenting low-level pseudocodes for all modules, only architecturally significant and multi-step workflows are documented. This approach ensures clarity while maintaining focus on core system behavior and architectural correctness.

5.5.1 Authentication and Session Management Flow

The authentication flow begins when a client submits credentials through a public authentication API endpoint. Upon successful validation, the system issues short-lived access tokens and long-lived refresh tokens. Refresh tokens are persisted as session records along with contextual metadata such as IP address and user agent.

Each incoming request to protected APIs is validated through middleware that verifies token integrity, evaluates role-based access control (RBAC) rules, and enforces session validity. Session revocation and logout operations invalidate refresh tokens and immediately terminate associated sessions. This flow ensures stateless request handling while maintaining secure session control.

5.5.2 Automation Execution Flow

Automation workflows are executed through a structured, asynchronous pipeline. Automation profiles define schedules and ordered task sequences, which are interpreted by the automation engine. When a profile is triggered—either manually or via a scheduler—the system enqueues execution jobs into a background job queue.

Worker instances retrieve jobs independently within the execution environment, execute tasks sequentially based on defined order, and record execution state transitions (pending, running, completed, failed). Execution results, logs, and timestamps are persisted for

auditability and troubleshooting. This design ensures non-blocking API behavior and supports horizontal scalability.

5.5.3 Plugin Loading and Execution Flow

Plugins are dynamically discovered and loaded through a controlled backend plugin loader. During installation or reload, each plugin's manifest is validated against a predefined schema before registration. Valid plugins are added to an internal registry along with declared actions and metadata.

When a plugin action is invoked—either directly or as part of an automation workflow—the execution is delegated to a sandboxed runtime environment. This isolation mechanism enforces execution limits, restricts host access, and prevents unauthorized interaction with core system components. Plugin-provided user interfaces are served through sandboxed endpoints with enforced content security policies.

5.5.4 Backup and Restore Flow

Backup operations are initiated through authenticated APIs and executed asynchronously to avoid blocking client requests. Upon request, the system validates storage configuration, streams backup data directly to the selected storage provider, and records execution metadata and logs.

Restore operations follow a similar controlled flow, where authorization checks are enforced, restore jobs are queued for background execution, and execution outcomes are logged for auditing. Lifecycle events generated during backup and restore operations may be forwarded to the automation engine for further processing.

5.5.5 Summary

These core execution flows demonstrate how WHMS coordinates complex operations using stateless APIs, asynchronous processing, and modular execution boundaries. By focusing on critical workflows rather than exhaustive pseudocode, the system maintains architectural clarity while ensuring scalability, security, and extensibility.

5.6 MODULE WISE IMPLEMENTATION DETAILS

5.6.1 Authentication and Identity Management Module

The authentication module handles user registration, login, token management, email verification, and session handling. Token-based authentication is implemented using short-lived access tokens and refresh tokens. Role-based access control (RBAC) ensures

that privileged operations are restricted to authorized users.

Additional security features such as multi-factor authentication (MFA), API keys, impersonation, and audit logging are implemented as independent services integrated with the core authentication flow.

5.6.2 Plugin Module Implementation

5.6.3 Automation Module Implementation

5.6.4 Backup Module Implementation

5.7 API DESIGN AND IMPLEMENTATION

The system is designed following an API-first and headless architecture, where all core functionality is implemented and exposed through RESTful APIs independent of any user interface. Client applications, including the administrative dashboard and automation components, interact with the backend exclusively via these APIs.

The API implementation adheres to established REST principles, including:

Clear separation of HTTP methods (GET, POST, DELETE) based on operation semantics

Stateless request handling to ensure scalability and reliability

Consistent and structured error responses across all endpoints

Token-based authentication and authorization for protected resources

Public APIs are exposed for external clients and administrative interfaces, while internal APIs and service interfaces are used for secure module-to-module communication. This separation ensures loose coupling between system components, improves maintainability, and reinforces security boundaries within the system.

5.8 DATA PERSISTENCE AND DATABASE DESIGN

The system implements data persistence using a relational database accessed through Prisma ORM, which serves as the primary data access layer. Prisma provides a structured schema definition, consistent data modeling, and transactional safety while abstracting low-level database operations from application logic.

The database schema is designed to support the system's modular and extensible architecture. Core entities include users, authentication tokens, sessions, roles, plugin configurations, automation metadata, backup records, and audit logs. Each module persists only the data relevant to its responsibility, ensuring clear separation of concerns at the data layer.

Configuration and operational data, such as plugin settings and automation parameters, are stored in normalized tables to enable dynamic behavior without requiring schema changes for each extension. Where appropriate, flexible data structures are used to accommodate plugin-specific metadata while maintaining overall schema consistency.

The system employs soft-delete strategies for entities where recovery is required, such as plugins and backups, allowing data to be restored when necessary. Transactional operations are used for critical workflows to maintain data integrity during operations such as plugin installation, configuration updates, and backup lifecycle management.

By combining Prisma ORM with a carefully structured relational schema, the system ensures reliable data persistence, maintainability, and consistency while supporting the API-first and headless nature of the overall architecture.

5.9 SECURTY CONSIDERATIONS IN IMPLEMENTATION

Security is addressed as a core architectural concern in the implementation of the system, particularly in response to limitations commonly observed in existing legacy systems that rely on tightly coupled user interfaces, server-side rendering, and unrestricted extension mechanisms. The proposed system adopts an API-first and headless architecture, which inherently reduces the attack surface and mitigates a wide range of common security threats.

Authentication and authorization are enforced at the API layer using token-based mechanisms and role-based access control (RBAC). All protected operations require valid authentication tokens, and privilege-sensitive actions are restricted based on user roles. Session management supports forced logout, impersonation tracking, and session invalidation, ensuring controlled access even in administrative or support scenarios.

Unlike legacy systems where user interfaces and extensions often execute within the same runtime and document context, this system completely separates presentation,

execution, and business logic. The backend never renders HTML content and exposes only structured JSON responses, eliminating traditional server-side injection and template-based attack vectors. Client applications interact exclusively through authenticated APIs, preventing unauthorized logic execution through manipulated user interfaces.

Extensibility is implemented through a controlled plugin architecture. Plugin code is treated as untrusted and executed within sandboxed runtime environments. JavaScript-based plugins are executed using a restricted virtual machine, while WebAssembly-based plugins are executed through a WASI-compatible runtime with scoped access. During installation, plugins may undergo optional signature verification through a validation hook to detect tampering. This approach contrasts with legacy systems where plugins often execute with full access to the application runtime.

Plugin-provided user interfaces are rendered using iframe-based sandboxing with strict Content Security Policy (CSP) enforcement. These interfaces are isolated from the core application and cannot access authentication tokens, modify the host document object model, or invoke privileged operations directly. This design significantly reduces the risk of cross-site scripting (XSS) and client-side injection attacks that are commonly associated with extensible legacy platforms.

All input data is validated at the API boundary using schema validation mechanisms, and database access is abstracted through an ORM layer, reducing the risk of injection attacks. Sensitive operations across authentication, plugin execution, automation, and administrative actions are recorded through audit logging, enabling traceability and post-incident analysis.

By combining architectural isolation, strict API enforcement, controlled extensibility, and comprehensive validation, the system addresses many of the security weaknesses present in existing legacy systems, providing a more robust and defensible security posture without relying on complex or intrusive security mechanisms.

5.10 BEST PRACTICES AND CODING STANDARDS

The implementation of the Web Hosting Management System (WHMS) followed established software engineering best practices and coding standards to ensure

maintainability, security, and long-term extensibility. These practices were applied consistently across all modules to support scalable development and reduce technical debt.

5.10.1 Software Engineering Practices

The system was designed using a modular and layered architecture, where each module encapsulates a distinct business capability such as authentication, automation, plugins, or backups. Clear separation of concerns was maintained between controllers, services, and data access layers to improve testability and simplify future modifications.

An API-first and headless design approach was adopted, ensuring that all system functionality is exposed through well-defined RESTful APIs. This approach enables independent evolution of backend services and frontend client and supports future integration with third-party systems. Role-Based Access Control (RBAC) and centralized authorization checks were enforced uniformly across all protected endpoints.

Architectural decisions emphasized stateless service interactions, asynchronous processing for long-running tasks, and clear interface contracts between modules. These practices collectively improve scalability, fault isolation, and readiness for future migration toward microservices if required.

5.10.2 Development Practices and Standards

Consistent coding standards were followed to enhance code readability and maintainability. Descriptive naming conventions were used for variables, functions, and modules, and code was structured to remain self-explanatory with minimal reliance on inline comments. Error handling was implemented using standardized response formats and centralized exception handling mechanisms to ensure predictable API behavior.

Version control was used throughout the project to manage incremental changes and track development history. Logging and audit mechanisms were integrated to support debugging, monitoring, and security analysis. Sensitive data handling followed best practices, including input validation, schema enforcement, and secure storage of credentials.

These development practices ensured that the WHMS implementation remained robust, secure, and aligned with modern software engineering standards while meeting the academic objectives of the Final Year Project.

5.11 LIMITATIONS AND DESIGN TRADE-OFFS

While the proposed system achieves its functional and architectural objectives, certain limitations and design trade-offs were intentionally adopted to align with the scope and constraints of the Final Year Project. The system is currently deployed as a modular monolithic application, where multiple functional modules operate within a single backend runtime in order to reduce deployment complexity and operational overhead. Despite this deployment choice, the system is designed to be microservices-ready. Each major subsystem, including authentication, plugin management, automation, and backup management, is implemented with clearly defined responsibilities and exposed through well-defined APIs. This API-first and headless architecture ensures that modules remain loosely coupled and can be extracted into independent microservices with minimal refactoring in future iterations.

Horizontal scaling and distributed execution were not implemented in the current version of the system. However, the largely stateless request-handling model and centralized data persistence allow multiple backend instances to be deployed behind a load balancer without changes to the API layer. Runtime responsibilities such as plugin execution and automation workflows are currently executed within the same process but can be delegated to dedicated worker services or execution nodes in a microservices-based deployment, enabling independent scaling of compute-intensive tasks.

Plugin execution is performed using sandboxed runtime environments for both JavaScript and WebAssembly plugins. While this provides a strong level of isolation, it does not offer the same guarantees as container-based or micro-virtualized execution. More advanced isolation mechanisms were considered beyond the scope of this project and are identified as future enhancements that can be integrated without altering the existing plugin execution contracts.

Certain runtime state, such as plugin registry metadata, is maintained in memory and reconstructed during system startup. This approach simplifies implementation, but limits shared state across multiple instances. In a horizontally scaled or microservices-based environment, such state can be externalized to shared data stores or service discovery mechanisms.

Finally, the system was evaluated primarily in a development environment, and comprehensive performance benchmarking, stress testing, and distributed deployment validation were not conducted. These aspects are considered future work. Overall, the identified limitations reflect deliberate design trade-offs that favor architectural clarity, extensibility, and security, while establishing a solid foundation for future evolution into a fully distributed, microservices-based system.

5.12 SOFTWARE QUALITY ASSURANCE (SQA) IMPLEMENTATION LEVEL (WHITE-BOX DEFECT DETECTION)

Software Quality Assurance (SQA) activities at the implementation level focus on white-box defect detection, where internal program logic, execution paths, and error-handling behavior are verified against the validated design and requirements. The objective of this phase is to ensure that the implemented backend logic behaves correctly under normal, boundary, and exceptional conditions.

As the Web Hosting Management System (WHMS) is API-first and headless, implementation-level SQA primarily targets backend controllers, service layers, middleware, and execution engines rather than presentation logic.

5.12.1 White-Box Testing Strategy

White-box testing was applied to security-critical and execution-sensitive components, including:

Authentication and session lifecycle logic

Role-based access control (RBAC) enforcement

Impersonation session handling

Plugin execution via sandboxed runtimes

Automation task execution and failure handling

Testing focused on control flow coverage, conditional branching, and exception handling paths to ensure predictable and secure system behavior.

5.12.2 White-Box Test Cases – Authentication & Security

Test Case ID	**Internal Logic Path**	**Expected Behavior**	**Related FRs**
WB-AUTH-01	Login with valid credentials	Session created, tokens issued	FR-AUTH-10

WB-AUTH-02	Login with invalid password	Authentication rejected, no session created	FR-AUTH-10
WB-AUTH-03	Access protected API without role	Request blocked by RBAC middleware	FR-ADMIN-03
WB-AUTH-04	Logout request	Session invalidated, token revoked	FR-AUTH-14

These test cases verify that authentication and authorization logic is enforced **before controller execution**, preventing unauthorized access paths.

5.12.3 White-Box Test Cases – Impersonation

Test Case ID	**Internal Logic Path**	**Expected Behavior**	**Related FRs**
WB-IMP-01	Start impersonation with permission	Impersonation session created and flagged	FR-IMP-01
WB-IMP-02	Start impersonation without permission	Request rejected before session creation	FR-IMP-02
WB-IMP-03	Stop impersonation	Original administrator session restored	FR-IMP-07
WB-IMP-04	Audit logging during impersonation	All actions logged with impersonator ID	FR-AUDIT-04

5.12.4 White-Box Test Cases – Plugin Execution

Test Case ID	**Internal Logic Path**	**Expected Behavior**	**Related FRs**
WB-PLUG-01	Load plugins at	Valid plugins	FR-PLUG-01

	startup	registered successfully	
WB-PLUG-02	Invalid plugin manifest	Installation aborted, error returned	FR-PLUG-02
WB-PLUG-03	Execute plugin action	Code executed inside sandbox runtime	FR-PLUG-05
WB-PLUG-04	Plugin attempts restricted access	Sandbox blocks execution	FR-PLUG-05

5.13.5 White-Box Test Cases – Automation

Test Case ID	**Internal Logic Path**	**Expected Behavior**	**Related FRs**
WB-AUTO-01	Execute automation with valid tasks	Tasks executed sequentially	FR-ATPL-01
WB-AUTO-02	Automation task failure	Execution halted, failure recorded	FR-AUTO-11
WB-AUTO-03	Plugin-based automation task	Plugin executor invoked correctly	FR-ATPL-03
WB-AUTO-04	Concurrent automation runs	Runs isolated, no shared state corruption	FR-AUTO-11

5.13.6 Logging and Audit Verification

White-box testing confirmed that all security-sensitive and execution-critical operations generate appropriate logs. The following behaviors were verified:

Authentication and impersonation events are recorded in audit logs

Automation failures produce traceable execution records

Plugin execution errors are captured without exposing system internals

This ensures operational transparency and post-execution traceability.

5.13.7 SQA Summary

Implementation-level SQA activities validate that the internal logic of the WHMS backend correctly enforces security, isolation, and execution guarantees defined in earlier phases. By applying white-box testing to critical components, implementation defects are detected early, ensuring system reliability and consistency with the approved design and requirements.

5.13 SUMMARY OF IMPLEMENTATION

This chapter detailed the implementation of the proposed system, highlighting how the modular, API-first architecture was realized in practice. The system successfully integrates authentication, plugins, automation, and backup management while maintaining extensibility, security, and clear separation of concerns.

Chapter 6

Testing & Evaluation

Chapter 6

Testing & Evaluation

6.1 Auth Module

The Auth Module test cases primarily follow **Black-Box Testing**, where APIs are tested through their exposed interfaces without relying on internal implementation details. This aligns with an **API-first architecture**, ensuring correctness from a consumer's perspective.

Certain areas such as **MFA flows, impersonation, and admin-restricted routes** apply **Grey-Box Testing**, as they require partial knowledge of internal system states (e.g., MFA enabled, admin role assigned) while still validating behavior strictly through API responses.

Auth Module API Test Cases (Tabular Summary)

Test Area	**Example Test IDs**	**Interface Visibility**	**Testing Type**	**Justification**
User Registration	AUTH-001 → AUTH-006	API only	Black-Box Testing	Tests validate input/output behavior without internal logic knowledge
Login & Authentication	AUTH-007 → AUTH-013	API only	Black-Box Testing	Focus on credentials, tokens, status codes, rate limits
Token Refresh & Logout	AUTH-014 → AUTH-019	API only	Black-Box Testing	Ensures contract correctness and session invalidation

Profile Fetch (/me)	AUTH-020 → AUTH-024	API only	Black-Box Testing	Validates authorization handling and token parsing
Email Verification	EMAIL-001 → EMAIL-013	API only	Black-Box Testing	Prevents user enumeration and validates token lifecycle
Password Reset	PASS-001 → PASS-013	API only	Black-Box Testing	Security-focused validation of reset workflow
MFA Setup & Verification	MFA-001 → MFA-018	API only	Black-Box + Grey-Box	API-level tests with awareness of MFA state transitions
Session Management	SESS-001 → SESS-013	API only	Black-Box Testing	Validates session lifecycle and access control
Impersonation	IMP-001 → IMP-012	API only	Grey-Box Testing	Requires knowledge of role constraints and system state

Auth Module Test Cases

Test ID	**Endpoint**	**Method**	**Scenario**	**Input**	**Expected Status**	**Expected Response**	**Preconditions**	**Notes**
AUTH-	/register	POST	Valid registr	email + strong	201	User created,	None	Rate limit applied

001			ation	password		emailVerified=false		
AUTH-002	/register	POST	Duplicate email	Existing email	409	Email already registered	User exists	Conflict handling
AUTH-003	/register	POST	Weak password	Short password	400	Validation error	None	Password policy
AUTH-004	/register	POST	Missing email	No email	400	email required	None	Required field
AUTH-005	/register	POST	Invalid email format	invalid email	400	Invalid email format	None	Format validation
AUTH-006	/register	POST	Rate limit exceeded	>3 requests	429	Too many attempts	None	Abuse prevention
AUTH-007	/login	POST	Valid credentials	Correct email/password	200	Tokens issued	User verified	Cookies set
AUTH-008	/login	POST	Unverified email	Unverified user	403	Email not verified	User exists	Access blocked
AUTH-009	/login	POST	Wrong password	Invalid password	401	Invalid credentials	User exists	No disclosure
AUTH-010	/login	POST	Non-existent user	Unknown email	401	Invalid credentials	None	Enumeration safe
AU	/login	POS	MFA	Valid	202	MFA	MFA	Partial

TH-011		T	enabled user	creds		required	enabled	login
AUTH-012	/login	POST	Disabled account	Disabled user	403	Account disabled	User disabled	Support message
AUTH-013	/login	POST	Rate limit exceeded	>5 attempts	429	Too many attempts	None	Brute force guard
AUTH-014	/refresh	POST	Valid refresh token	Valid token	200	New tokens	Token exists	Rotation
AUTH-015	/refresh	POST	Expired token	Expired token	401	Token expired	Token expired	Re-login
AUTH-016	/refresh	POST	Invalid token	Invalid format	401	Invalid token	None	Token parsing
AUTH-017	/refresh	POST	Token via cookie	Cookie token	200	Tokens issued	Cookie exists	Multi-source
AUTH-018	/logout	POST	Valid logout	Valid token	200	Logged out	Session active	Token revoked
AUTH-019	/logout	POST	Already logged out	Invalid token	401	Invalid token	None	Idempotent
AUTH-	/me	GET	Valid token	Bearer token	200	User profile	Authenticated	Roles returned

020								
AUTH-021	/me	GET	No token	None	401	Not authenticated	None	Auth required
AUTH-022	/me	GET	Expired token	Expired token	401	Token expired	None	Refresh needed
AUTH-023	/me	GET	Invalid token	Invalid token	401	Invalid token	None	Malformed
AUTH-024	/me	GET	Malformed header	Wrong format	401	Invalid header	None	Header validation

Email Verification Test Cases

Test ID	**Endpoint**	**Method**	**Scenario**	**Expected Status**	**Notes**
EMAIL-001	/email/send	POST	Send verification	200	Public endpoint
EMAIL-002	/email/send	POST	Invalid email	400	Format validation
EMAIL-003	/email/send	POST	Missing origin	400	Required param
EMAIL-004	/email/send	POST	Rate limit exceeded	429	Abuse control
EMAIL-005	/email/send	POST	Non-existent email	200	No enumeration
EMAIL-006	/email/verify	GET	Valid token	200	Email verified
EMAIL-	/email/verify	GET	Expired token	400	Re-send

007					needed
EMAIL-008	/email/verify	GET	Already verified	400	Idempotency
EMAIL-009	/email/verify	GET	Invalid token	400	Validation
EMAIL-010	/email/verify	GET	Missing token	400	Required
EMAIL-011	/email/send-verification	POST	Authenticated resend	200	Logged-in user
EMAIL-012	/email/send-verification	POST	No auth	401	Auth required
EMAIL-013	/email/send-verification	POST	Already verified	200	Allowed

Password Reset Test Cases

Test ID	**Endpoint**	**Method**	**Scenario**	**Expected Status**	**Notes**
PASS-001	/password/request-reset	POST	Valid email	200	Email sent
PASS-002	/password/request-reset	POST	Non-existent email	200	No enumeration
PASS-003	/password/request-reset	POST	Invalid email	400	Validation
PASS-004	/password/request-reset	POST	Missing origin	400	Required
PASS-005	/password/reset	GET	Valid token	302	Redirect
PASS-006	/password/reset	GET	Expired token	400	Re-send
PASS-	/password/reset	GET	Invalid token	400	Validation

007					
PASS-008	/password/reset	GET	Used token	400	One-time use
PASS-009	/password/reset	POST	Valid password	200	Updated
PASS-010	/password/reset	POST	Weak password	400	Policy
PASS-011	/password/reset	POST	Same password	400	Security
PASS-012	/password/reset	POST	Expired token	400	Invalid
PASS-013	/password/reset	POST	Missing password	400	Required

MFA Test Cases

Test ID	**Endpoint**	**Method**	**Scenario**	**Expected Status**
MFA-001	/mfa/setup	POST	Setup MFA	200
MFA-002	/mfa/setup	POST	No auth	401
MFA-003	/mfa/setup	POST	Already enabled	409
MFA-004	/mfa/verify	POST	Valid TOTP	200
MFA-005	/mfa/verify	POST	Invalid code	400
MFA-006	/mfa/verify	POST	Expired code	400
MFA-007	/mfa/verify	POST	Missing code	400
MFA-008	/mfa/disable	POST	Disable MFA	200
MFA-009	/mfa/disable	POST	Invalid code	400
MFA-010	/mfa/disable	POST	MFA not enabled	409
MFA-011	/mfa/backup-codes	POST	Generate codes	200
MFA-012	/mfa/backup-codes	POST	Regenerate	200
MFA-013	/mfa/backup-codes	POST	No auth	401
MFA-014	/mfa/verify-login	POST	Valid TOTP	200

MFA-015	/mfa/verify-login	POST	Backup code	200
MFA-016	/mfa/verify-login	POST	Invalid code	400
MFA-017	/mfa/verify-login	POST	Used code	400
MFA-018	/mfa/verify-login	POST	Invalid user	404

Session Management Test Cases

Test ID	**Endpoint**	**Method**	**Scenario**	**Expected Status**
SESS-001	/session	GET	List sessions	200
SESS-002	/session	GET	No sessions	200
SESS-003	/session	GET	No auth	401
SESS-004	/session/:id	DELETE	Revoke session	200
SESS-005	/session/:id	DELETE	Invalid session	404
SESS-006	/session/:id	DELETE	Other user session	403
SESS-007	/session/others/all	DELETE	Revoke others	200
SESS-008	/session/others/all	DELETE	No others	200
SESS-009	/session/security/logs	GET	View logs	200
SESS-010	/session/security/logs	GET	No logs	200
SESS-011	/session/current	GET	Normal session	200
SESS-	/session/current	GET	Impersonation	200

012				
SESS-013	/session/current	GET	No auth	401

Impersonation Test Cases

Test ID	**Endpoint**	**Method**	**Scenario**	**Expected Status**
IMP-001	/impersonate/start	POST	Valid admin	200
IMP-002	/impersonate/start	POST	Non-admin	403
IMP-003	/impersonate/start	POST	Invalid target	404
IMP-004	/impersonate/start	POST	No reason	200
IMP-005	/impersonate/stop	POST	Stop session	200
IMP-006	/impersonate/stop	POST	Invalid session	404
IMP-007	/impersonate/stop	POST	Not impersonating	400
IMP-008	/impersonate/list	GET	List sessions	200
IMP-009	/impersonate/list	GET	Non-admin	403
IMP-010	/impersonate/list	GET	None active	200
IMP-	/impersonation-status	GET	Not	200

011			impersonating	
IMP-012	/impersonation-status	GET	Impersonating	200

Admin User Test Cases

Test ID	Endpoint	Method	Scenario	Expected Status
ADMIN-001	/admin/users	GET	List users	200
ADMIN-002	/admin/users	GET	Search	200
ADMIN-003	/admin/users	GET	Filter role	200
ADMIN-004	/admin/users	GET	Filter status	200
ADMIN-005	/admin/users	GET	Pagination	200
ADMIN-006	/admin/users	GET	Non-admin	403
ADMIN-007	/admin/users/:id	GET	User details	200
ADMIN-008	/admin/users/:id	GET	Invalid user	404
ADMIN-009	/admin/users/:id/roles	POST	Update roles	200
ADMIN-010	/admin/users/:id/roles	POST	Add admin	200
ADMIN-011	/admin/users/:id/roles	POST	Empty roles	400
ADMIN-012	/admin/users/:id/roles	POST	Invalid role	400
ADMIN-	/admin/users/:id/deactivate	POST	Deactivate	200

013				
ADMIN-014	/admin/users/:id/deactivate	POST	Already disabled	409
ADMIN-015	/admin/users/:id/activate	POST	Activate	200
ADMIN-016	/admin/users/:id/activate	POST	Already active	409
ADMIN-017	/admin/users/:id/logout	POST	Force logout	200
ADMIN-018	/admin/users/stats	GET	Statistics	200

2’

Test ID	**API Endpoint**	**Method**	**Test Case**	**Request Body**	**Expected Status**	**Expected Behavior**	**FR Reference**
SRV-POS-01	/api/admin/services	POST	Create valid service	{"code":"shared_hosting","name":"Shared Hosting","description": "Linux hosting"}	201	Service created, active=true	FR-SRV-01
SRV-POS-02	/api/admin/services	GET	List all services (admin)	N/A	200	All services returned	FR-SRV-02
SR	/api/admin/servi	GET	Get	N/A	200	Servic	FR-

V-POS-03	ces/:id		service by ID			e with plans returned	SRV-03
SRV-POS-04	/api/admin/services/:id	PUT	Update service	{"name":"Premium Hosting"}	200	Service updated	FR-SRV-04
SRV-POS-05	/api/admin/services/:id	DELETE	Deactivate service	N/A	204	Service set inactive	FR-SRV-05
SRV-POS-06	/api/admin/services/:id/plans	POST	Create plan	{"name":"Basic","summary":"Entry"}	201	Plan created	FR-SRV-06
SRV-POS-07	/api/admin/plans/:id	PUT	Update plan	{"name":"Basic Plus"}	200	Plan updated	FR-SRV-08
SRV-POS-08	/api/admin/plans/:id/pricing	POST	Create pricing	{"cycle":"monthly","price":9.99}	201	Pricing created	FR-SRV-10
SRV-	/api/client/services	GET	List active	N/A	200	Only active	FR-SRV-

POS-09			services			services	09
SRV-POS-10	/api/admin/plans/:id/pricing	POST	Create multiple pricing cycles	{"cycle":"annually","price":99.99}	201	Pricing added	FR-SRV-10
SRV-POS-11	/api/client/services	GET	Hide deactivated services	N/A	200	Inactive hidden	FR-SRV-05

SERVICES MODULE – NEGATIVE TEST CASES

Test ID	**API Endpoint**	**Method**	**Test Case**	**Invalid Input**	**Expected Status**	**Expected Behavior**	**Validation**
SRV-NEG-01	/api/admin/services	POST	Missing code	name only	400	Validation error	Required
SRV-NEG-	/api/admin/services	POST	Missing name	code only	400	Validation error	Required

02							
SRV-NEG-03	/api/admin/services	POST	Missing description	code + name	400	Validation error	Required
SRV-NEG-04	/api/admin/services	POST	Duplicate code	existing code	409	Conflict error	Unique
SRV-NEG-05	/api/admin/services	POST	Short code	"ab"	400	Validation error	Length
SRV-NEG-06	/api/admin/services	POST	Invalid code format	"test-123"	400	Validation error	Pattern
SRV-NEG-07	/api/admin/services/999999	GET	Non-existent service	N/A	404	Not found	Existence
SRV-NEG-08	/api/admin/services/999999	PUT	Update non-existent	N/A	404	Not found	Existence

SRV-NEG-09	/api/admin/services/999999	DELETE	Delete non-existent	N/A	404	Not found	Existence
SRV-NEG-10	/api/admin/services/999999/plans	POST	Plan for invalid service	N/A	404	FK error	FK
SRV-NEG-11	/api/admin/services/:id/plans	POST	Duplicate plan name	Existing plan	409	Conflict	Unique
SRV-NEG-12	/api/admin/plans/999999	PUT	Update non-existent plan	N/A	404	Not found	Existence
SRV-NEG-13	/api/admin/plans/999999/pricing	POST	Pricing invalid plan	N/A	404	FK error	FK
SRV-NEG-14	/api/admin/plans/:id/pricing	POST	Duplicate cycle	monthly twice	409	Conflict	Unique
SR	/api/admin/plans/:id/pric	POST	Invalid	weekl	400	Validat	Enum

V-NEG-15	ing		cycle	y		ion error	
SRV-NEG-16	/api/admin/plans/:id/pricing	POST	Negative price	-10	400	Validation error	Range
SRV-NEG-17	/api/admin/plans/:id/pricing	POST	Zero price	0	400	Validation error	Range
SRV-NEG-18	/api/admin/plans/:id/pricing	POST	Missing cycle	price only	400	Validation error	Required
SRV-NEG-19	/api/admin/plans/:id/pricing	POST	Missing price	cycle only	400	Validation error	Required

ORDERS MODULE – POSITIVE TEST CASES

Test ID	**API Endpoint**	**Method**	**Test Case**	**Expected Status**	**Expected Behavior**	**FR Reference**
OR	/api/orders	POST	Create	201	Order	FR-

D-POS-01			order		pending, snapshot saved	ORD-01
ORD-POS-02	/api/orders	GET	List client orders	200	Client orders returned	FR-ORD-02
ORD-POS-03	/api/orders/:id	GET	Client order details	200	Own order only	FR-ORD-03
ORD-POS-04	/api/admin/orders/:id	GET	Admin order details	200	Any order visible	FR-ORD-03
ORD-POS-05	/api/admin/orders	GET	List all orders	200	All orders returned	FR-ORD-04
ORD-POS-06	/api/admin/orders/:id/activate	POST	Activate order	200	pending → active	FR-ORD-05
ORD-POS-07	/api/orders/:id/renew	POST	Renew order	200	Renewal extended	FR-ORD-06
ORD-POS-08	/api/admin/orders/:id/renew	POST	Admin renew	200	Renewal extended	FR-ORD-06

ORD-POS-09	/api/admin/orders/:id/suspend	POST	Suspend order	200	active → suspended	FR-ORD-07
ORD-POS-10	/api/admin/orders/:id/resume	POST	Resume order	200	suspended → active	FR-ORD-08
ORD-POS-11	/api/admin/orders/:id/terminate	POST	Terminate order	200	Order terminated	FR-ORD-09
ORD-POS-12	/api/orders/:id/cancel	POST	Cancel pending	200	Order cancelled	Client
ORD-POS-13	/api/orders/:id	GET	Snapshot immutability	200	Snapshot unchanged	FR-ORD-14
ORD-POS-14	/api/admin/orders	GET	Audit retention	200	All states retained	FR-ORD-15
ORD-POS-15	Multiple	Mixed	Full lifecycle	200	Valid transitions	FR-ORD-05–09

INTEGRATION TEST CASES (API-FIRST)

Test ID	APIs Involved	Scenario	Expected Result

INT-01	Services → Orders	Full lifecycle	Success
INT-02	Orders + Services	Snapshot immutability	Snapshot unchanged
INT-03	Orders	Multiple orders	Independent snapshots
INT-04	Services → Orders	Deactivated service	Order blocked
INT-05	Orders	State validation	Invalid blocked
INT-06	Orders	Concurrency	Data integrity
INT-07	Orders	Admin vs Client	Access enforced
INT-08	Orders	Audit retention	All visible

TEST EXECUTION PRIORITY (TABLE)

Priority	Test Coverage
P1 – Critical	Order creation, activation, snapshot, audit
P2 – High	CRUD, state transitions
P3 – Medium	Validation & integration
P4 – Low	Edge cases

OVERALL TEST COVERAGE

Module	Positive	Negative	Integration	Total
Services	11	19	4	34
Orders	15	25	4	44
Total	**26**	**44**	**8**	**78**

EDGE CASE TESTS (API-First Automation System)

Test ID	Test Case	Method	Endpoint	Input	Expected Output	Status Code	Notes
ED	Empty	POS	/api/automation/profil	{ "name":	Validat	400	Minimu

GE-001	cron expression	T	es	"Test", "cron": "" }	ion error returned		m length validation
ED GE-002	Very long profile name (300 chars)	POST	/api/automation/profiles	{ "name": "AAAA... (300 chars)", "cron": "0 2 * * *" }	Validation error returned	400	Maximum length enforced
ED GE-003	Special characters & Unicode in name	POST	/api/automation/profiles	{ "name": "Test™® ! @#$%", "cron": "0 2 * * *" }	Profile created successfully	201	Unicode supported
ED GE-004	Null value in optional fields	POST	/api/automation/profiles	{ "name": "Test", "description": null, "cron": "0 2 * * *" }	Profile created successfully	201	Null handling
ED GE-005	Negative pagination offset	GET	/api/automation/audit/logs?offset=-10	None	Offset treated as 0	200	Negative offset handled
ED GE-006	Zero pagination limit	GET	/api/automation/audit/logs?limit=0	None	Default limit applied (50)	200	Zero handled safely
ED GE-	Extremely high	GET	/api/automation/audit/logs?limit=10000	None	Limit capped	200	Max limit

007	pagination limit				to maximum (100)		enforced
EDGE-008	Workflow with 1000 tasks	POST	/api/automation/workflows/validate	Workflow definition with 1000 tasks	Workflow validated successfully	200	High-scale workflow support
EDGE-009	Empty request body	POST	/api/automation/profiles	{}	Validation error returned	400	Required fields enforced
EDGE-010	Extra unknown fields in payload	POST	/api/automation/profiles	{ "name": "Test", "cron": "0 2 * * *", "unknown": "value" }	Profile created, unknown field ignored	201	Strict schema handling
EDGE-011	Rapid repeated enable/disable	POST	/api/automation/profiles/1/enable & /disable	Rapid toggling	Final state consistent	200	Idempotency check
EDGE-012	Execution during profile deletion	POST	/api/automation/run/1	While deleting profile	Execution fails gracefully	409	Race condition handling

1. AUTHENTICATION TESTS

Test ID	Test Case	Method	Endpoint	Input	Expected Output	Status Code	Notes
AUTH-001	Login with valid admin credentials	POST	/api/auth/login	Valid email & password	success=true, JWT token returned	200	Bearer token issued
AUTH-002	Login with invalid credentials	POST	/api/auth/login	Invalid password	success=false, auth error	401	Authentication failure
AUTH-003	Access protected route without token	GET	/api/automation/profiles	No Authorization header	Unauthorized error	401	Token missing
AUTH-004	Access protected route with invalid token	GET	/api/automation/profiles	Bearer invalid_token	Unauthorized error	401	Token validation failed

2. AUTOMATION PROFILE TESTS

Test ID	Test Case	Method	Endpoint	Input	Expected Output	Status	Notes
PROF-001	List profiles (empty)	GET	/api/automation/profiles	None	Empty list returned	200	Initial state
PROF-002	Create valid profile	POST	/api/automation/profiles	Valid name, cron	Profile created	201	Success
PROF-003	Create profile without name	POST	/api/automation/profiles	Missing name	Validation error	400	Required field
PROF-004	Create profile with invalid cron	POST	/api/automation/profiles	Invalid cron	Validation error	400	Cron format
PRO	Creat	POST	/api/automation/profiles	name	Validatio	400	Min

F-005	e profile with empty name			="""	n error		length
PROF-006	Get profile by valid ID	GET	/api/automation/profiles/1	None	Profile returned	200	OK
PROF-007	Get profile by invalid ID	GET	/api/automation/profiles/99999	None	Not found	404	Non-existent
PROF-008	Invalid profile ID format	GET	/api/automation/profiles/abc	None	Validation error	400	ID check
PROF-009	Update profile	PUT	/api/automation/profiles/1	Valid update	Profile updated	200	Success
PROF-010	Update profile	PUT	/api/automation/profiles/1	Invalid cron	Validation error	400	Cron check

	with invalid cron						
PROF-011	Update non-existent profile	PUT	/api/automation/profiles/99999	Valid input	Not found	404	
PROF-012	Enable profile	POST	/api/automation/profiles/1/enable	None	Enabled=true	200	Scheduled
PROF-013	Disable profile	POST	/api/automation/profiles/1/disable	None	Disabled=true	200	Unscheduled
PROF-014	Enable non-existent profile	POST	/api/automation/profiles/99999/enable	None	Not found	404	
PROF-015	Delete profile	DELETE	/api/automation/profiles/1	None	Deleted=true	200	Soft delete
PRO	Delet	DELE	/api/automation/profiles/9	None	Not	404	

F-016	e non-existent profile	TE	9999		found		
PROF-017	List profiles after creation	GET	/api/automation/profiles	None	Profiles present	200	Data retained

3. AUTOMATION TASK TESTS

Test ID	Test Case	Method	Endpoint	Input	Expected Output	Status	Notes
TASK-001	List tasks (empty)	GET	/profiles/1/tasks	None	Empty list	200	Initial
TASK-002	Create valid task	POST	/profiles/1/tasks	Valid action	Task created	201	
TASK-003	Missing actionType	POST	/profiles/1/tasks	actionType missing	Validation error	400	Required
TASK-004	Null actionMeta	POST	/profiles/1/tasks	actionMeta=null	Task created	201	Allowed
TASK-	Task for invalid	POST	/profiles/99999/tasks	Valid input	Not found	404	

005	profile						
TASK-006	Negative order value	POST	/profiles/1/tasks	order=-1	Validation error	400	>=0
TASK-007	Get task by ID	GET	/tasks/1	None	Task returned	200	
TASK-008	Get non-existent task	GET	/tasks/99999	None	Not found	404	
TASK-009	Update task	PUT	/tasks/1	Valid update	Task updated	200	
TASK-010	Invalid actionType	PUT	/tasks/1	Empty actionType	Validation error	400	
TASK-011	Update non-existent task	PUT	/tasks/99999	Valid input	Not found	404	
TASK-012	Delete task	DELETE	/tasks/1	None	Deleted=true	200	
TASK-013	Delete invalid task	DELETE	/tasks/99999	None	Not found	404	
TASK-014	Run task manually	POST	/tasks/1/run	None	Run queued	200	Async
TASK-	Run invalid	POST	/tasks/99999/run	None	Not found	404	

015	task						

4. EDGE CASE TESTS (COMPLETED)

Test ID	Test Case	Method	Endpoint	Input	Expected Output	Status	Notes
EDGE-001	Empty cron	POST	/profiles	cron=""	Validation error	400	
EDGE-002	Very long name	POST	/profiles	300 chars	Validation error	400	Max length
EDGE-003	Unicode characters	POST	/profiles	Special chars	Profile created	201	UTF-8
EDGE-004	Null optional fields	POST	/profiles	description=null	Created	201	
EDGE-005	Negative offset	GET	/audit/logs	offset=-10	Offset=0	200	Safe
EDGE-006	Zero limit	GET	/audit/logs	limit=0	Default limit	200	
EDGE-007	Very high limit	GET	/audit/logs	limit=10000	Max=100	200	
EDGE-008	Workflow with 1000 tasks	POST	/workflows/validate	Large definition	Valid=true	200	Stress

EDGE-009	Rapid enable/disable	POST	/profiles/1/enable	Rapid toggles	Final state consistent	200	Idempotent

Chapter 7:
Conclusion & Outlook

Chapter 7

Conclusion & Outlook

7.1

7.2

7.3

7.4

Reference and Bibliography

- Kotonya, G., & Sommerville, I. (1998). *Requirements Engineering: Processes and Techniques*. Wiley.
- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
- IEEE Computer Society. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK)*.

Appendices

Appendix A:
Heading (20-Point Size, Times New Roman, Bold)

Text in 12-Point Size, Times New Roman, 1.5 Line Spacing.

1.1 First Level heading [14-Point Size, Times New Roman, Bold and left aligned

Text in 12-Point Size, Times New Roman, 1.5 Line Spacing.

1.1.1 Second level heading [12-Point Size, Times New Roman, Bold and left aligned]

Text in 12-Point Size, Times New Roman, 1.5 Line Spacing.

References

- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- IEEE Computer Society. (2018). *IEEE 29148:2018 - Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*.
- Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Development* (3rd ed.). Prentice Hall.
- Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.